

Unterlagen für das obligatorische Fach Informatik

Gymnasium – Einführung in die Programmierung mit Python

Jacques Mock Schindler

5. September 2026

Inhaltsverzeichnis

Welcome	3
Schedule	3
Hardware and Software Requirements	3
Assessment	4
I Anleitungen	5
1 Speicherorganisation	6
1.1 Dateinamen und Pfade	6
1.2 Dateien in der Cloud	7
1.3 Dateistruktur für die Schule	7
2 Arbeitsumgebung (Arbeiten mit Jupyter Notebooks)	8
2.1 Ausgangslage	8
2.2 Installation von Python	8
2.3 Hello World	9
2.4 Arbeitsumgebung	10
2.5 Öffnen bestehender Jupyter Notebooks	14
2.6 Häufige Fehlermeldungen	15
II Einführung in Python	17
3 Problemlösung in der Informatik	18
3.1 Ausgangslage	18
3.2 Beispiel: Tricolore	19
3.3 Beispiel: Österreichische Flagge	20
3.4 Beispiel: Schweizerfahne	21
3.5 Beispiel: Tessiner Wappen	21
3.6 Musterlösungen	21
4 Variablen in Python	24
4.1 Vorbemerkungen: Python als Rechner	24
4.2 Variablen	24
4.3 Funktionen in Python	25
4.4 Datentypen	26
4.5 Funktionen mit Type-Hints	30
4.6 Ausgewählte Musterlösungen	30

5	Wiederholungen in Python (For-Loops)	31
5.1	Anwendungsübung zu Wiederholungen in Python	31
6	Programmverzweigungen (Bedingungen)	40
6.1	Anwendungsübung zu Bedingungen in Python	42
7	Programmieren einfacher Funktionen	45
7.1	Kreisfläche	45
7.2	Quotient	45
7.3	Fakultät	45
7.4	Quadratische Gleichungen (Mitternachtsformel)	46
7.5	Musterlösungen	46
8	Prüfungsvorbereitung	50
8.1	Belegung von Variablen	50
8.2	Funktionen	50
8.3	Musterlösungen	53
9	Datenstrukturen: Listen	55
9.1	Daten in Listen speichern	55
9.2	Index	55
9.3	Überschreiben von Elementen	56
9.4	Schleifen	56
9.5	Länge	57
9.6	Ergänzen und entfernen	57
9.7	Aufgabe: Fibonacci	58
9.8	Aufgabe: zeichne Chomp	58
9.9	Aufgabe: spiele Chomp	59
9.10	Übersicht	60
9.11	Zusatzstoff	60
10	Datenstrukturen: Dictionary	61
10.1	Operationen für Dictionaries	61
11	Beurteilung von Algorithmen	64
11.1	Bubblesort	64
11.2	Mergesort	64
12	Divide and Conquer	66
12.1	Fibonacci-Folge	66
12.2	Endlose Rekursion	67
12.3	Beispiel: Anstossen	67
12.4	Fakultät	70
12.5	Wörter umdrehen	71
12.6	Bonus: Anzahl Möglichkeiten zu Zahlen	71

13 Prüfungsvorbereitung 2	73
13.1 Lernziele	73
13.2 Listen	73
13.3 Musterlösungen	75
 III Data Representation	 78
14 Number Representation	79
14.1 One-to-one Number Representation	79
14.2 Symbolic Number Representation	79
14.3 Number Representation and Positional Notation	80
14.4 Number Conversion	81
 15 Image Representation in Computers	 88
15.1 Colour Representation	88
15.2 Colour Table	90
 16 The Concept of an Algorithm	 91
16.1 Definition	91
16.2 Properties of an Algorithm	91
16.3 Example: Tying Shoe Laces	91
16.4 From Everyday Algorithms to Mathematical Algorithms	92
16.5 The Euclidean Algorithm	92
16.6 Implementing the Euclidean Algorithm in Python	93
16.7 Ways to Describe an Algorithm	94
 17 Algorithm Representation: Pseudocode & Struktogramme	 95
17.1 What Is Pseudocode?	95
17.2 Pseudocode Conventions	95
17.3 Worked Example: GCD in Pseudocode	96
17.4 Exercise 1: Python → Pseudocode	97
17.5 Exercise 2: Pseudocode → Python	97
17.6 Exercise 3: Pseudocode → Python	98
17.7 Structure Diagrams (Struktogramme)	99
17.8 Summary: Comparing Notations	101
 18 Sorting Algorithms	 102
18.1 The Sorting Problem	102
18.2 Pseudocode Conventions (Quick Reference)	102
18.3 Helper: Visualising Each Step	103
18.4 Your Sorting Algorithm	103
18.5 Named Sorting Algorithms	104
18.6 Selection Sort	105
18.7 Bubble Sort	106
18.8 Testing Your Implementations	108
18.9 Counting Comparisons	109

19 Algorithms and Computational Complexity	111
19.1 Selection Sort	111
19.2 Big O Notation	111
19.3 Merge Sort: A More Efficient Approach	112
19.4 Comparison of Growth Rates	112
 IV Functions	 113
20 Functions: Revision and Consolidation	114
20.1 Function: A Definition	114
20.2 Practical Example: The VAT Calculator	115
20.3 Coding Exercise: The BMI Calculator	116
20.4 Coding Exercise: The Cuboid Space Diagonal	119
 21 Functions II: Calculating Triangles	 121
21.1 Functions within functions	121
 22 Classes: Triangle as Object	 128
22.1 Introduction	128
22.2 Enhanced Triangle Class	130
22.3 Coding Task: Circle Class	139
 23 Functions: Exercises	 143
23.1 Area of a trapezium	143
23.2 Volume of a right circular cylinder	143
23.3 Progression decisions	144
 24 Triangulation: Inheriting from Triangle	 146
24.1 The Surveying Problem	147
24.2 Stage 1 — Helper Functions	147
24.3 Stage 2 — Bundling into a Class	151
24.4 Stage 3 — Inheritance: an Auxiliary Triangle <i>is a</i> Triangle	153
24.5 Coding Task	157
 V Object Oriented Programming	 158
25 Interest-Bearing Accounts: Inheriting from Account	159
25.1 The Design at a Glance	159
25.2 Stage 1 — A Class for a Single Entry	160
25.3 Stage 2 — Bundling into an Account	163
25.4 Stage 3 — Inheritance: a Current Account <i>is an</i> Account	164
 Literaturverzeichnis	 170

Welcome

Here you will find information about computer science lessons.

Schedule

Date	Topic
21.08.2026	Introduction and Revision
28.08.2026	National Surveying I
04.09.2026	National Surveying II
11.09.2026	Cyber Risks
18.09.2026	Cyber Self-Defence I
25.09.2026	Cyber Self-Defence II
23.10.2026	Computer Networks I: Layer Models
30.10.2026	Exam 1: National Surveying and Cyber Security
06.11.2026	Computer Networks II: IP and DNS
13.11.2026	Computer Networks III: NAT
20.11.2026	Computer Networks IV: Top-Level Domains
27.11.2026	Computer Networks V: Monitoring Connections
04.12.2026	Python Sockets I
11.12.2026	Python Sockets II
18.12.2026	Data Structures I: Stack and Queue
08.01.2027	Data Structures II: Queues in Python
15.01.2027	Data Structures III: Binary Search Tree
22.01.2027	Exam 2: Computer Networks and Data Structures
29.01.2027	Databases: Introduction
05.02.2027	Basic SQL Queries and End of Semester

The programme reflects the current state of planning. Changes are to be expected throughout the semester.

Hardware and Software Requirements

A laptop is required for the computer science lessons (you will not be able to solve the tasks set in class using an iPad). You will need administrator rights on your laptop to install the required software.

Furthermore, you must ensure that your battery is sufficiently charged at the start of the lesson to last for a double period.

Assessment

Two written tests are planned per semester. In addition, oral participation will be graded. **Asking questions** is specifically considered as part of oral participation.

The final grade is calculated as a weighted average of the two written tests and the grade for oral participation. The average grade of the two written tests is weighted at 90%, while the grade for oral participation is weighted at 10%.

If anyone would like a personal consultation, you can sign up for a meeting here (Rent a Mock).

Teil I

Anleitungen

1 Speicherorganisation

Informationen sind in Computern in Dateien gespeichert. Die gespeicherten Informationen können dabei ganz unterschiedlicher Art, wie zum Beispiel Texte, Bilder oder Videos, sein. Als Modell, wie man sich Dateien vorstellen kann, hat sich das Bild von Dossiers in Ordnern etabliert. Die einzelnen Dossiers sind die Dateien und die Ordner sind die Strukturen, in denen die Dateien abgelegt sind. Diese Konstruktion kann über mehrere Ebenen hinaus verschachtelt werden (Ordner in Gestellen, die wiederum in einzelnen Räumen stehen, etc.).

1.1 Dateinamen und Pfade

Damit Dateien identifiziert und gefunden werden können, müssen sie einen Namen haben. Grundsätzlich gibt es keine Einschränkungen, wie Dateien benannt werden. Die meisten Dateinamen bestehen allerdings aus zwei Teilen: dem eigentlichen Dateinamen und der Dateinamenserweiterung.

Der eigentliche Dateiname wird idealerweise so festgelegt, dass er einen Rückschluss auf den Inhalt der Datei zulässt.

Die Dateinamenserweiterung ist ein Zusatz, der Auskunft über den Dateityp gibt. Sie steht hinter einem Punkt hinter dem eigentlichen Dateinamen. Auf Windows Rechnern wird die Dateinamenserweiterung im Dateimanager in der Standardeinstellung nicht angezeigt. Um dies zu ändern, muss in den Einstellungen des Dateimanagers die Option “Dateinamenserweiterungen anzeigen” aktiviert werden (Ansicht > Anzeigen > Dateinamenserweiterung).

Damit Dateien besser ausgetauscht werden können, empfiehlt es sich, für die Namen lediglich Buchstaben, Zahlen und Unterstriche (sog. [ASCII](#)-Zeichen) zu verwenden.

Damit man Dateien finden kann, muss man wissen, wo sie abgelegt worden sind. Übertragen auf das Modell von Dossiers in Ordnern bedeutet das, zu wissen, welches Dossier in welchem Ordner in welchem Gestell in welchem Raum abgelegt ist. Wie in einem realen Archiv, geht man dabei vom Raum zum Gestell, zum Ordner und schliesslich zum Dossier. In der Informatik wird dieser Weg als Pfad bezeichnet. Auf einem Windows-Rechner beginnt dieser Pfad mit dem sogenannten Laufwerksbuchstaben, gefolgt von einem Doppelpunkt und einem Backslash (\). Aus historischen Gründen ist der Laufwerksbuchstabe auf Windows-Rechnern Standardmässig der Buchstabe C.

Ein Beispiel für einen Pfad könnte so aussehen:

```
1 C:\Users\fritz\Documents\text.docx
```

In diesem Beispiel ist **C:** der Laufwerksbuchstabe, **Users** der Ordner, **fritz** der Unterordner, **Documents** der Unterordner von **fritz** und **text.docx** die Datei, die im Ordner **Documents** abgelegt ist. **Users** ist ein von Windows standardmässig angelegter Ordner, in dem die persönlichen Daten der Benutzer abgelegt werden. Der Ordner **fritz** ist der persönliche Ordner des Benutzers **fritz**. Der Ordner **Documents** wird ebenfalls standardmässig von Windows im Ordner jedes Benutzers angelegt. Dem Benutzer **fritz** steht es frei, diesen Ordner zu verwenden und darin Dateien oder Unterordner anzulegen. Der Dateiname **text.docx** verweist mit seiner Dateinamenserweiterung **.docx** auf eine Datei, die mit dem Programm Microsoft Word erstellt worden ist.

1.2 Dateien in der Cloud

Dateien können nicht nur lokal auf dem Computer gespeichert werden. Damit von überall und jederzeit auf Dateien zugegriffen werden kann, werden Dateien in der *Cloud* gespeichert. Dabei handelt es sich um Server in Rechenzentren, die über das Internet erreichbar sind. Beispiele für solche Cloud-Dienste sind OneDrive von Microsoft oder Google Drive. Ablageorte auf OneDrive werden in Windows direkt in der Verzeichnisstruktur des Betriebssystems angezeigt, solche von Google erhalten auf Windows einen eigenen Laufwerksbuchstaben (**G:**).

Damit auf die Dateien in der Cloud zugegriffen werden kann, ist eine Internetverbindung erforderlich.

1.3 Dateistruktur für die Schule

Für den schulischen Bedarf erscheint es sinnvoll eine Dateistruktur nach Fächern anzulegen. Im Ordner **Documents** wird dazu für jedes Fach ein eigener Unterordner angelegt. Innerhalb der jeweiligen Fachordner kann eine weitere Struktur nach Semester oder nach Thema sinnvoll sein. Ein Beispiel für die Ordnerstruktur eines Erstklässlers an der KBW kann so aussehen:

```
1 Documents\  
2     |—Schule\  
3     |   |—Deutsch  
4     |   |—Franz  
5     |   |—Mathe  
6     |   |—WR  
7     |   |—...  
8     |—Privat\  
9     |   |—Rechnungen  
10    |   |—...
```

2 Arbeitsumgebung (Arbeiten mit Jupyter Notebooks)

2.1 Ausgangslage

In der Informatik geht es darum, wie Informationsverarbeitung mit Hilfe von Computern automatisiert werden kann.

Die Automatisierung der Informationsverarbeitung erfordert die Verwendung von Programmiersprachen. Im Informatikunterricht wird in erster Linie mit der Programmiersprache Python gearbeitet.

Im folgenden findet sich eine Anleitung für die Installation der für den Unterricht erforderlichen Programme.

2.2 Installation von Python

Dieser Abschnitt führt Sie Schritt für Schritt durch die Installation von Python auf einem Windows-Rechner.

Microsoft Store Falle

Achten Sie beim Herunterladen von Python darauf, dass Sie sich auf der offiziellen Seite von Python (<https://www.python.org>) und **nicht** im Microsoft Store befinden. Wenn Sie versehentlich die Python Version aus dem Microsoft Store installiert haben, kann das bei der Arbeit an den Schulprojekten zu Problemen führen.

Deinstallieren Sie die Microsoft Version von Python und installieren Sie die Version von der offiziellen Website.

1. Laden Sie die neueste Version von Python von der offiziellen Website herunter: [python.org](https://www.python.org).
2. Führen Sie das heruntergeladene Installationsprogramm durch Doppelklick auf die Datei aus. Stellen Sie sicher, dass Sie die Option “Add Python to PATH” aktivieren, bevor Sie auf “Install Now” klicken.

💡 Die 'PATH'-Umgebungsvariable

Stellen Sie sich die PATH-Variable wie ein Adressbuch für die Kommandozeile (Terminal) vor. Wenn Sie einen Befehl wie **python** eingeben, schaut der Computer in diesem Adressbuch nach, wo das entsprechende Programm zu finden ist.

Indem Sie das Häkchen bei "Add Python to PATH" setzen, fügen Sie die Adresse des Python-Interpreters zu diesem Adressbuch hinzu. Ohne diesen Eintrag weiss der Computer nicht, wo er suchen soll, und meldet, dass er den Befehl nicht kennt.

3. Überprüfen Sie die Installation, indem Sie die Eingabeaufforderung öffnen (Terminal → Windows-Taste + R, dann **cmd** eingeben) und den Befehl **python --version** eingeben. Dies sollte die installierte Python-Version anzeigen.

2.3 Hello World

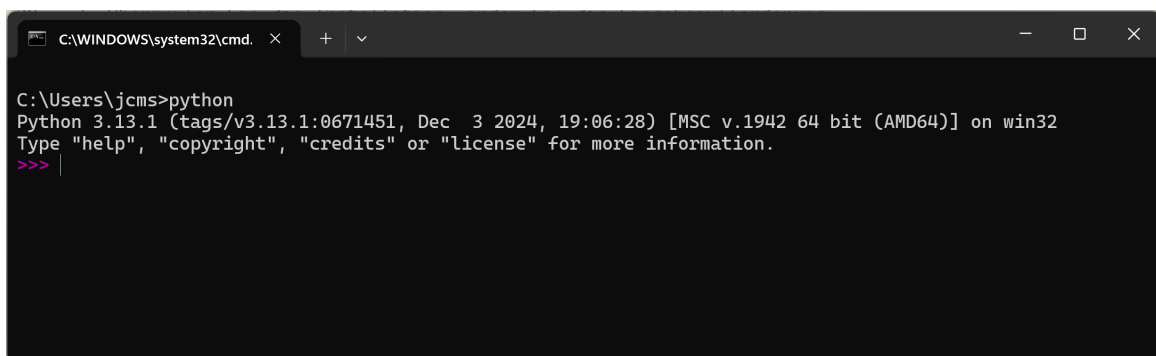
Es hat sich eingebürgert, dass das erste Programm, das ausgeführt wird, ein Programm ist, das den Text "Hello World" auf dem Bildschirm ausgibt. Um dieser Tradition zu folgen, führen Sie die folgenden Schritte aus:

1. Öffnen Sie ein Terminal (Windows-Taste + R, dann **cmd** eingeben).

💡 Das Terminal

Unter dem Begriff "Terminal" versteht man ein Programm, das eine textbasierte Benutzeroberfläche bereitstellt, um mit dem Betriebssystem zu interagieren. In einem Terminal können Sie Befehle eingeben und erhalten die Ausgaben direkt im Fenster.

2. Geben Sie den Befehl **python** ein, um die Python-Shell zu starten. Die Python Shell sollte ungefähr so, wie das folgende Bild aussehen.



```
C:\WINDOWS\system32\cmd. x + - □ ×
C:\Users\jcms>python
Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

Abbildung 2.1: Python Shell

3. Geben Sie den folgenden Befehl ein und drücken Sie anschliessend die Eingabetaste:

```
1 print("Hello World")
```

Das Resultat sollte wie das folgende Bild aussehen.

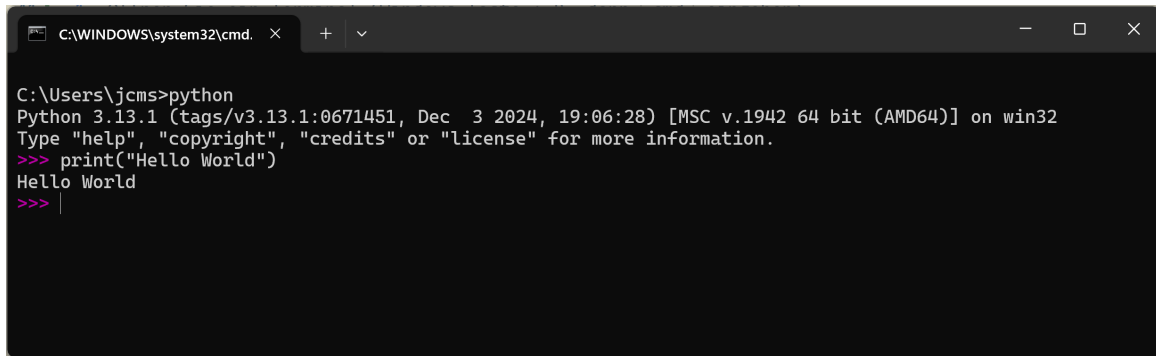


Abbildung 2.2: Python Shell

Gratuliere - Sie haben Ihr erstes Python-Programm erfolgreich ausgeführt!

2.4 Arbeitsumgebung

Im Unterricht wird nicht direkt in der Python-Shell gearbeitet, sondern mit sogenannten Jupyter Notebooks. Jupyter Notebooks ermöglichen es, in der gleichen Datei sowohl Code (Programm Teile) als auch formatierten Text (in Markdown) zu verarbeiten. Eine Jupyter Notebook Datei hat die Endung `.ipynb`. Vom Jupyter Notebook unterschieden werden muss die Arbeitsoberfläche in welcher die Jupyter Notebooks bearbeitet werden. Diese Oberfläche nennt sich JupyterLab und läuft in einem Webbrowser.

💡 Das Jupyter Ökosystem

Die im Unterricht verwendeten Jupyter Notebooks sind Teil eines ganzen Jupyter Ökosystems. Der Name Jupyter setzt sich aus den drei Programmiersprachen **J**ulia, **P**ython und **R** zusammen, die in diesem Ökosystem eine zentrale Rolle spielen. Zum Jupyter Ökosystem gehören auch zahlreiche Erweiterungen und Tools, die die Arbeit mit Notebooks und Daten erleichtern.

Der Unterricht beschränkt sich auf die Verwendung von Jupyter Notebooks mit der Programmiersprache Python sowie den Einsatz von JupyterLab als Arbeitsumgebung.

Damit dies alles funktioniert, braucht es ein paar weitere Vorbereitungsarbeiten.

1. Erstellen Sie im Ordner “Informatik” einen Unterordner mit dem Heutigen Datum als Namen. Formatieren Sie das Datum nach dem Schema “YYMMDD”, für den 1. August 2025 wäre das zum Beispiel “250801”.
2. Öffnen Sie den soeben erstellten Ordner.
3. Geben Sie die Tastenfolge **Ctrl + L** ein, um die Adresszeile des Dateimanagers zu aktivieren.
4. Überschreiben Sie den Inhalt der Adresszeile mit dem Text **cmd** und drücken Sie die Eingabetaste. Dadurch wird ein Terminal geöffnet, das direkt im aktuellen Ordner arbeitet.
5. Geben Sie im neu geöffneten Terminal den folgenden Befehl ein und drücken Sie die Eingabetaste:

```
1 python -m venv .venv
```

Dadurch wird eine sogenannte Python Virtual Environment erstellt (venv wegen **Virtual Environment**). Dieses Python Virtual Environment dient dazu, die eigenen Programmierprojekte unabhängig voneinander gestalten zu können.

6. Aktivieren Sie das Python Virtual Environment mit dem folgenden Befehl:

```
1 .venv\Scripts\activate
```

Ihr Terminal sieht nach dem Erstellen und Aktivieren der Python Virtual Environment ungefähr so aus:

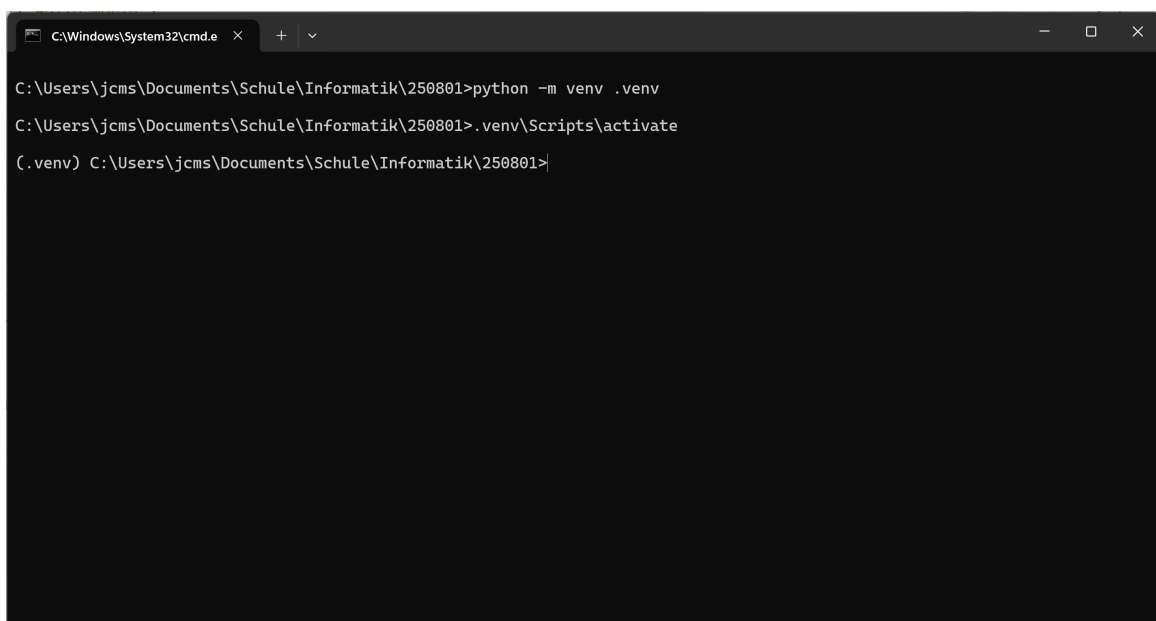


Abbildung 2.3: Aktivierte Python Virtual Environment

Das Wort in der Klammer am Anfang der Zeile zeigt den Namen der aktiven Python Virtual Environment an. Im vorliegenden Fall ist das .venv.

7. In der nun aktivierten Python Virtual Environment installieren Sie die benötigten Pakete mit dem folgenden Befehl:

```
1 pip install jupyter
```

Das dauert eine Weile.

Während der Installation werden die benötigten Pakete (Ergänzungen zur bestehenden Python Installation) heruntergeladen und in der Python Virtual Environment gespeichert. Das Terminal sieht dabei ungefähr so aus:

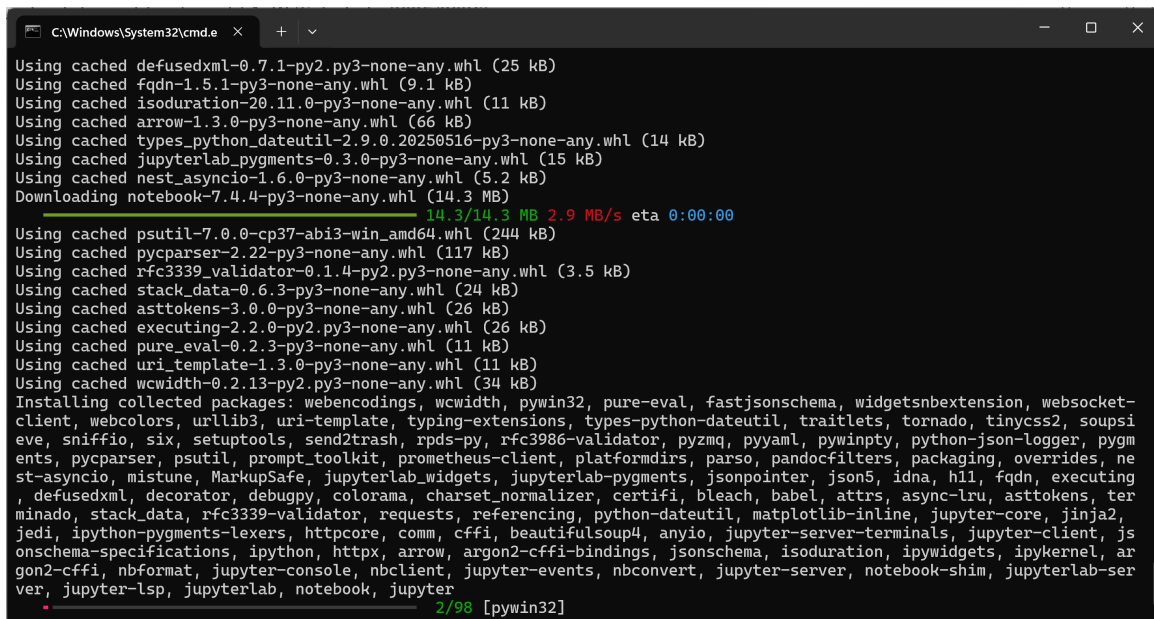


Abbildung 2.4: Terminal während der Jupyter Installation

Alle in einer Python Virtual Environment installierten Pakete sind innerhalb dieser Umgebung dauerhaft verfügbar und müssen daher für das gleiche Projekt kein zweites Mal installiert werden.

8. Starten Sie den Jupyter Server mit dem folgenden Befehl:

```
1 jupyter-lab
```

Dies startet den Jupyter Notebook Server und öffnet automatisch ein Browserfenster mit der Jupyter Notebook Oberfläche.

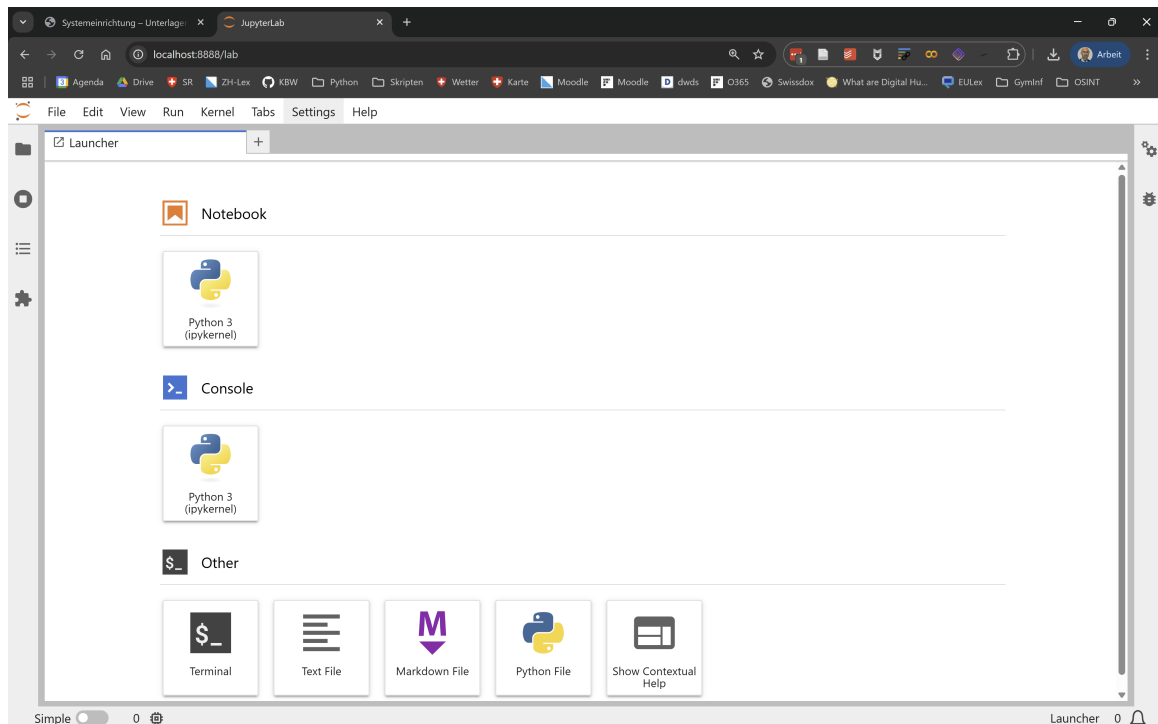


Abbildung 2.5: Startseite Jupyter Lab

9. Klicken Sie auf den Button “Python 3 (ipykernel)” unter dem Titel Notebook.

Damit starten Sie ein neues Jupyter Notebook. Der Cursor blinkt in einer leeren Zelle. Bei dieser Zelle handelt es sich um eine sogenannte Code-Zelle. In einer Code-Zelle können Sie Python Code eingeben und ausführen.

Überprüfen Sie das, indem Sie in der Zelle den Befehl `print("Hello World")` eingeben und anschliessend die Tastenfolge **Shift + Enter** drücken (alternativ können Sie auch auf den Button “Run” in der Werkzeugleiste klicken).

Das Resultat sollte wie das folgende Bild aussehen.

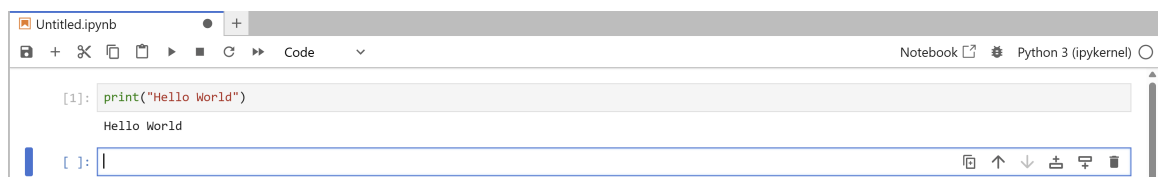


Abbildung 2.6: Hello World in einem Jupyter Notebook

In einem Jupyter Notebook können Sie nicht nur Python Code ausführen, sondern auch Text (formatiert in Markdown) darstellen.

Für die Darstellung von Text müssen Sie die Zelle als Text-Zelle markieren. Dazu klicken Sie auf den Button “Code” in der Werkzeugleiste und wählen im Dropdown-Menü die Option “Markdown” aus.

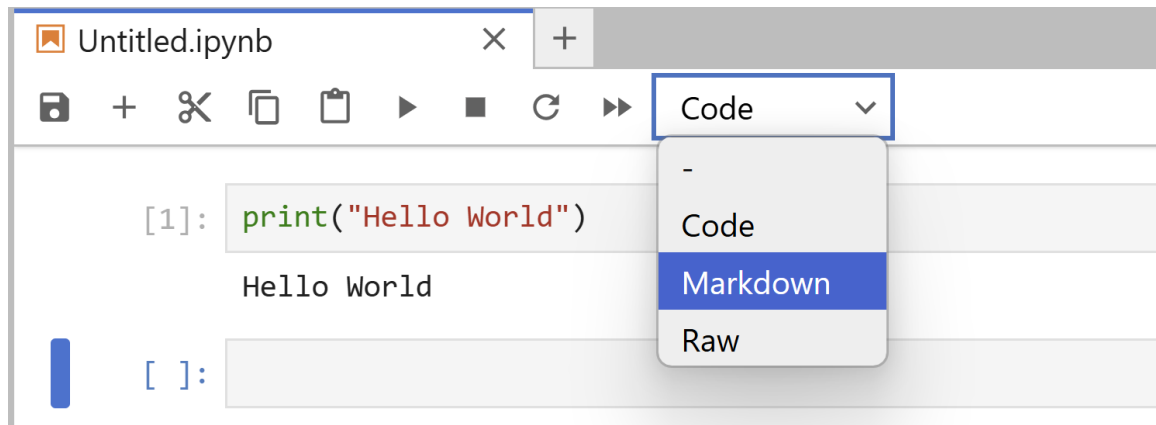


Abbildung 2.7: Umstellen der Zelle auf Markdown

Probieren Sie das aus. Schreiben Sie einen Titel und einen kurzen Text in die Zelle unterhalb der Code-Zelle mit `print("Hello World")`. Damit der Text in der Zelle formatiert angezeigt wird, müssen Sie die Zelle mit der Tastenfolge **Shift + Enter** ausführen (analog zum Ausführen von Code-Zellen).

Eine Zelle ist entweder eine Code-Zelle oder eine Text-Zelle. Für den Wechsel zwischen Code- und Text-Darstellung müssen Sie je eine neue Zelle anlegen. Das geht mit der Tastenfolge **Esc + B** (für “Below”) oder **Esc + A** (für “Above”) während Sie sich in einer Zelle befinden. Alternativ können Sie auch die Buttons “Insert Cell Below” oder “Insert Cell Above” aus den Werkzeugen der Zelle verwenden.

10. Das Jupyter Notebook ist eine Datei mit der Endung `.ipynb`. Neu erstellte Jupyter Notebooks erhalten den Namen “Untitled.ipynb”. Um diesen Namen zu ändern, klicken Sie mit der rechten Maustaste auf den Titel “Untitled” in der oberen linken Ecke des Jupyter Notebooks und wählen Sie die Option “Rename” aus dem Kontextmenü. Anschliessend können Sie den Namen des Jupyter Notebooks eingeben.

2.5 Öffnen bestehender Jupyter Notebooks

Häufiger als das Erstellen eines neuen Jupyter Notebooks ist das Öffnen eines bereits bestehenden Jupyter Notebooks. Hier wird das entsprechende Vorgehen beschrieben.

1. Navigieren Sie in den Ordner in dem sich das Jupyter Notebook befindet.
2. Stellen Sie sicher, dass der Ordner über eine Python Virtual Environment mit installierten Jupyter Paketen verfügt.

Öffnen Sie dazu im ausgewählten Ordner das Terminal (**ctrl + L** anschliessend **cmd** und Eingabetaste). Dann starten Sie die Python Virtual Environment und geben den Befehl `pip list` ein. Dieser Listet alle in der Python Virtual Environment installierten Pakete auf. Falls die Jupyter Pakete nicht aufgelistet werden, müssen Sie diese wie [oben](#) beschrieben installieren.

3. Starten Sie den Jupyter Server mit dem Befehl `jupyter-lab`.
4. Öffnen Sie das Dateiverzeichnis. Dazu müssen Sie auf dem linken Rand das Ordner-Symbol anklicken.
5. Wählen Sie das Jupyter Notebook aus, das Sie öffnen möchten. Mit einem Doppelklick auf das Jupyter Notebook wird dieses geöffnet.

Wenn Sie das Dateiverzeichnis wieder schliessen möchten, klicken Sie auf das Ordner-Symbol auf der linken Seite erneut.

Diese Schritte funktionieren auch, wenn Sie ein Jupyter Notebook öffnen möchten, das Sie von jemand anderem erhalten haben. Sie müssen dieses dazu lediglich in den Ordner kopieren, in dem sich die Python Virtual Environment mit den installierten Jupyter Paketen befindet.

2.6 Häufige Fehlermeldungen

Problem / Fehlermeldung (Was Sie sehen)	Mögliche Ursache (Warum es passiert)	Lösung (Was Sie tun können)
Der Befehl <code>python</code> ist entweder falsch geschrieben oder konnte nicht gefunden werden.	Python wurde bei der Installation nicht zur PATH-Variable hinzugefügt. Der Computer weiss nicht, wo er <code>python.exe</code> finden soll.	1. Deinstallieren Sie Python über die Systemsteuerung. 2. Installieren Sie Python erneut. 3. Achten Sie diesmal unbedingt darauf, das Häkchen bei "Add Python to PATH" zu setzen.
Der Befehl <code>jupyter-lab</code> ist entweder falsch geschrieben oder konnte nicht gefunden werden.	Sie haben vergessen, die virtuelle Umgebung zu aktivieren. Der Befehl <code>jupyter-lab</code> existiert nur innerhalb der aktivierten Umgebung.	1. Überprüfen Sie, ob (<code>.venv</code>) am Anfang der Kommandozeile steht. 2. Falls nicht, führen Sie den Aktivierungsbefehl erneut aus: <code>.venv\Scripts\activate</code>.

Problem / Fehlermeldung (Was Sie sehen)	Mögliche Ursache (Warum es passiert)	Lösung (Was Sie tun können)
ImportError: DLL load failed... oder ähnliche Fehler unter Windows	Ein häufiges Problem mit der Installation von pywin32 , einer wichtigen Windows-Bibliothek, die von Jupyter benötigt wird.	1. Stellen Sie sicher, dass Ihre virtuelle Umgebung aktiv ist. 2. Führen Sie den Befehl pip install --upgrade pywin32 aus, um die Bibliothek zu reparieren.
Kernel Error oder der Status "Kernel starting, please wait..." ändert sich nicht	Die Verbindung zwischen der Browser-Oberfläche und dem Python-"Gehirn" (dem Kernel) ist gestört. Dies kann viele Ursachen haben.	1. Der einfachste erste Schritt: Klicken Sie im JupyterLab-Menü auf "Kernel" -> "Restart Kernel...". 2. Wenn das nicht hilft, schliessen Sie JupyterLab im Terminal (mit der Tastenkombination Strg + C) und starten Sie es mit jupyter-lab neu.
Permission denied (Zugriff verweigert) bei der Installation von Paketen	Sie versuchen, Pakete an einem systemweiten Ort zu installieren (z. B. in C:\Program Files), für den Sie keine Schreibrechte haben.	Dies ist genau das Problem, das virtuelle Umgebungen lösen! Stellen Sie sicher, dass Ihre venv aktiv ist (.venv muss sichtbar sein). Dadurch wird sichergestellt, dass alle Pakete lokal in Ihren Projektordner installiert werden, wo Sie die vollen Rechte haben.

Teil II

Einführung in Python

3 Problemlösung in der Informatik

3.1 Ausgangslage

Jede Disziplin hat ihre eigene Art, Probleme zu lösen. Das ist in der Informatik nicht anders. In der Informatik versucht man, grosse Probleme in kleinere Teilprobleme zu zerlegen. Das macht man so lange, bis die Teilprobleme so klein sind, dass sie einfach zu lösen sind.

Dies soll hier anhand von verschiedenen Grafiken gezeigt werden.

Damit in Python einfach mit Grafiken gearbeitet werden kann, wird das Paket **PyTamaro** verwendet. Dieses Paket wurde von der Università della Svizzera italiana (USI) extra für die Informatik-Ausbildung entwickelt.

Damit das Paket verwendet werden kann, muss es zuerst in der aktuellen Python Virtual Environment installiert werden.

Dazu öffnen Sie ein Terminal im Ordner, in dem sich dieses Jupyter Notebook befindet. Anschliessend starten Sie die Python Virtual Environment mit dem Befehl:

```
1 .venv\Scripts\activate
```

Danach können Sie **PyTamaro** mit dem Befehl

```
1 pip install pytamaro
```

installieren.

Um das Paket zu verwenden, muss es *importiert* werden. Die genauen Zusammenhänge müssen im Moment nicht bekannt sein. Wichtig ist lediglich, dass die folgende Zelle ausgeführt wird.

```
1 from pytamaro.de import (  
2     rechteck, kreis_sektor,  
3     blau, rot, weiss, schwarz,  
4     neben, ueber, ueberlagere,  
5     drehe, kombiniere,  
6     zeige_grafik, speichere_grafik,  
7 )
```

3.2 Beispiel: Tricolore

Die Vorgehensweise wird anhand der Französischen Nationalflagge (Tricolore) gezeigt.

Um die Zeichnung der Tricolore zu planen, wird die Grafik in ihre Einzelteile zerlegt. Die Tricolore besteht aus drei gleich grossen Rechtecken in den Farben blau, rot und weiss. Diese Rechtecke werden nebeneinander angeordnet.

Das bedeutet, dass die Länge und die Breite der Rechtecke definiert werden muss und basierend auf diesen Werten die drei Rechtecke gezeichnet werden. Anschliessend werden die drei Rechtecke nebeneinander angeordnet.

Der Befehl zum Zeichnen eines Rechtecks lautet

```
1 name = rechteck(länge, breite, farbe)
```

Bevor die Zeichnung tatsächlich erstellt wird, soll hier der Befehl im Detail erklärt werden:

- **name** ist der Name, unter dem das Rechteck gespeichert wird. Dieser Name kann später verwendet werden, um auf das Rechteck zuzugreifen.
- **rechteck** ist der Befehl, der ein Rechteck zeichnet. In der Klammer hinter dem Befehl werden die sogenannten *Argumente* angegeben. Diese steuern, wie das Rechteck aussieht.
- **länge** und **breite** sind die Argumente, die die Grösse des Rechtecks bestimmen. Diese Werte können beliebig gewählt werden.
- **farbe** ist das Argument, das die Farbe des Rechtecks bestimmt. Aufgrund der Eigenschaften von **PyTamaro** können ausschliesslich die Farben verwendet werden, welche importiert worden sind.

3.2.1 Rechtecke zeichnen

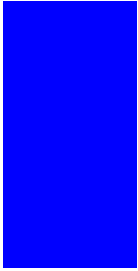
Als erstes wird hier gezeigt, wie das blaue Rechteck gezeichnet wird.

Damit das Resultat kontrolliert werden kann, wird die Grafik mit dem Befehl

```
1 zeige_grafik(name)
```

angezeigt.

```
1 bleu = rechteck(50, 100, blau)
2 zeige_grafik(bleu)
```



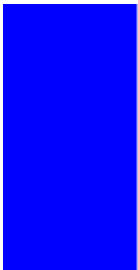
Nachdem das blaue Rechteck gezeichnet wurde, kann das weisse und das rote Rechteck analog gezeichnet und angezeigt werden.

```
1  # TODO: Rechtecke blanc und rouge zeichnen -> Schreiben Sie hier Ihren  
2  # Code  
3  blanc = rechteck(50, 100, weiss)  
4  rouge = rechteck(50, 100, rot)
```

Als nächstes werden die drei Rechtecke nebeneinander angeordnet. Dazu wird der Befehl **neben** verwendet. Dieser Befehl nimmt zwei Argumente entgegen: das erste Rechteck und das zweite Rechteck. Das erste Rechteck wird links vom zweiten Rechteck gezeichnet.

```
1  resultat = neben(linke grafik, rechte grafik)
```

```
1  zwei_drittel = neben(bleu, blanc)  
2  zeige_grafik(zwei_drittel)
```



Analog können Sie nun das rote Rechteck rechts der zwei Drittel anordnen. Nennen Sie das Resultat **tricolore** und zeigen Sie es an.

```
1  # TODO: Tricolore zusammenfügen -> Schreiben Sie hier Ihren  
2  # Code
```

3.3 Beispiel: Österreichische Flagge

Zeichnen Sie die Österreichische Flagge. Das Seitenverhältnis der Flagge ist 2:3.

Um Elemente übereinander anzuordnen, wird der Befehl `ueber` verwendet. Die Syntax dieses Befehls lautet:

```
1 resultat = ueber(obere grafik, untere grafik)
```

```
1 # TODO: Östereichische Flagge zeichnen -> Schreiben Sie hier Ihren  
2 # Code
```

3.4 Beispiel: Schweizerfahne

Zeichnen Sie eine korrekt proportionierte Schweizerfahne. Die Dimensionen können Sie der folgenden Grafik entnehmen:

Verwenden Sie dazu die Befehle `rechteck`, `drehe` und `ueberlagere`.

```
1 # TODO: Schweizerfahne zeichnen -> Schreiben Sie hier Ihren  
2 # Code
```

3.5 Beispiel: Tessiner Wappen

Als Referenz an die USI zeichnen Sie als letztes Beispiel das Tessiner Wappen.

Verwenden Sie dazu neben den bereits bekannten Befehlen zusätzlich die Befehle `ueber` und `kreis_sektor`.

```
1 # TODO: Tessiner Wappen zeichnen -> Schreiben Sie hier Ihren  
2 # Code
```

3.6 Musterlösungen

3.6.1 Trikolore

```
1 bleu = rechteck(50, 100, blau)  
2 blanc = rechteck(50, 100, weiss)  
3 rouge = rechteck(50, 100, rot)  
4  
5 deux_tiers = neben(bleu, blanc)  
6 tricolore = neben(deux_tiers, rouge)
```



```
7
8  zeige_grafik(tricolore)
```

3.6.2 Österreichische Flagge

```
1  roter_balken = rechteck(90, 20, rot)
2  weisser_balken = rechteck(90, 20, weiss)
3
4  oberer_teil = ueber(roter_balken, weisser_balken)
5  osterreichische_flagge = ueber(oberer_teil, roter_balken)
6
7  zeige_grafik(osterreichische_flagge)
```

3.6.3 Schweizerfahne

```
1  hintergrund = rechteck(320, 320, rot)
2  weisser_balken = rechteck(200, 60, weiss)
3  weisser_balken2 = drehe(90, weisser_balken)
4
5  kreuz = ueberlagere(weisser_balken, weisser_balken2)
6
7  schweizerfahne = ueberlagere(kreuz, hintergrund)
8
9  zeige_grafik(schweizerfahne)
```

3.6.4 Tessiner Wappen

```
1  roter_teil = rechteck(160, 240, rot)
2  blauer_teil = rechteck(160, 240, blau)
3  roter_sektor = kreis_sektor(160, 90, rot)
4  roter_sektor = drehe(180, roter_sektor)
5  blauer_sektor = kreis_sektor(160, 90, blau)
6  blauer_sektor = drehe(270, blauer_sektor)
7
8  ti_unten = neben(roter_sektor, blauer_sektor)
9  ti_oben = neben(roter_teil, blauer_teil)
10
11  tessin = ueber(ti_oben, ti_unten)
12
```

```
13  zeige_grafik(tessin)
```

4 Variablen in Python

4.1 Vorbemerkungen: Python als Rechner

Python verfügt über eingebaute mathematische Fähigkeiten. Es kann die Grundrechenarten und kennt die Hierarchie der Operationen. Sie können das überprüfen, in dem Sie in der folgenden Zelle die Rechnung

$$2 + 3 \cdot 4$$

ausführen.

```
1 # hier können Sie die Rechnung ausführen
```

Die folgende Tabelle gibt einen Überblick über die direkt in Python verfügbaren mathematischen Funktionen:

Beschreibung	Befehl	Beispiel	Resultat
Addition	+	$2 + 3$	5
Subtraktion	-	$3 - 2$	1
Multiplikation	*	$3 * 2$	6
Division	/	$3 / 2$	1.5
Potenzen	**	$3 ** 2$	9
Wurzeln	$**(1/n)$	$16 ** (1/2)$	4.0
Ganzzahlige Division	//	$7 // 2$	3
Modulo	%	$7 \% 2$	1

4.2 Variablen

In Python sind Variablen symbolische Namen für gespeicherte Daten. Variablen verweisen dabei auf den Speicherbereich im Computer, in welchem die entsprechenden Daten physikalisch abgelegt sind. Aus diesem Grund werden Variablen gelegentlich auch als Zeiger bezeichnet. Was genau für Daten in diesem Speicherbereich abgelegt werden, spielt keine Rolle und kann während der Ausführung eines Programmes auch ändern.

In Python werden Variablen Werte mit dem Gleichheitszeichen zugewiesen. Um der Variable x den Wert 2 zuzuweisen, ist die Eingabe $x = 2$ erforderlich. Die Variable muss links vom Gleichheitszeichen, der zuzuweisende Wert rechts davon stehen.

Überprüfen Sie dies, indem Sie in der folgenden Zelle der Variabel *y* den Wert 3 und der Variabel *z* den Wert 4 zuweisen. Anschliessend multiplizieren Sie die beiden Variablen miteinander.

```
1 # hier die Aufgabe einfüllen
```

Wenn Variablen neue Werte zugewiesen werden, wird die Referenz auf den Speicherbereich mit dem alten Wert gelöscht. Die Daten, welche ohne Verweis durch eine Variable im Speicher liegen, werden vom in Python eingebauten *Garbage Collector* im Hintergrund gelöscht und der so freigewordene Speicherplatz kann wieder verwendet werden.

Sie können überprüfen, dass Variablen neue Werte zugewiesen werden können, indem Sie in der untenstehenden Zelle die Variablen *y* und *z* addieren. Sie erhalten dann das Resultat 7. Das heisst, den Variablen *y* und *z* sind immer noch die Werte 3 und 4 zugewiesen.

```
1 # addieren Sie hier y und z
```

Wenn Sie in der folgenden Zelle der Variabel *y* den Wert 5 zuweisen und anschliessend *y* und *z* addieren erhalten Sie als neues Resultat 9.

```
1 # weisen Sie hier y den neuen Wert zu
```

Variablen können auch Resultate von Berechnungen zugewiesen werden. Ausserdem können Variablen ganze Wörter als Namen haben. Dies ist gegenüber einzelnen Buchstaben vorzuziehen, weil dann aussagekräftige Namen gewählt werden können. Grundsätzlich sind die Namen von Variablen frei wählbar. Es gibt allerdings eine Reihe von [reservierten Begriffen](#), welche in der Programmiersprache Python eine eigene Bedeutung haben. Unzulässig sind ausserdem Namen, die mit Ziffern beginnen.

Für die Darstellung von Namen für Variablen hat sich in Python eingebürgert, Variablen klein zu schreiben und Wörter durch Underlines zu trennen (*this_is_a_valid_variable*). Diese Darstellung nennt sich *Snake Case*. Zudem werden Variablen meist mit englischen Begriffen bezeichnet.

Weisen Sie in der nächsten Zelle der Variable **result** das Resultat der Rechnung $y + z$ zu und geben Sie das Resultat mit **print(result)** aus. **print()** ist eine Funktion, die Python zur Verfügung stellt. Was Funktionen sind, wird im nächsten Abschnitt erklärt.

```
1 # weisen Sie hier der Variable result das Resultat zu
```

4.3 Funktionen in Python

Python verfügt über viele bereits vordefinierte Funktionen. Die oben verwendete Funktion **print()** ist ein Beispiel dafür. Um zu demonstrieren, wie Funktionen in Python definiert

werden, zeige ich Ihnen als Beispiel eine Funktion, mit der zwei Zahlen addiert werden.

```
1  # Definition der Funktion
2  def get_sum(x, y):
3      return x + y
4
5  # Aufruf der Funktion
6  result = get_sum(3,4)
7  # Ausgabe des Resultats des Funktionsaufrufs
8  print(result)
```

`def` ist das Schlüsselwort für die Definition einer Funktion. `get_sum` ist der von mir gewählte Name dieser Funktion. Für die Wahl des Namens einer Funktion gelten die gleichen Regeln, wie für Variablen. In den Klammern stehen die sogenannten Parameter, welche der Funktion übergeben werden, damit sie etwas damit macht. Mit dem Doppelpunkt wird die *Signatur* der Funktion abgeschlossen. Die *Signatur* zeigt idealerweise, *was* eine Funktion *womit* macht. Sie gibt aber keine Auskunft darüber, wie sie das macht.

Python gruppiert Befehle, die zusammengehören, durch die gleiche Tiefe der Einrückung. Eine Einrückung hat üblicherweise die Tiefe von vier Leerzeichen. Im Beispiel oben gibt es nur eine eingerückte Zeile, weil die Funktion nur aus einem Befehl besteht. Mit `return` gibt die Funktion das Resultat zurück.

Im Beispiel wird das Resultat der Berechnung, welche die Funktion ausführt der Variable `result` zugewiesen. Der Wert der Variable `result` wird mit `print(result)` ausgegeben.

Definieren Sie in der folgenden Zelle eine Funktion, mit der zwei Zahlen multipliziert werden.

```
1  # hier kommt Ihre Funktion hin
```

4.4 Datentypen

Als nächstes geht es um die Frage, auf welche Inhalte eine Variable zeigen kann.

Im Grundsatz kann eine Variable auf beliebige Inhalte verweisen.

Am einfachsten ist die Verwendung der grundlegenden Datentypen (basic data types), welche Python zur Verfügung stellt. Dies sind (mit ihren englischen Bezeichnungen):

- Integer (Ganzzahl)
- Floating-Point Number (Gleitkommazahl)
- Complex Number (komplexe Zahl)
- String (Zeichenkette)
- Boolean Type (Wahrheitswert)

Darüber hinaus ist es möglich, eigene Datentypen zu programmieren. Hier aber zuerst eine Beschreibung der grundlegenden Datentypen von Python.

4.4.1 Integer

Die Bezeichnung für Integer in Python ist ein kurzes `int`.

Anders als in anderen Programmiersprachen gibt es in Python theoretisch keine Beschränkung, wie gross ein Integer sein kann. Die einzige Grenze ist der Speicherplatz des auf dem der Integer gespeichert werden soll.

Wenn einer Variable ein grosser Integer zugewiesen wird, kann dieser zur besseren Lesbarkeit auch mit einem Underline als Tausendertrennzeichen geschrieben werden (`100_000`).

Um das Auszuprobieren, weisen Sie in der folgenden Zelle der Variable `a` den Wert von einer Million und der Variable `b` den Wert von einer Milliarde zu. Anschliessend addieren Sie `a` und `b` und weisen das Resultat der Variable `big_sum` zu. Zum Schluss geben Sie den Wert von `big_sum` mit der Funktion `print()` aus.

```
1 # hier können Sie Ihre Berechnung vornehmen
```

Eingegebene Zahlen werden automatisch als Dezimalzahlen interpretiert.

Integers können jedoch auch als Binär-, Oktal- oder Hexadezimalzahlen eingegeben werden. Die Eingabe erfordert dann allerdings ein Präfix, welches das Zahlensystem identifiziert. Die folgende Tabelle stellt die möglichen Präfixe zusammen.

Präfix	Bedeutung	Basis
<code>0b</code> (Null + Kleinbuchstabe <code>b</code>)	Binärzahl	2
<code>0B</code> (Null + Grossbuchstabe <code>B</code>)		2
<code>0o</code> (Null + Kleinbuchstabe <code>o</code>)	Oktalzahl	8
<code>0O</code> (Null + Grossbuchstabe <code>O</code>)		8
<code>0x</code> (Null + Kleinbuchstabe <code>x</code>)	Hexadezimalzahl	16
<code>0X</code> (Null + Grossbuchstabe <code>X</code>)		16

In der folgende Zelle finden Sie ein entsprechendes Beispiel.

Python kann Integer in verschiedenen Zahlensystemen darstellen. Um das Zahlensystem bei der Zuweisung zu spezifizieren, verwenden Sie die entsprechenden Präfixe. Die folgenden Beispiele zeigen die Zuweisung der Zahlen 10, 8, 255 und 16 in den Zahlensystemen Dezimal, Oktal, Hexadezimal und Binär:

```
1 dezimal      = 10
2 oktal        = 0o10
3 hexadezimal  = 0x10
```

```
4  binaer      = 0b10
```

Die Anzeige der Werte der entsprechenden Variablen erfolgt grundsätzlich im Dezimalsystem.

```
1  b = 0b101010
2  o = 0o52
3  x = 0x2a
4
5  print(b, o, x)
```

4.4.2 Gleitkommazahl

Die Bezeichnung für Gleitkommazahlen in Python ist `float`. Python interpretiert Zahlen mit einem Dezimalpunkt als Gleitkommazahlen. Optional können Zahlen mit `e` oder `E` in “wissenschaftlicher” Schreibweise eingegeben werden ($1000 = 1e3$ bzw. $1e-3 = 0.001$).

Weisen Sie in der folgenden Zelle den Variablen `million` und `billionth` die passenden Werte in wissenschaftlicher Schreibweise zu.

```
1  # hier die Werte den beiden Variablen zuweisen
```

4.4.3 Komplexe Zahlen

Python kann auch mit komplexen Zahlen umgehen. Der Abschnitt zu diesem Thema kann wieder aufgegriffen werden, wenn Sie in Mathe die komplexen Zahlen besprochen haben.

4.4.4 String

Zeichenketten (String) werden von Python als `str` bezeichnet.

Zeichenketten sind beliebige Zeichenfolgen. Damit Python Zeichenketten als solche erkennt, müssen sie durch die Verwendung von einfachen oder doppelten Anführungs- und Schlusszeichen als solche gekennzeichnet werden.

"Ich bin eine Zeichenkette." oder 'Ich bin auch eine Zeichenkette.'

Wenn man innerhalb einer Zeichenkette Anführungszeichen braucht, müssen die eingrenzenden Anführungszeichen von der “anderen Sorte” sein ("It's cool learning Python!" oder 'Der Lehrer sagt: "Es ist cool Python zu lernen."'). Eine andere Möglichkeit reservierte Zeichen zu verwenden ist der Gebrauch eines “escape”-Zeichens. In Python ist das der “backslash” (`\`). Die beiden Beispielsätze von vorher hätten entsprechend auch folgendermassen geschrieben werden können:

'It\'s cool learning Python!' bzw. "Der Lehrer sagt: \"Es ist cool Python zu lernen.\""

Die Länge von Zeichenketten wird lediglich durch die Speicherkapazität des verwendeten Systems begrenzt. Zeichenketten können nicht nur sehr lang, sondern auch leer sein ('').

Zeichenketten können, wie alle Datentypen, Variablen zugewiesen werden. Dies zeigt das folgende Beispiel.

```
1 # Zuweisung eines Strings zu einer Variabel
2 standard_greeting = "Hello World"
3
4 # Ausgabe der Variabel
5 print(standard_greeting)
```

4.4.5 Boolean Type

Wahrheitswerte werden in Python als `bool` bezeichnet. Wahrheitswerte können entweder "wahr" oder "falsch" sein.

```
1 wahr      = True
2 falsch    = False
```

Die Ausgabe findet sich in der folgenden Zelle.

```
1 wahr      = True
2 falsch    = False
3 print(wahr, falsch)
```

Welche Wahrheitswerte sich bei Vergleichen ergeben, kann mit den folgenden Operatoren überprüft werden:

```
1 == # Gleichheit
2 != # Ungleichheit
3 >  # Größer als
4 <  # Kleiner als
5 >= # Größer oder gleich
6 <= # Kleiner oder gleich
```

```
1 # Testen Sie hier den Wahrheitswert von Vergleichen mit Zahlen
```

Wahrheitswerte werden zur Steuerung von Programmflüssen verwendet. Mit einem Wahrheitswert kann zum Beispiel gesteuert werden, wie oft ein Programmteil wiederholt werden soll.

4.5 Funktionen mit Type-Hints

Zum Abschluss komme ich noch einmal auf die Definition von Funktionen zurück. In Python können Variablen - anders als zum Beispiel in Java - beliebige Datentypen zugewiesen werden. Wenn Variablen im Verlauf eines Programms mehrfach verwendet werden, können ihnen auch unterschiedliche Datentypen zugewiesen werden. Dies ist allerdings schlechter Programmierstil.

Aus diesem Grund ist es sinnvoll, bei der Definition einer Funktion zu deklarieren, welche Datentypen die Parameter haben und welcher Datentyp der Rückgabewert hat. Dies soll mit dem Beispiel der Funktion `get_quotient` verdeutlicht werden.

```
1 def get_quotient(x : int, y : int) -> float:
2     return x / y
```

Hier wird angegeben, dass die Parameter x und y vom Datentyp `int` sein sollen. Der Datentyp des Rückgabewertes wird hinter `->` geschrieben. Im Beispiel ist der Rückgabewert vom Typ `float`. Das ist so, weil die Funktion zum Beispiel $3/4 = 3.5$ rechnet.

Aber Achtung: die Funktion arbeitet auch dann korrekt, wenn ein anderer als der deklarierte Datentyp übergeben wird. Voraussetzung ist lediglich, dass der Datentyp mit den verwendeten Operationen kompatibel ist. Die "Type-Hints" dienen lediglich der besseren Nachvollziehbarkeit, was die Funktion macht.

4.6 Ausgewählte Musterlösungen

4.6.1 Funktion zur Multiplikation zweier Zahlen

```
1 def get_product(a, b):
2     return a * b
3
4 product = get_product(3, 4)
5 print(product)
```

4.6.2 Gleitkommazahlen in wissenschaftlicher Schreibweise

```
1 million = 1e6
2 billonth = 1e-9
3 print(million, billonth)
```

1000000.0 1e-09

5 Wiederholungen in Python (For-Loops)

Eine Stärke von Computerprogrammen ist die wiederholte Ausführung von Anweisungen. Viele Programmiersprachen stellen dafür ein Konstrukt mit dem Namen ‘For-Schleife’ zur Verfügung. Eine ‘For-Schleife’ funktioniert unabhängig von einer konkreten Programmiersprache folgendermassen:

```
FÜR variable VON startwert BIS endwert [MIT schrittweite]
    Anweisungen
ENDE FÜR
```

Übersetzt nach Python sieht das so aus:

```
1  for i in range(n):
2      do...
```

`startwert BIS endwert [MIT schrittweite]` wird dabei durch `range(n)` ausgedrückt. Dabei ist `n` der Endwert. Gezählt wird bis zum aber ohne den Endwert. Startwert und Schrittweite haben Vorgabewerte. Der Vorgabewert für den Start ist `0`, derjenige der Schrittweite `1`. Weil `range()` diese vorgegebenen Werte hat, müssen diese nicht explizit angegeben werden. Wenn der Startwert abweichend vom Vorgabewert festgelegt werden soll, kann dieser explizit angegeben werden. Der Aufruf von `range()` sieht dann so aus:

```
1  range(startwert, endwert)
```

Falls eine von `1` abweichende Schrittweite festgelegt werden soll lautet der Aufruf

```
1  range(startwert, endwert, schrittweite)
```

In diesem Fall müssen neben dem Endwert sowohl der Startwert und die Schrittweite angegeben werden. Andernfalls kann nicht zwischen den einzelnen Angaben zu Endwert, Startwert und Schrittweite unterschieden werden.

5.1 Anwendungsübung zu Wiederholungen in Python

Im Folgenden bauen Sie eine Blume mit Python for-loops und PyTamaro.

In der folgenden Zelle werden zuerst [alle von PyTamaro zur Verfügung gestellten Funktionen](#) importiert.

```
1 from pytamaro.de import *
```

Das Ziel ist es, eine Blume wie die abgebildete zu zeichnen.

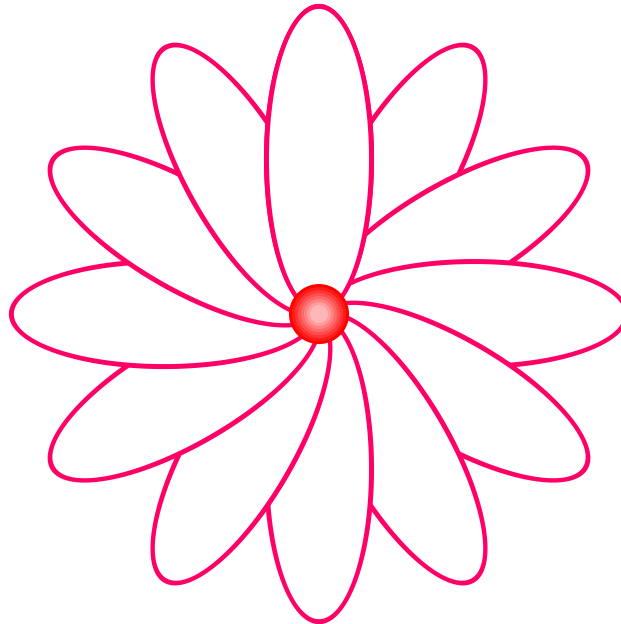


Abbildung 5.1: Blume

Die wichtigsten Funktionen von PyTamaro für diese Aufgabe sind `fixiere()` und `kombiniere()`.

Die Funktion `fixiere()` legt für eine Grafik einen frei gewählten Ankerpunkt fest. Dazu muss man wissen, dass jede in PyTamaro gezeichnete Grafik von einer sogenannten “Bounding Box” umgeben ist. Diese Bounding Box ist ein Rechteck, das die Grafik vollständig umschließt. Standardmässig liegt der Ankerpunkt einer Grafik in der Mitte der Bounding Box. Mit `fixiere()` kann man den Ankerpunkt jedoch an eine andere Stelle der Bounding Box verschieben. PyTamaro stellt die folgenden Positionen für Ankerpunkte zur Verfügung:

- `mitte`: Mitte der Bounding Box (Standard)
- `mitte_links`: Mitte der linken Seite der Bounding Box
- `mitte_rechts`: Mitte der rechten Seite der Bounding Box
- `oben_links`: Obere linke Ecke der Bounding Box
- `oben_mitte`: Mitte der oberen Seite der Bounding Box
- `oben_rechts`: Obere rechte Ecke der Bounding Box
- `unten_links`: Untere linke Ecke der Bounding Box
- `unten_mitte`: Mitte der unteren Seite der Bounding Box
- `unten_rechts`: Untere rechte Ecke der Bounding Box

Der Ankerpunkt wird mit dem Befehl `fixiere(position, grafik)` gesetzt.

Der Ankerpunkt ist wichtig für die Funktion `drehe()`, die eine Grafik um einen bestimmten Winkel dreht. Die Drehung erfolgt immer um den Ankerpunkt.

Die Funktion `kombiniere(vordere Grafik, hintere Grafik)` fügt zwei Grafiken zusammen. Die erste gegebene Grafik liegt im Vordergrund und die zweite im Hintergrund. Die Grafiken werden so ausgerichtet, dass ihre Ankerpunkte übereinanderliegen.

5.1.1 Schritt 1: Zerlege das Bild in seine Einzelteile

Die Blume besteht im wesentlichen aus zwei Formen:

1. Blütenblätter in Form von Ellipsen sowie
2. einer Scheibe in der Form eines Kreises in der Mitte.

Um die Blume zu zeichnen, müssen diese beiden Formen zuerst einzeln erstellt werden.

```
1 bluetenblatt = ellipse(50, 150, blau)
2 # die Farbe blau wurde gewählt, damit das Blatt vor dem Hintergrund
3 # sichtbar ist
4
5 zeige_grafik(bluetenblatt)
```



```
1 scheibe = ellipse(30, 30, rot)
2 zeige_grafik(scheibe)
```



5.1.2 Schritt 2: Positioniere die Blütenblätter

Die Blütenblätter sind rund um die Mitte der Blume angeordnet. Da es zwölf Blütenblätter sind, beträgt der Winkel zwischen zwei Blütenblättern $\frac{360^\circ}{12} = 30^\circ$. Die Drehung der Blütenblätter erfolgt um den Ankerpunkt `unten_mitte` der Blütenblätter.

Die Blütenblätter müssen also zuerst fixiert und dann gedreht werden.

```
1 bluetenblatt_fixiert = fixiere(unten_mitte, bluetenblatt)
2 bluetenblatt_30 = drehe(30, bluetenblatt_fixiert)
3 zeige_grafik(bluetenblatt_30)
```



5.1.3 Schritt 3: Zeichnen aller erforderlichen Blütenblätter

Die Blume hat zwölf Blütenblätter. Diese können von Hand jedes einzelne erstellt werden. Das dritte Blütenblatt ist um 60° gedreht und wird entsprechend mit

```
1 bluetenblatt_60 = drehe(60, bluetenblatt_fixiert)
```

erstellt. Das kann man für alle zwölf Blütenblätter machen bis man beim zwölften Blütenblatt angekommen ist, das um 330° gedreht ist.

```
1 bluetenblatt_330 = drehe(330, bluetenblatt_fixiert)
```

Das ist aber sehr mühsam und fehleranfällig. Viel einfacher ist es, eine Schleife (**for**-loop) zu verwenden, die die Blütenblätter basierend auf der Grundform automatisch erstellt.

```
1 for i in range(12):
2     winkel = i * 30
3     bluetenblatt_gedreht = drehe(winkel, bluetenblatt_fixiert)
4
5 zeige_grafik(bluetenblatt_gedreht)
```

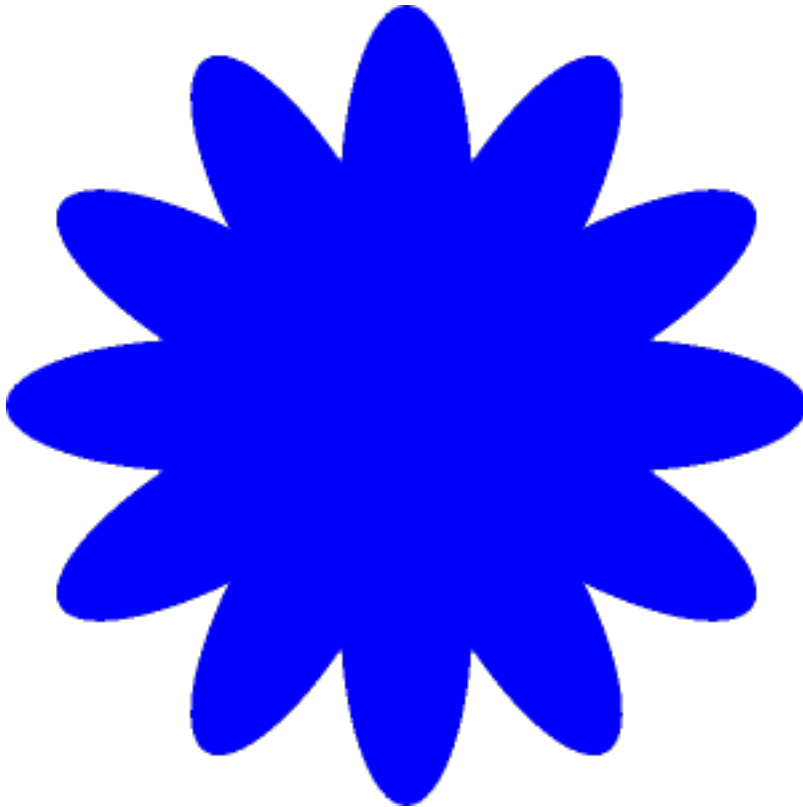


Diese Schleife durchläuft die Zahlen von 0 bis 11 (also 12 Zahlen) und berechnet für jede Zahl den entsprechenden Drehwinkel. Dann wird das Blütenblatt um diesen Winkel gedreht. Allerdings wird das gedrehte Blütenblatt in jeder Iteration der Schleife in der gleichen Variable `blütenblatt_gedreht` gespeichert. Am Ende der Schleife enthält diese Variable nur das letzte gedrehte Blütenblatt (das um 330° gedrehte Blütenblatt).

5.1.4 Schritt 4: Kombiniere alle Blütenblätter

Um das unerwünschte Verhalten zu vermeiden, dass am Ende der Schleife nur das letzte gedrehte Blütenblatt in der Variable `blütenblatt_gedreht` enthalten ist, müssen die gedrehten Blütenblätter in jeder Iteration der gewünschten Gesamtgrafik hinzugefügt werden. Dazu wird die Funktion `kombiniere()` verwendet.

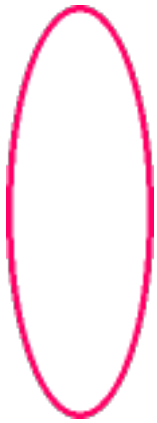
```
1 blume = blütenblatt_fixiert
2
3 for i in range(12):
4     winkel = i * 30
5     blütenblatt_gedreht = drehe(winkel, blütenblatt_fixiert)
6     blume = kombiniere(blume, blütenblatt_gedreht)
7
8 zeige_grafik(blume)
```



Damit sind die Blütenblätter in der gewünschten Position und Anzahl zusammengefügt. Damit Sie allerdings aussehen, wie die Blume in der Vorlage, müssen die einzelnen Blütenblätter weiss eingefärbt und mit einem rosa Rand versehen werden.

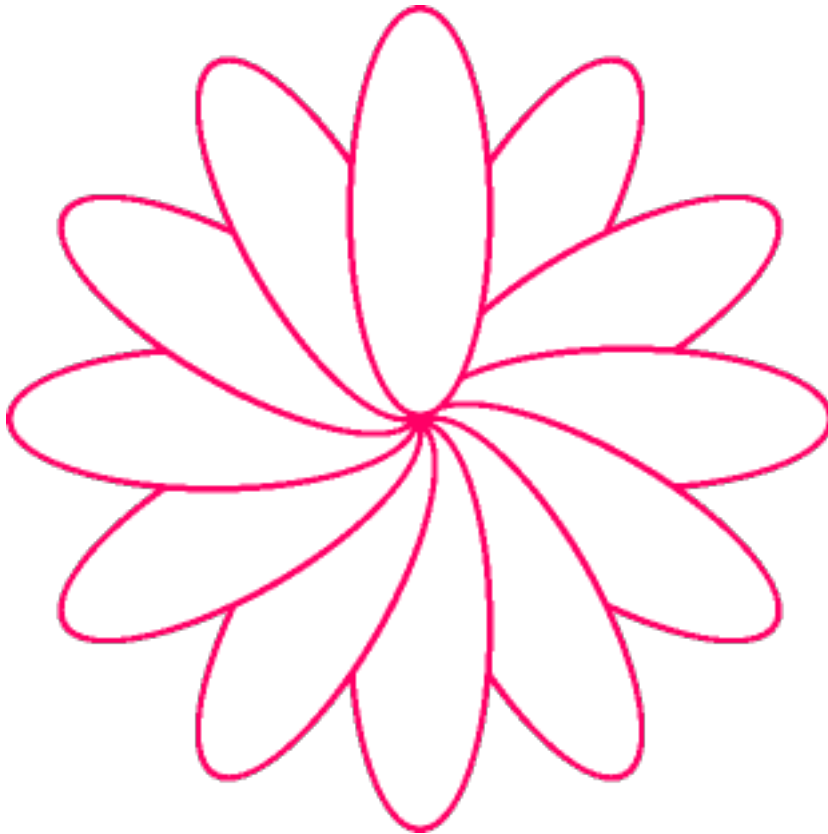
Rosa ist keine vordefinierte Farbe in PyTamaro. Sie können jedoch eine beliebige Farbe mit der Funktion `rgb_farbe(rot, gruen, blau)` erstellen. Dabei sind `rot`, `gruen` und `blau` Zahlenwerte von 0 bis 255, die den Anteil der jeweiligen Farbe an der Gesamtfarbe angeben. Damit nicht alle Kombinationen durchprobiert werden müssen, können Sie mit einem Online-Tool wie [RGB Color Picker](#) die gewünschte Farbe auswählen und die entsprechenden Werte für rot, grün und blau ablesen.

```
1 weisses_blatt = ellipse(50, 150, weiss)
2 rosa_rand = ellipse(55, 155, rgb_farbe(255, 0, 102))
3
4 bluetenblatt_mit_rand = ueberlagere(weisses_blatt, rosa_rand)
5
6 zeige_grafik(bluetenblatt_mit_rand)
```



Anschliessend können die Blütenblätter mit Rand in der vorher geschriebenen Schleife zur Gesamtgrafik zusammengefügt werden.

```
1  blutenblatt_mit_rand_fixiert = fixiere(unten_mitte,  
    ↪  blutenblatt_mit_rand)  
2  
3  blume_weiss = blutenblatt_mit_rand_fixiert  
4  
5  for i in range(12):  
6      winkel = i * 30  
7      blutenblatt_gedreht = drehe(winkel, blutenblatt_mit_rand_fixiert)  
8      blume_weiss = kombiniere(blume_weiss, blutenblatt_gedreht)  
9  
10 zeige_grafik(blume_weiss)
```

5.1.5 Schritt 5: Scheibe im Zentrum der Blume

Bei genauer Betrachtung der Blume fällt auf, dass die Scheibe in der Mitte der Blume nicht einfach einfarbig gelb ist, sondern gegen die Mitte hin einen Farbverlauf aufweist. Diesen Farbverlauf wird erzeugt, indem mehrere Kreise mit kleiner werdendem Radius und abnehmender Farbintensität übereinandergelegt werden.

```

1  scheibe = ellipse(30, 30, rgb_farbe(255, 0, 0))
2
3  for i in range(1, 8):
4      tmp = ellipse(30 - 3*i, 30 - 3*i, rgb_farbe(255, i*26, i*26))
5      scheibe = ueberlagere(tmp, scheibe)
6
7  zeige_grafik(scheibe)

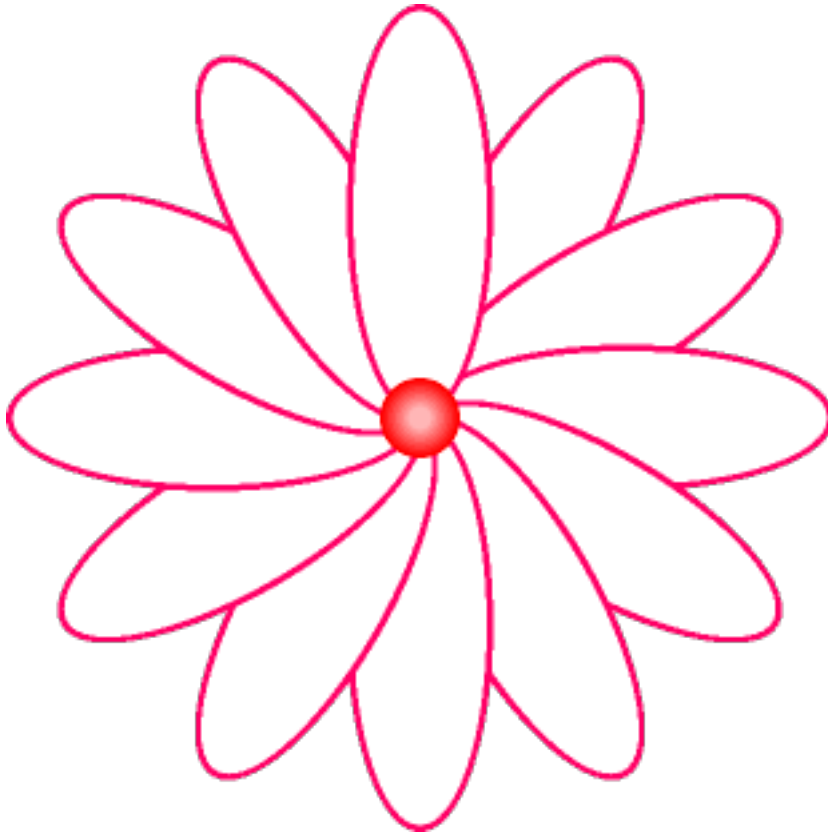
```



5.1.6 Schritt 6: Kombination der Elemente zur vollständigen Blume

Zum Schluss wird die Scheibe auf die Blütenblätter gelegt und Blume so vervollständigt.

```
1 blume_komplett = ueberlagere(scheibe, blume_weiss)
2 zeige_grafik(blume_komplett)
```



6 Programmverzweigungen (Bedingungen)

Es gibt Situationen, in denen es wünschenswert ist, dass ein Teil eines Programms nur ausgeführt wird, wenn eine bestimmte Voraussetzung erfüllt ist.

Als Beispiel dafür mag die Blume von vergangener Woche dienen. Wenn eine Blume gezeichnet werden soll, welche abwechselungsweise dunkelblaue und hellblaue Blütenblätter hat.

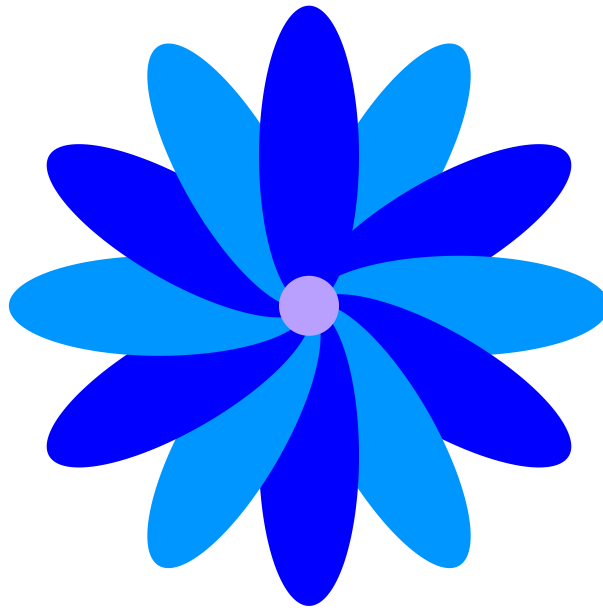


Abbildung 6.1: Beispielblume

Diese Blume wurde ebenfalls mit einer `for`-Schleife gezeichnet. Allerdings wurde abwechselungsweise ein dunkelblaues und ein hellblaues Blütenblatt verwendet.

Abwechselungsweise kann auch als wenn eine gerade Zahl kommt bzw. wenn eine ungerade Zahl kommt verstanden werden. Kommen beim Zählen doch abwechselungsweise gerade und ungerade Zahlen. Übersetzt in die `for i in range(12)`-Schleife heisst das, immer wenn der Wert in der Variabel `i` eine gerade Zahl ist dann soll ein dunkelblaues Blüentblatt verwendet werden und immer wenn der Wert der Variabel `i` eine ungerade Zahl ist, soll ein hellblaues Blütenblatt verwendet werden.

In Python verwendet man dazu die folgende Syntax:

```
1  if BEDINGUNG:  
2      ANWEISUNG(en)
```

```

3  else:
4      ANWEISUNG(en)

```

Das heisst, immer wenn die **BEDINGUNG** erfüllt ist (wenn sie **wahr**) ist, werden die eingerückten Anweisungen ausgeführt. In allen anderen Fällen werden die nach **else:** eingerückten Anweisungen ausgeführt.

Für die Zeichnung der Blume bedeutet das also, dass immer wenn **i** eine gerade Zahl ist, soll ein dunkelblaues Blütenblatt verwendet werden. Eine gerade Zahl ist dabei jede Zahl, die ohne Rest durch zwei Teilbar ist.

Ob dies der Fall ist, kann mit dem Modulo Operator (%) getestet werden (vgl. Abschnitt Vorbemerkungen: Python als Rechner.) Eine Zahl ist ohne Rest durch zwei Teilbar, wenn gilt

$$i \bmod 2 = 0$$

Übersetzt nach Python schreibt sich das als

```

1  if i % 2 == 0:

```

Für die Prüfung, ob das Resultat 0 ist, wird dabei **==** verwendet. Dies rührt daher, dass das einfache Gleichheitszeichen bereits als Operator für die Zuweisung eines Wertes zu einer Variabel besetzt war.

Der vollständige Code für die Abwechslungsweise Verwendung eines dunkelblauen bzw. eines hellblauen Blütenblattes sieht folgendermassen aus:

```

1  if i % 2 == 0:
2      blume = kombiniere(blume, bluetenblatt_dunkel)
3  else:
4      blume = kombiniere(blume, bluetenblatt_hell)

```

Falls mehr als zwei Fälle unterschieden werden müssen, lautet die Python Syntax

```

1  if BEDINGUNG:
2      ANWEISUNG(en)
3  elif BEDINGUNG:
4      ANWEISUNG(en)
5  else:
6      ANWEISUNG(en)

```

Dabei können beliebig viele **elif** Bedingungen wiederholt werden.

Als Bedingung kann nicht nur eine Gleichheit überprüft werden. Es ist auch möglich grösser >, grösser oder gleich >=, kleiner < und kleiner oder gleich <= zu prüfen.

6.1 Anwendungsübung zu Bedingungen in Python

6.1.1 Aufgabenstellung

Zeichnen Sie die folgende Blume.

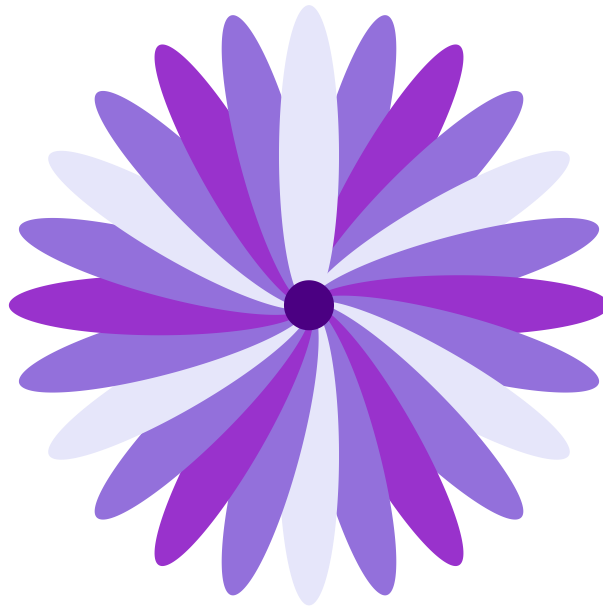


Abbildung 6.2: Blumenvorlage

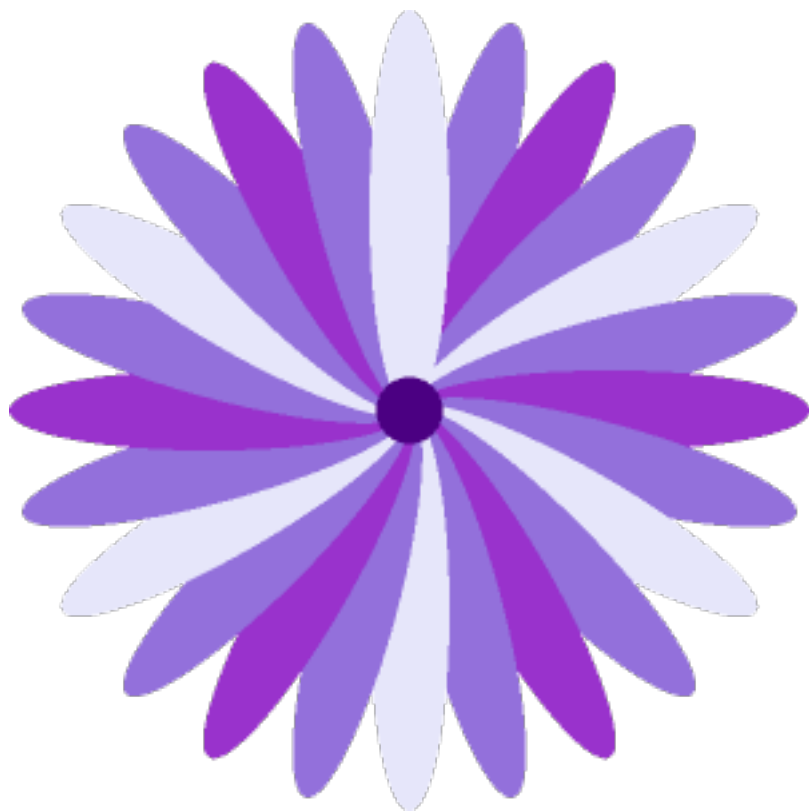
Die erforderlichen Importe und Farbdefinitionen finden Sie in der folgenden Code Zelle.

```
1 from pytamaro.de import *
2
3 lavender = rgb_farbe(230, 230, 250)
4 mediumpurple = rgb_farbe(147, 112, 219)
5 darkorchid = rgb_farbe(153, 50, 204)
6 indigo = rgb_farbe(75, 0, 130)

1 # Hier ist Platz für Ihre Blume
2 # Hinweis: die Verschiebung einer Farbe um eine Postion kann erreicht
3 # werden, wenn i % 4 nicht 0 sondern eine Zahl ergibt. Bei % 4 können
4 # dies die Zahlen 1, 2 oder 3 sein.
```

6.1.2 Musterlösung

```
1  bluetenblatt_l = ellipse(30, 150, lavender)
2  bluetenblatt_m = ellipse(30, 150, mediumpurple)
3  bluetenblatt_d = ellipse(30, 150, darkorchid)
4  scheibe_i = ellipse(25, 25, indigo)
5
6  bluetenblatt_m_fix = fixiere(unten_mitte, bluetenblatt_m)
7  bluetenblatt_l_fix = fixiere(unten_mitte, bluetenblatt_l)
8  bluetenblatt_d_fix = fixiere(unten_mitte, bluetenblatt_d)
9
10 blume3 = scheibe_i
11 for i in range(24):
12
13     if i % 4 == 0:
14         bl_temp = drehe(i * 15, bluetenblatt_l_fix)
15         blume3 = kombiniere(blume3, bl_temp)
16     elif i % 4 == 2:
17         bd_temp = drehe(i * 15, bluetenblatt_d_fix)
18         blume3 = kombiniere(blume3, bd_temp)
19     else:
20         bm_temp = drehe(i * 15, bluetenblatt_m_fix)
21         blume3 = kombiniere(blume3, bm_temp)
22
23 zeige_grafik(blume3)
```



7 Programmieren einfacher Funktionen

7.1 Kreisfläche

Die Fläche eines Kreises berechnet sich nach der Formel

$$A = \pi r^2$$

wobei A die Fläche π die Kreiszahl - hier mit zwei Nachkommastellen - 3.14 und r den Radius des Kreises darstellt.

Aufgabe: Implementieren Sie eine Funktion `kreisflaeche()` welche ein Argument `r` für Radius entgegennimmt und die Fläche eines Kreises zurückgibt.

```
1 def kreisflaeche():  
2     # TODO: implementieren Sie die Funktion
```

7.2 Quotient

Ein Quotient kann unterschiedlich dargestellt werden. Eine Möglichkeit ist die Darstellung als Dezimalzahl.

Aufgabe: Implementieren Sie eine Funktion `quotient()` welche zwei Dezimalzahlen als Argumente entgegennimmt und den Quotienten als Dezimalzahl zurückgibt. Sehen Sie für den Sonderfall der Division durch Null eine Lösung vor.

```
1 def quotient():  
2     # TODO: implementieren Sie die Funktion
```

7.3 Fakultät

Die Fakultät ist jene Funktion, welche das Produkt der natürlichen Zahlen bis n berechnet und wird als $n!$ geschrieben. Es gilt also

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

Wobei zu beachten ist, dass $0! = 1$ gilt.

Aufgabe: Implementieren Sie eine Funktion `fakultaet` welche als Argument $n \in \mathbb{N}$ entgegennimmt.

```
1 def fakultaet():
2     # TODO: implementieren Sie die Funktion
```

7.4 Quadratische Gleichungen (Mitternachtsformel)

Quadratische Gleichungen der Form $ax^2 + bx + c = 0$ können mit der Formel

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

gelöst werden.

Für die Implementierung ergibt sich daraus allerdings das Problem, dass die Formel zwei Resultate ergibt und eine Python Funktion nur ein Objekt zurückgeben kann.

Als Workaround können die beiden Resultate in ein Tupel gepackt werden. Ein Tupel ist eine unveränderliche Zusammenstellung mehrerer Werte. In Python werden sie in Klammern dargestellt. Die Zuweisung eines Tupels zu einer Variabel sieht dementsprechend folgendermassen aus:

```
1 t = (wert_1, wert_2, ..., wert_n)
```

Die detaillierten eigenschaften von Tupeln können hier ausser Acht gelassen werden. Es reicht x_1 und x_2 einem Tupel `resultat` zuzuweisen und `resultat` zurückzugeben.

Aufgabe: Implementieren Sie eine Funktion `mitternachtsformel`, welche die Argumente a , b und c entgegennimmt und das Tupel x_1 und x_2 zurückgibt.

```
1 def mitternachtsformel():
2     # TODO: implementieren Sie die Funktion
```

7.5 Musterlösungen

7.5.1 Kreisfläche

```
1 def kreisflaeche(r: float) -> float:
2     """
3     Berechnet die Fläche eines Kreises anhand des gegebenen Radius.
4     """
```

```

5     Parameter
6     -----
7     r : float
8         Der Radius des Kreises.
9
10    Rückgabewert
11    -----
12    float
13        Die Fläche des Kreises.
14    """
15    PI = 3.14 # Näherungswert für die Kreiszahl Pi
16    a = PI * r ** 2 # Formel: Fläche = Pi * r^2
17    return a

```

7.5.2 Quotient

```

1  def quotient(z: float, n: float) -> float:
2      """
3      Berechnet den Quotienten zweier Zahlen.
4
5      Gibt eine Fehlermeldung zurück, falls der Nenner null ist.
6
7      Parameter
8      -----
9      z : float
10         Der Zähler der Division.
11      n : float
12         Der Nenner der Division.
13
14      Rückgabewert
15      -----
16      float oder str
17         Der berechnete Quotient oder eine Fehlermeldung als
↪ Zeichenkette,
18         falls eine Division durch Null versucht wurde.
19      """
20
21      # Fehlermeldung für Division durch Null
22      fehlermeldung = "Eine Division durch Null ist unzulässig."
23      if n == 0:
24          return fehlermeldung # Fehlerfall: Division durch Null
25      else:

```

```

26         q = z / n # Berechnung des Quotienten
27         return q

```

7.5.3 Fakultät

```

1  def fakultaet(n: int) -> int:
2      """
3      Berechnet die Fakultät einer nicht-negativen ganzen Zahl n.
4
5      Die Fakultät n! ist das Produkt aller positiven ganzen Zahlen
6      von 1 bis n. Für n < 0 ist die Fakultät nicht definiert.
7
8      Parameter:
9          n (int): Die Zahl, deren Fakultät berechnet werden soll.
10
11     Rückgabewert:
12         int: Die Fakultät von n, oder eine Fehlermeldung als String bei
13         ↪ n < 0.
14     """
15     if n < 0:
16         # Fakultät ist für negative Zahlen nicht definiert
17         return f'Die Fakultät für {n} ist nicht definiert.'
18     elif n == 0:
19         # Per Definition ist 0! = 1
20         return 1
21     else:
22         prod = 1 # Initialisiere das Produkt mit 1
23         for i in range(1, n + 1):
24             prod *= i # Multipliziere prod mit dem aktuellen Wert von i
25         return prod # Gib das berechnete Produkt zurück

```

7.5.4 Mitternachtsformel

```

1  def mitternachtsformel(a: float, b: float, c: float) -> tuple:
2      """
3      Löst eine quadratische Gleichung der Form  $ax^2 + bx + c = 0$ 
4      mit der Mitternachtsformel.
5
6      Parameter
7      -----
8      a : float

```

```

9         Der Koeffizient vor x2 (muss ungleich 0 sein).
10    b : float
11        Der Koeffizient vor x.
12    c : float
13        Der konstante Term.
14
15    Rückgabewert
16    -----
17    tuple oder str
18        Ein Tupel mit den beiden Lösungen (x1, x2) oder eine
↪ Fehlermeldung
19        als Zeichenkette, falls die Gleichung keine reellen Lösungen
↪ besitzt.
20    """
21    # Berechnung der Diskriminante: d = b2 - 4ac
22    d = b ** 2 - 4 * a * c
23    if d < 0:
24        # Keine reellen Lösungen vorhanden
25        return "Diese Gleichung hat keine Lösung."
26    else:
27        # Berechnung der beiden Lösungen mit der Mitternachtsformel:
28        # x1 = (-b + d ** (1/2)) / (2a)
29        # x2 = (-b - d ** (1/2)) / (2a)
30        x1 = (-b + d ** (1/2)) / (2 * a)
31        x2 = (-b - d ** (1/2)) / (2 * a)
32        return (x1, x2)

```

8 Prüfungsvorbereitung

8.1 Belegung von Variablen

Das folgende Code Snippet demonstriert die Belegung von Variablen. Insbesondere soll es auch zeigen, was geschieht, wenn die Variablen in Operationen einbezogen werden.

Überlegen Sie sich bevor Sie das Snippet ausführen zuerst Anweisung für Anweisung, welche Werte in den Variablen *a*, *b* und *c* nach Ausführung der jeweiligen Anweisung gespeichert sind. Notieren Sie sich die Werte auf ein Blatt Papier. Anschliessend führen Sie die Zelle zur Kontrolle Ihrer Überlegungen aus.

```
1  # Anfangswerte zuweisen
2  a = 5
3  b = 10
4  c = a
5  print(f"Nach Initialisierung:, a = {a}, b = {b}, c = {c}")
6
7  # Wert von a ändern
8  a = a + 3
9  print(f"Nach a = a + 3: a = {a}, b = {b}, c = {c}")
10
11 # Wert von b ändern
12 b = a * c
13 print(f"Nach b = a * c: a = {a}, b = {b}, c = {c}")
14
15 # Wert von c ändern
16 c = b - a
17 print(f"Nach c = b - a: a = {a}, b = {b}, c = {c}")
```

8.2 Funktionen

Die Definition einer neuen Funktion erweitert den Umfang der in Python zur Verfügung stehenden Befehle. Entsprechend ist es möglich, einmal definierte Funktionen in neuen Funktionen zu verwenden.

Unten finden Sie ein einfaches Beispiel für diese Vorgehensweise.

```

1  def erfolgsberechnung(ertrag: float, aufwand: float) -> float:
2      """Berechnet den wirtschaftlichen Erfolg als Differenz von Ertrag
3      und Aufwand.
4
5      Args:
6          ertrag: Der gesamte erzielte Ertrag.
7          aufwand: Der gesamte angefallene Aufwand.
8
9      Returns:
10         Der berechnete Erfolg (Ertrag minus Aufwand).
11     """
12     erfolg = ertrag - aufwand
13     return erfolg
14
15
16  def renditeberechnung(ertrag: float, aufwand: float,
17                        anfangskapital: float,
18                        schlusskapital: float) -> float:
19      """Berechnet die Rendite in Prozent basierend auf Erfolg
20      und durchschnittlichem Kapital.
21
22      Die Funktion nutzt die 'erfolgsberechnung', um den Erfolg
23      zu ermitteln, und setzt diesen ins Verhältnis zum durchschnittlich
24      gebundenen Kapital.
25
26      Args:
27          ertrag: Der gesamte erzielte Ertrag.
28          aufwand: Der gesamte angefallene Aufwand.
29          anfangskapital: Das Kapital zu Beginn der Periode.
30          schlusskapital: Das Kapital am Ende der Periode.
31
32      Returns:
33         Die berechnete Rendite, ausgedrückt in Prozent (%).
34     """
35     erfolg = erfolgsberechnung(ertrag, aufwand)
36     durchschnittskapital = (anfangskapital + schlusskapital) / 2
37     rendite = erfolg / durchschnittskapital * 100
38     return rendite

```

8.2.1 Berechnung des Volumens eines Zylinders

Die folgende Grafik zeigt einen Zylinder.

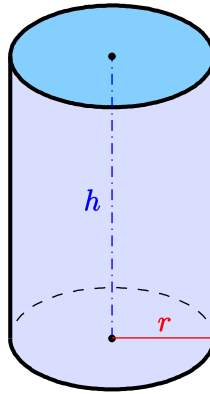


Abbildung 8.1: Zylinder

Die Formel für die Berechnung des Volumens lautet

$$v = \pi \cdot r^2 \cdot h$$

Zerlegen Sie dazu die Berechnung in zwei Teile. Definieren Sie zuerst eine Funktion für die Berechnung der Bodenfläche und verwenden Sie diese anschliessend in einer Funktion zur Berechnung des Volumens.

Für die Kreiszahl π können Sie das Modul `math` importieren. Dies stellt Ihnen mit dem Befehl `math.pi` π in einer grossen Genauigkeit zur Verfügung.

```
1 # TODO: implementieren Sie hier Ihre Funktionen
```

8.2.2 Berechnung der kinetischen Energie eines Körpers

Die kinetische Energie eines Körpers berechnet sich nach der Formel

$$E_{kin} = \frac{1}{2} \cdot m \cdot v^2$$

wobei m für die Masse des Körpers und v für dessen Geschwindigkeit steht. Implementieren Sie eine Funktion für die Berechnung der kinetischen Energie eines Körpers. Beachten Sie dabei, dass experimentell lediglich Daten über die Masse des Körpers, die zurückgelegte Strecke des Körpers sowie die dafür erforderliche Zeit zugänglich sind.

```
1 # TODO: implementieren Sie hier Ihre Funktionen
```

8.3 Musterlösungen

8.3.1 Berechnung des Volumens eines Zylinders

```
1 import math
2
3 def kreisflaeche(r : float) -> float:
4     area = math.pi * r ** 2
5     return area
6
7 def zylindervolumen(r : float, h : float) -> float:
8     bodenflaeche = kreisflaeche(r)
9     volumen = bodenflaeche * h
10    return volumen
```

8.3.2 Berechnung der kinetischen Energie eines Körpers

```
1 def geschwindigkeit(s: float, t: float) -> float:
2     """Berechnet die Geschwindigkeit v aus zurückgelegter Strecke und
3     Zeit.
4
5     Args:
6         s: Die zurückgelegte Strecke (in Metern).
7         t: Die dafür benötigte Zeit (in Sekunden).
8
9     Returns:
10        Die berechnete Geschwindigkeit v (in m/s).
11    """
12    v = s / t
13    return v
14
15
16 def kinetische_energie(m: float, s: float, t: float) -> float:
17     """Berechnet die kinetische Energie eines Körpers basierend auf
18     experimentellen Daten.
19
20     Die Funktion berechnet zuerst die Geschwindigkeit v aus s und t
21     mithilfe der Funktion 'geschwindigkeit' und wendet dann die Formel
22      $E_{\text{kin}} = 0.5 * m * v^2$  an.
23
24     Args:
25         m: Die Masse des Körpers (in Kilogramm).
```



```
26         s: Die zurückgelegte Strecke (in Metern).
27         t: Die dafür benötigte Zeit (in Sekunden).
28
29     Returns:
30         Die berechnete kinetische Energie E_kin (in Joule).
31     """
32     v = geschwindigkeit(s, t)
33     e_kin = (m * v ** 2) / 2
34     return e_kin
```

9 Datenstrukturen: Listen

9.1 Daten in Listen speichern

Bisher hatten Sie Datentypen wie `int`, `float`, `str` und `bool` angeschaut. Wir möchten nun mehrere Werte in einer Liste speichern.

Ich möchte zum Beispiel alle Wochentagekürzel in einer Liste speichern. Um eine Liste im Code zu speichern, muss man die Liste mit `[` beginnen und mit `]` enden. Die Elemente werden mit einem Komma `,` getrennt.

```
1 wochentage = ["Mo", "Di", "Mi", "Do", "Fr", "Sa", "So"]
2 print(wochentage)
```

9.1.1 Aufgabe: Monatsliste erstellen

Machen Sie eine Liste mit den 12 Monaten und printen Sie die Monate.

```
1 """Aufgabe: Monatsliste erstellen"""
2
3 monate = # TODO
4 print(monate)
```

9.2 Index

Die Werte werden durchnummeriert. Die dazugehörige Nummer eines Werts nennen wir den *Index*.

Das erste Element hat den Index 0!

Um ein Element aus der Liste auszugeben können wir nach dem Variablennamen die eckigen Klammern und den Index angeben.

`wochentage[0]` ist `"Mo"`. `wochentage[1]` ist `"Di"`.

9.2.1 Aufgabe: Index lesen

Lesen Sie den Codeblock und schreiben Sie hier auf, was Sie als Output erwarten, *bevor* Sie den Codeblock ausführen!

Ich erwarte, dass das Programm folgendes ausgibt:

TODO: Diese Zeile ersetzen.

```
1  liebblingstag = wochentage[3]
2  print(f"Mein Lieblingstag ist {liebblingstag}.")
```

9.3 Überschreiben von Elementen

Wir betrachten in diesem Kapitel eine Liste mit Zahlen (`int`). Wir speichern für jeden Monat die Länge des Monats. Im Falle eines Schaltjahrs möchten wir den Februar überschreiben mit 29. Studieren Sie das Beispiel:

```
1  monatslaenge = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
2  schaltjahr = True
3  if schaltjahr:
4      monatslaenge[1] = 29 # Der Februar hat den Index 2.
5  print(monatslaenge)
```

9.4 Schleifen

Listen eignen sich hervorragend um eine for-Schleife zu bauen. Betrachten Sie das Beispiel, welches den Index als `i` und den Wochentagsname als `wochentag` ausgibt. Die Liste `wochentage` möchten wir auf keinen Fall überschreiben. Also verwenden wir für die *Laufvariable* andere Variablennamen. In diesem Fall nehmen wir `i` für den Index und `wochentag` (Singular) für den Name des Tages.

```
1  for i, wochentag in enumerate(wochentage):
2      print(f"{i}: {wochentag}")
```

Falls Sie den Index nicht benötigen, können Sie die Schleife mit `for wochentag in wochentage:` machen.

9.4.1 Aufgabe: Monatsschleife komplettieren

Erstellen Sie eine Schleife, welche folgendes Ausgibt:

Der Januar hat 31 Tage.
Der Februar hat 29 Tage.
Der März hat 31 Tage.
Der April hat 30 Tage.
Der Mai hat 31 Tage.
Der Juni hat 30 Tage.
Der Juli hat 31 Tage.
Der August hat 31 Tage.
Der September hat 30 Tage.
Der Oktober hat 31 Tage.
Der November hat 30 Tage.
Der Dezember hat 31 Tage.

```
1  """Aufgabe: Monatsschleife komplettieren"""
2
3  for ... m_name in ...:
4      laenge = ... i ...
5      print(f"Der {m_name} hat {laenge} Tage.")
```

9.5 Länge

Mit der Funktion `len(obj)` kann man die Länge der Liste `obj` abfragen.

```
1  """Beispiel für len(obj)"""
2  print(f"Die Woche hat {len(wochentage)} Tage.")
```

9.6 Ergänzen und entfernen

Mit der Methode `append` kann ein Element am Ende der Liste hinzufügen und mit `pop` wieder entfernen.

```
1  tasks = ["einkaufen", "spazieren gehen", "staubsaugen"]
2  tasks.append("essen")
3  print(f"Neu ist auch 'essen' auf der Taskliste: {tasks}")
4
5  tasks.pop(1)
```

```

6 print(f"Welcher Task ist nun erledigt? {tasks}")
7
8 tasks.pop()
9 print("Wird kein Index angegeben, so wird das letzte Element"
10       f"in der Liste entfernt: {tasks}")

```

9.7 Aufgabe: Fibonacci

Die Fibonacci Reihe lautet 1, 1, 2, 3, 5, 8, 13, 8+13, ...

Die Summe der letzten zwei Zahlen ergeben die nächste Zahl.

Ergänzen Sie den Code, damit die Folge länger wird. Editieren Sie Stellen mit drei Punkten.

```

1 """Aufgabe: Fibonacci"""
2
3 fibonacci: list = [1, 1, 2, 3, 5, 8, 13] # Diese Zeile nicht verändern.
4 for _ in range(4):
5     n = ... # Die Länge von der Folge
6     fibonacci ... (fibonacci ... + fibonacci ...)
7     print(fibonacci)

```

9.8 Aufgabe: zeichne Chomp

Wie am Anfang der Lektion mündlich besprochen kodieren wir eine Situation von [Chomp](#).



Abbildung 9.1: Chomp_gameplay

Die dritte Situation beschreiben wir als Liste [4, 3, 3, 3, 2].

Schreiben Sie eine Funktion `zeichne_chomp(chomp: list) -> None` welche die Situation zeichnet.

```

1 """Aufgabe: zeichne Chomp"""
2 from pytamaro.de import *

```

```

3
4 def zeichne_chomp(chomp: list) -> None:
5     zelle = kombiniere(
6         rechteck(68, 38, rgb_farbe(153, 103, 69)),
7         rechteck(70, 40, rgb_farbe(51, 34, 23)),
8     )
9     leere_zelle = rechteck(70, 40, weiss)
10    tafel = leere_grafik()
11    maximum = # TODO # Groesse der erste Spalte
12    for ... in chomp:
13        spalte = leere_grafik()
14        ...
15        ...
16        tafel = neben(tafel, spalte)
17    zeige_grafik(tafel)

```

```

1 # Test für die Aufgabe
2 chomp_test: list = [4, 3, 3, 3, 2]
3 zeichne_chomp(chomp_test)

```

9.9 Aufgabe: spiele Chomp

Schreiben Sie eine Funktion `chomp_spielzug(chomp: list, spielzug: list) -> list` welche aus der ersten Situation die neue Situation berechnet. Der Spielzug ist eine Liste von der Form `[x, y]`.

Test: `chomp_spielzug([4, 3, 3, 3, 2], spielzug=[3, 2])` ergibt `[4, 3, 1, 1, 1]`

Bonusidee: Überprüfen Sie, ob der Spielzug überhaupt erlaubt ist.

```

1 """Aufgabe: programmiere den Spielzug in Chomp"""
2
3 # TODO: Programmieren Sie es selber!
4
5
6 # Hier können Sie den Code testen:
7 assert chomp_spielzug([4, 3, 3, 3, 2], spielzug=[3, 2]) == [4, 3, 1, 1,
    ↪ 1]

```

9.10 Übersicht

Sie wissen, wie Sie

- eine Liste erstellen.
- Elemente abfragen.
- Elemente ändern.
- mit einer Schleife durch die Liste gehen.
- die Länge einer Liste bestimmen.
- Elemente hinzufügen oder entfernen.

9.11 Zusatzstoff

Weitere Infos und Features von Listen auf <https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range>

Stichwörter: Slice, Spezialfall str, insert, in, join

10 Datenstrukturen: Dictionary

Dictionaries (Wörterbücher oder Maps) können mehrere Werte in einem Objekt speichern, wie es Listen auch können. Die Werte haben aber keinen Index sondern einen Schlüssel.

Ein *Dictionary* in Python ist vergleichbar mit einem echten Wörterbuch. Stellen Sie sich vor, Sie suchen die Telefonnummer (Wert) einer Person (Schlüssel). In einem Python-Dictionary könnten Sie den Namen der Person als Schlüssel und ihre Telefonnummer als Wert speichern.

```
1  """Beispiel von einem dict"""
2  telefonbuch = {
3      "Feuerwehr": 118,
4      "Polizei": 117,
5      "Ambulanz": 144,
6      "Rega": 1414,
7      "Beratung + Hilfe 147": 147,
8      "allgemeiner Notruf": 112,
9  }
```

10.1 Operationen für Dictionaries

- Erstellen eines Dictionaries: `dict_name: dict = {}`
Erstellt ein neues, leeres Dictionary mit dem Variablennamen *dict_name*.
- Elemente aktualisieren: `dict_name['Schlüssel'] = 'Neuer Wert'`
Aktualisiert den Wert eines bestehenden Schlüssels oder fügt einen neuen Schlüssel mit dem angegebenen Wert hinzu, wenn er nicht existiert.
- Elemente aufrufen: `var: str = dict_name['Schlüssel']`
`var` ist ein `str` mit dem Wert `'Neuer Wert'`. Man kann den Wert auch direkt zum `printen` oder anderes brauchen.
- Elemente löschen: `dict_name.pop('Schlüssel')`
Entfernt das Element mit dem angegebenen Schlüssel aus dem Dictionary.

- Durch das Dictionary durchgehen: `for key, value in dict_name.items()`

Dieser Prozess heisst *Iterieren*. Wir kennen ihn schon von der Liste oder Range. Iterieren heisst die Liste von Anfang bis Ende durchgehen. Jedes Mal wird in die Variablen `key`, `value` den Schlüssel, respektive den Wert des Eintrags zugewiesen. Sie können den Variablen auch andere Namen geben.

10.1.1 Aufgabe: Vervollständigen Sie das dict

```

1  """Aufgabe: dict vervollständigen"""
2
3  voci = {
4      "und": "and",
5      "Auto": ... ,
6      "Katze": ... ,
7      "Tag": ... ,
8      "Ende": ... ,
9      "Familie": ... ,
10     "Zuhause": ... ,
11     "Name": ... ,
12     "Menschen": ... ,
13     "lesen": ... ,
14     "Schule": ... ,
15     "sprechen": ... ,
16 }
17
18 # Test
19 print(voci["Schule"])

```

10.1.2 Aufgabe: Schreiben Sie ein dict

Schreiben Sie ein `dict` für Ihre Klasse, sodass der Nachname dem Schlüssel/key und der Vorname dem Wert/value entspricht.

```

1  """Aufgabe: schreiben Sie ein dict"""
2  klasse = ...

```

10.1.3 Aufgabe: Schleife

Iterieren Sie durch die ganze Liste, sodass alle aus der Klasse mit "Hallo [Vorname] [Nachname]" begrüsst werden.

```

1  """Aufgabe: Schleife"""
2  for nachname, vorname in ... :
3      print(f"Hallo {vorname} {nachname}")

```

10.1.4 Aufgabe: Selektion

Geben Sie nur die Nachnamen aus, welche ein **e** im Nachnamen haben.

Tipp: Verwenden Sie `if 'e' in nachname:`

```

1  """Aufgabe: Selektion"""
2
3  ...

```

10.1.5 Aufgabe: Telefonliste

Erstellen Sie eine Liste `threedigitslist`, die nur die Namen der Notfallorganisationen aus dem dict `telefonbuch` welche genau 3 Ziffern beinhalten.

```

1  """Aufgabe: Telefonliste"""
2
3  ...
4
5
6  # Test
7  assert type(threedigitslist) == list
8  assert len(threedigitslist) == 5
9  for souldtion in ['Feuerwehr', 'Polizei', 'Ambulanz', 'Beratung + Hilfe
   ↪ 147', 'allgemeiner Notruf']:
10     assert souldtion in threedigitslist

```

11 Beurteilung von Algorithmen

Die Effizienz eines Algorithmus kann grundsätzlich nach seinem Rechenaufwand oder Speicherbedarf beurteilt werden. Man spricht in diesem Zusammenhang auch von Zeit- bzw. Platzkomplexität.

Wir haben uns mit dem Zählen von Vergleichen nur mit der Rechenaufwand, also mit der Zeitkomplexität beschäftigt. Die Berechnungen werden sehr stark vereinfacht.

11.1 Bubblesort

Wenn wir mit dem Bubblesort n Objekte sortieren, haben wir $n - 1$ Vergleiche im ersten Durchgang. Vereinfacht sagen wir einfach n Vergleiche. Im schlimmsten Fall, müssen ist das kleinste Element am falschen Ende. Bei jedem Durchgang verschieben wir das Element nur um einen Platz. Dass heisst wir müssen etwa n Durchgänge machen. Die Laufzeit vom Bubblesort beträgt somit $n \cdot n = n^2$.

11.2 Mergesort

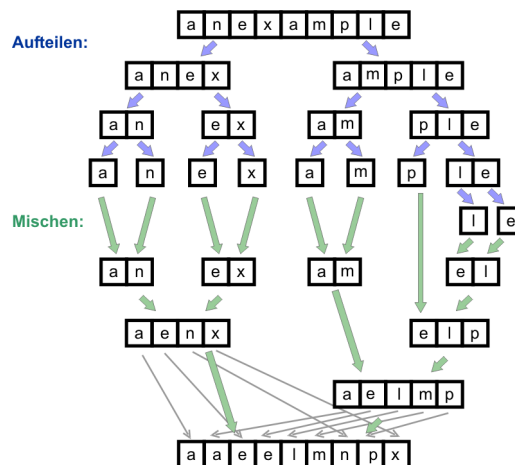


Abbildung 11.1: Mergesort_example

Bevor wir uns die Laufzeit von Mergesort anschauen, müssen wir quantifizieren, wie oft wir die Listen aufteilen müssen. Haben wir 2 Elemente, separieren wir 1 Mal. Haben wir 4 Elemente separieren wir 2 mal.

len	Teilungen
2	1
4	2
8	3
16	4
32	5
64	6
1024	10

Diese Funktion, welche der Länge zur Anzahl Teilungen zuordnet, nennt man *Logarithmus zur Basis 2*. Man notiert sie wie folgt: $\log_2(n)$

Auf jeder der $\log_2(n)$ Ebene machen wir etwa n Vergleiche. Die Laufzeit vom Mergesort beträgt somit $n \cdot \log_2(n)$.

12 Divide and Conquer

Das Prinzip *divide and conquer* (teile und herrsche) wurde am Beispiel von Mergesort gezeigt, aber nicht explizit erwähnt. Dieses Prinzip basiert auf der Idee, dass kleine Probleme einfacher zu lösen sind als grosse. Deshalb versucht man, das zu lösende Problem in Teilprobleme aufzuteilen und diese, kleineren Probleme, zu lösen um anschliessend die Teillösungen zu einer Lösung für das Ursprungsproblem zusammenzusetzen.

Diese Idee wird oft durch *Rekursion* umgesetzt. Der Begriff Rekursion kann am besten am Beispiel einer Kindergeschichte erklärt werden:

Es isch e mal en Maa gsi, de hät en hole Zah gha. I dem Zah häts es Truckli gha und i däm Truckli häts es Briefli gha. I dem Briefli isch gstande, es isch e mal en Maa gsi, de hät en hole Zah gha. I dem Zah häts es Truckli gha und id däm Truckli häts es Briefli gha...

Rekursive Funktionen sind entsprechend Funktionen, die sich selber aufrufen. Mergesort rief sich selber auf, um die Liste zu sortieren.

12.1 Fibonacci-Folge

Die [Fibonacci-Folge](#) - benannt nach dem Italienischen Mathematiker [Leonardo Fibonacci](#) - ist eine Zahlenfolge, in welcher die nächste Zahl die Summe der beiden vorangegangenen Zahlen bildet. In Zahlen sieht das folgendermassen aus:

1, 1, 2, 3, 5, 8, 13, ...

Die Fibonacci-Folge wird rekursiv definiert:

$$f_n = f_{n-1} + f_{n-2}, \text{ sofern } n \geq 3$$

Sie haben die Fibonacci-Folge bei den Listen bereits programmiert. Obwohl die Programmierung als Liste *viel* effizienter ist, machen wir hier eine Übung, Fibonacci als rekursive Funktion zu programmieren:

```
1  """Beispiel: fibonacci rekursiv definiert mit Printstatement"""
2  def fibonacci(n: int) -> int:
3      print(f"Es wird fibonacci({n}) ausgeführt.")
4      if n < 2:
```

```

5         return 1
6     return fibonacci(n-1) + fibonacci(n-2)

```

12.1.1 Aufgabe: Fibonacci-Folge

1. Lesen Sie die Funktion `fibonacci`. Denken Sie im Kopf durch, was passiert, wenn Sie `fibonacci(0)` oder `fibonacci(1)` ausführen. Schreiben Sie Ihre Vermutung auf und führen Sie erst dann die Funktion aus. Vergleichen Sie Ihre Vermutung mit dem Resultat.
2. Denken Sie im Kopf durch, was passiert, wenn Sie `fibonacci(2)` ausführen. Schreiben Sie Ihre Vermutung auf und führen Sie erst dann die Funktion aus. Vergleichen Sie Ihre Vermutung mit dem Resultat.
3. Bonus: Denken Sie im Kopf durch, was passiert, wenn Sie `fibonacci(3)` ausführen. Schreiben Sie Ihre Vermutung auf und führen Sie erst dann die Funktion aus. Vergleichen Sie Ihre Vermutung mit dem Resultat.

12.2 Endlose Rekursion

Wenn wir bei der Funktion `fibonacci` die Zeilen

```

1     if n < 2:
2         return 1

```

weglassen würden, so würde die Funktion endlos sich selber aufrufen, wie im Beispiel der Kindergeschichte.

Meistens wird dieses Problem mit Basisfälle gelöst.

12.3 Beispiel: Anstossen

Gestern assen wir mit 6 Personen zu Abend. Vorgestern mit 140 Personen. Wie oft schlagen die Gläser zusammen wenn alle miteinander anstossen?

Die Aufgabe können wir einfach reduzieren. Wenn ich die 6. Person am Tisch bin, stosse ich mit 5 Personen an. Die 5 Personen müssen untereinander noch anstossen. Das heisst $a_n = a_{n-1} + (n - 1)$.

Für die Rekursive Funktion, brauchen wir somit nur noch die Basisfälle. In unserem Beispiel ist es: Wenn nur 0, 1 oder 2 Personen da sind, wie oft wird angestossen?

Personen	Anstossen
0	0

Personen	Anstossen
1	0
2	1

```

1  """Aufgabe: finden Sie 2 Fehler"""
2  def anstossen(n: int) -> int:
3      ## Basisfälle
4      if n <= 1:
5          return 0
6      if n == 2:
7          return 2
8
9      return anstossen(n-1) + (n + 1)

1  """Test"""
2  for i in range(10):
3      print(f"{i} Personen stossen {anstossen(i)} Mal an.")
4
5  i = 140
6  print(f"{i} Personen stossen {anstossen(i)} Mal an.")

```

12.3.1 Bonus: Die Gaussche Summenformel

Der Deutsche Mathematiker Carl Friedrich Gauss soll nach der Legende seinen Mathematiklehrer bei einer Strafaufgabe überlistet haben. Gemäss dieser Legende soll der Lehrer Gauss (und seinen Klassenkameraden) den Auftrag geben haben, die Zahlen von 1 bis 100 zusammenzuzählen.

Gauss soll nach kurzer Zeit mit der Lösung zum Lehrer gegangen sein. Gauss' Lösung basierte auf folgender Formel:

$$1 + 2 + 3 + \dots + (n-1) + n = \sum_{k=1}^n k = \frac{n(n+1)}{2}$$

Diese Formel kann direkt als Funktion implementiert werden. Das entsprechende Beispiel finden sie in der folgenden Zelle.

```

1  def gauss_direct(n : int) -> int:
2      return int((n*(n+1))/2)
3
4  print(f'Die Summe der Zahlen von 1 '

```

```
5 f'bis 100 ist {gauss_direct(100)}.'
```

Alternativ kann die Formel aber auch rekursiv implementiert werden. Entsprechend stellt sich die Frage, was wäre ein kleineres, aber grundsätzlich gleichartiges Problem?

Im vorliegenden Fall wäre das einfachste gleichartige Problem, die Summe aus einer Liste von Zahlen mit der Länge 1 und dem Wert 1 zu bilden. Dann ist die Summe auch 1. Als Formel könnte man das so schreiben:

$$\sum_{k=1}^n k = 1, \text{ sofern } n = 1$$

Daraus ergibt sich die Frage, wie man von $n = 100$ zu $n = 1$ kommt.

Für alle Fälle, in denen $n > 1$ ist, gilt

$$\sum_{k=1}^n k = n + \sum_{k=1}^{n-1} k$$

In dieser Formel kann nun immer wieder $n - 1$ eingesetzt werden. Das ist die Rekursion.

Zusammengefasst kann dies folgendermassen dargestellt werden:

$$\sum_{k=1}^n k = \begin{cases} 1, & n = 1 \quad \text{Basisfall} \\ n + \sum_{k=1}^{n-1} k, & \forall n > 1 \quad \text{Rekursionsfall} \end{cases}$$

Der Basisfall ist einerseits der einfachste Fall und andererseits auch der letzte Fall, der bearbeitet werden muss.

Beides ist wichtig. Dass der Basisfall der einfachste Fall ist, hilft das Problem zu lösen. Dass der Basisfall der Letzte Fall ist, der bearbeitet werden muss, ermöglicht es, die Rekursion zu beenden.

Im Rekursionsfall steht “ $\forall n > 1$ ”. Das $\forall$ Zeichen ist der sogenannte Allquantor und bedeutet hier “**für alle** n grösser als 1”.

Dass eine Problemlösung rekursiv implementiert wird, bedeutet, dass die Funktion sich selber aufruft.

In der folgenden Zelle wird die gaussssche Summenformel gemäss obiger Darstellung rekursiv implementiert.

```
1 """Bonusaufgabe: Gauss Rekursiv"""
2 def gauss_recursive(n : int) -> int:
3     # TODO: rekursive Implementation des kleinen Gauss
4     pass
5
6 print(f'Die Summe der Zahlen von 1 ')
```



```
7         f'bis 100 ist {gauss_recursive(100)}.'
```

12.4 Fakultät

Mit Fakultät ($n!$) wird in der Mathematik jene Funktion bezeichnet mit der alle natürlichen Zahlen $\leq n$ miteinander multipliziert werden.

$$n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n = \prod_{k=1}^n k$$

Als Besonderheit muss man wissen, dass $0! = 1$ gilt.

In Python kann dies mit einer `for` Schleife berechnet werden. In der folgenden Zelle wird eine entsprechende Funktion definiert.

```
1  def factorial_loop(n : int) -> int:
2      """result wird gleich 1 gesetzt,
3      weil 1 das neutrale Element der
4      Multiplikation ist """
5      result = 1
6      for i in range(1, n+1):
7          result *= i
8
9      return result
10
11 print(f'Das Resultat von 5! ist '
12       f'{factorial_loop(5)}')
```

Die Funktion kann allerdings auch rekursiv definiert werden. Die Definition sieht dann folgendermassen aus:

$$n! = \begin{cases} 1, & n = 0 \vee n = 1 \quad \text{Basisfall} \\ n \times (n-1)!, & \forall n > 1 \quad \text{Rekursionsfall} \end{cases}$$

Entsprechend kann eine Funktion zur Berechnung von $n!$ auch rekursiv implementiert werden.

```
1  """Aufgabe: Fakultät"""
2  def factorial_recursive(n : int) -> int:
3      # TODO: rekursive Implementation der Fakultätsfunktion
4      pass
5
```

```

6 print(f'Das Resultat von 5! ist '
7       f'{factorial_recursive(5)}'.')

```

12.5 Wörter umdrehen

Gesucht ist eine rekursive Funktion, welche ein Wort rückwärts schreibt (bsp. Kantonsschule nach eluhcssnotnaK).

```

1 """Aufgabe: umdrehen"""
2 def reverse(s : str) -> str:
3     if len(s) == 0 or len(s) == 1:
4         return ...
5     else:
6         head = s[0]
7         tail = s[1:]
8         return ...

```

```

1 print(reverse('Kantonsschule'))

```

12.6 Bonus: Anzahl Möglichkeiten zu Zahlen

Ich Verkaufe eine Heft für 6 CHF. Wie viele Möglichkeiten gibt es 5 CHF in Münzen (wir betrachten nur die Münzen 1, 2, 5 CHF).

- $5 + 1$
- $2 + 2 + 2$
- $2 + 2 + 1 + 1$
- $2 + 1 + 1 + 1 + 1$

Es gibt 4 Möglichkeiten.

Um die Frage zu beantworten, kann man die Frage auch aufteilen. Die grösste mögliche Münze ist 5. Also gibt es die Möglichkeiten mit einem 5 Fränkler zu bezahlen (1 Möglichkeit) oder ohne keinem 5 Fränkler zu bezahlen (3 Möglichkeiten). Diese Anzahl Möglichkeiten zusammen addiert ergibt die Möglichkeiten. Die Funktion benötigt 2 Argumente: Den zu bezahlenden Betrag und die grösste erlaubte Münze.

```

1 """
2 Bonusaufgabe

```

```

3 Studieren Sie die Funktion und füllen Sie die Lücken mit den
  ↳ Auslassungspunkten.
4 """
5 MUENZEN = [1000, 200, 100, 50, 20, 10, 5, 2, 1]
6 def anz_zahlmöglichkeiten(
7     value: int,
8     largest_index: int=0,
9     ) -> int:
10     """Rekursive Funktion"""
11     if value < 0:
12         return 0
13     if value == 0:
14         return 1
15     if largest_index + 1 == len(MUENZEN):
16         # Pay with smaless coins
17         return 1
18     # Pay with largest Coin
19     ret = ...
20
21     # Pay without larges coin
22     ret += ...
23     return ret
24
25 print(anz_zahlmöglichkeiten(5))

```

```

1 print(anz_zahlmöglichkeiten(500)) # 6295434
2 # print(anz_zahlmöglichkeiten(700)) # 40208370
3 # print(anz_zahlmöglichkeiten(1000)) # 321335887

```

```

1 """
2 Bonusaufgabe
3 Printen Sie die Möglichkeiten!
4
5 [5]
6 [2, 2, 1]
7 [2, 1, 1, 1]
8 [1, 1, 1, 1, 1]
9 """
10 MUENZEN = [1000, 200, 100, 50, 20, 10, 5, 2, 1]
11 def print_zahlmöglichkeiten(...)
12     ...
13
14 print_zahlmöglichkeiten(5)

```

13 Prüfungsvorbereitung 2

13.1 Lernziele

Ich erwarte, dass Sie in der Lage sind

- eine Jupyter Arbeitsumgebung zu starten (auswendig);
- Python Listen zu erstellen und zu verändern;
- Werte aus Python Listen direkt oder mit Hilfe von `for` Schleifen zu verarbeiten;
- Python Dictionaries zu erstellen und zu verändern;
- Wert aus Python Dictionaries direkt oder mit Hilfe von `for` Schleifen zu verarbeiten sowie
- rekursive Funktionen zu implementieren.

Die Inhalte der letzten Prüfung werden als Grundlagen vorausgesetzt. Die Prüfung findet mit Papier und Bleistift statt. Als Hilfsmittel dürfen Sie eine handschriftliche Zusammenfassung im Umfang einer Seite A4 mitbringen. Die Zusammenfassung darf alles ausser die Schritte zum Start einer Jupyter Arbeitsumgebung enthalten.

13.2 Listen

13.2.1 Listen erstellen

Das Fähnlein Fieselschweif besteht aus Donald Ducks Neffen Tick, Trick und Track. Erstellen Sie eine Liste `fieselschweif` mit den drei Mitgliedern.

```
1 # TODO: Liste erstellen
```

13.2.2 Listen bearbeiten

Ergänzen Sie die vorher erstellte Liste 'fieselschweif' um den Namen *Dagobert*.

```
1 # TODO: fieselschweif ergänzen
```

Dagobert gefällt es nicht im Fähnlein Fieselschweif. Entfernen Sie Ihn wieder und lassen Sie Ihren Code den Text *Dagobert hat das Fähnlein Fieselschweif wieder verlassen.* ausgeben. Dagobert soll dabei als Variabel `quitter` in den Code eingefügt werden.

```
1 # TODO: implementieren Sie die obige Vorgabe
```

13.2.3 Über Listen iterieren

Gegeben sei eine Liste `numbers`. Die Liste beinhaltet die natürlichen Zahlen von 1 bis 10 (`numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`). Erstellen Sie mit Hilfe einer `for` schleife basierend auf der Liste `numbers` eine neue Liste `triplet` welche aus den Elementen der Dreierreihe des kleinen Einmaleins besteht.

```
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2
3 # TODO: implementieren Sie die obige Vorgabe
```

13.2.4 Python Dictionary erstellen

Erstellen Sie ein Python Dictionary `mk`, welches die Geschäftsadresse von Jacques Mock Schindler enthält. Die ersten zwei Einträge erhalten Sie als Vorlage.

```
1 mk = {
2     'name': 'Mock Schindler',
3     'first_name': 'Jacques',
4     # TODO: kompletieren Sie das Dictionary
5 }
```

13.2.5 Über ein Python Dictionary iterieren

Geben Sie den Inhalt des Dictionary `mk` mit Hilfe einer `for` Schleife aus. Dabei soll die Ausgabe jedes Eintrags auf einer neuen Zeile erfolgen.

```
1 # TODO: implementieren Sie obige Vorgabe
```

13.2.6 Rekursion

Ein Palindrom ist ein Wort, das in beide Richtungen gelesen werden kann. Ein Beispiel für ein Palindrom ist *Sugus*. Schreiben Sie eine rekursive Funktion `palindrom_tester`, welche testet, ob eine gegebene Zeichenfolge ein Palindrom ist.

Hinweis: Die einzelnen Zeichen eines Strings können wie die Elemente einer Liste mit Hilfe eines Index abgerufen werden. Das erste Zeichen eines Strings hat dabei den Index 0.

Damit Sie einfacher prüfen können, ob Ihre Funktion korrekt arbeitet, übergeben Sie ihr ausschliesslich Strings in Kleinbuchstaben.

```
1 # TODO: implementieren Sie die obige Vorgabe
```

13.3 Musterlösungen

13.3.1 Listen erstellen

Das Fähnlein Fieselschweif besteht aus Donald Ducks Neffen Tick, Trick und Track. Erstellen Sie eine Liste `fieselschweif` mit den drei Mitgliedern.

```
1 fleselschweif = ['Tick', 'Trick', 'Track']
```

13.3.2 Listen bearbeiten

Ergänzen Sie die vorher erstellte Liste ‘fieselschweif’ um den Namen *Dagobert*.

```
1 fleselschweif.append('Dagobert')
```

Dagobert gefällt es nicht im Fähnlein Fieselschweif. Entfernen Sie Ihn wieder und lassen Sie Ihren Code den Text *Dagobert hat das Fähnlein Fieselschweif wieder verlassen.* ausgeben. Dagobert soll dabei als Variabel `quitter` in den Code eingefügt werden.

```
1 quitter = fleselschweif.pop()
2 print(f'{quitter} hat das Fähnlein Fieslschweif wieder verlassen.')
```

13.3.3 Über Listen iterieren

Gegeben sei eine Liste `numbers`. Die Liste beinhaltet die natürlichen Zahlen von 1 bis 10 (`numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`). Erstellen Sie mit Hilfe einer `for` schleife basierend auf der Liste `numbers` eine neue Liste `triplet` welche aus den Elementen der Dreierreihe des kleinen Einmaleins besteht.

```
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2
3 triplet = []
4
5 for number in numbers:
```

```
6 triplet.append(number * 3)
```

13.3.4 Python Dictionary erstellen

Erstellen Sie ein Python Dictionary `mk`, welches die Geschäftsadresse von Jacques Mock Schindler enthält. Die ersten zwei Einträge erhalten Sie als Vorlage.

```
1 mk = {  
2     'name': 'Mock Schindler',  
3     'first_name': 'Jacques',  
4     'street': 'Rosenstrasse 1',  
5     'postcode': 8400,  
6     'town': 'Winterthur',  
7 }
```

13.3.5 Über ein Python Dictionary iterieren

Geben Sie den Inhalt des Dictionary `mk` mit Hilfe einer `for` Schleife aus. Dabei soll die Ausgabe jedes Eintrags auf einer neuen Zeile erfolgen.

```
1 for key, value in mk.items():  
2     print(f'{key}: {value}')
```

13.3.6 Rekursion

Ein Palindrom ist ein Wort, das in beide Richtungen gelesen werden kann. Ein Beispiel für ein Palindrom ist *Sugus*. Schreiben Sie eine rekursive Funktion `palindrom_tester`, welche testet, ob eine gegebene Zeichenfolge ein Palindrom ist.

Hinweis: Die einzelnen Zeichen eines Strings können wie die Elemente einer Liste mit Hilfe eines Index abgerufen werden. Das erste Zeichen eines Strings hat dabei den Index 0.

Damit Sie einfacher prüfen können, ob Ihre Funktion korrekt arbeitet, übergeben Sie ihr ausschliesslich Strings in Kleinbuchstaben.

```
1 def palindrom_tester(text):  
2     """  
3     Prüft rekursiv, ob ein String ein Palindrom ist  
4     (vorwärts und rückwärts gelesen gleich).  
5     Beispiele: "Lagerregal", "Anna", "Reittier"  
6     """
```

```

7      # Vorverarbeitung: Kleinbuchstaben für einfacheren Vergleich
8      # (für den Fall, dass Sie sich nicht an den Tipp gehalten haben)
9      text = text.lower()
10
11     # Rekursionsanker 1: Leerer String oder Einzelbuchstabe
12     # ist immer ein Palindrom.
13     if len(text) <= 1:
14         return True
15
16     # Rekursionsanker 2: Wenn erster und letzter Buchstabe
17     # nicht übereinstimmen,
18     # kann es kein Palindrom sein. Abbruch mit False.
19     if text[0] != text[-1]:
20         return False
21
22     # Rekursionsschritt:
23     # Wenn die Ränder passen, prüfe den Teilstring ohne den
24     # ersten und letzten Buchstaben.
25     return palindrom_tester(text[1:-1])

```


Teil III

Data Representation

14 Number Representation

Counting has a long history, as does the representation of numbers. At the beginning of numerical recording, there was a one-to-one representation. This means that each object to be counted was represented by a symbol. This approach is not particularly efficient. Therefore, early in human history people started developing more compact representations of numbers.

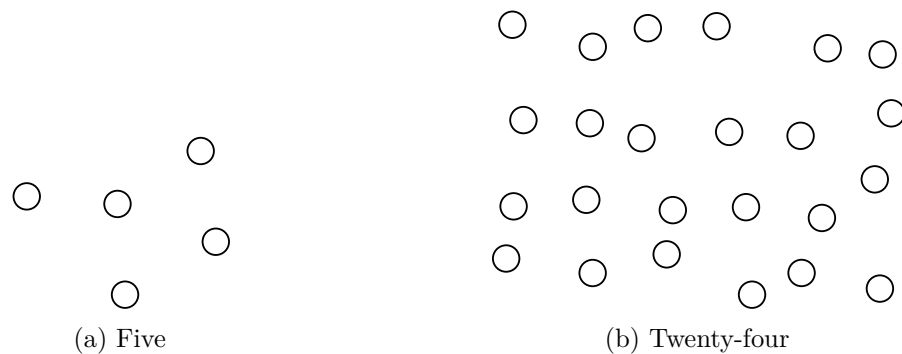


Abbildung 14.1: Objects to count

14.1 One-to-one Number Representation

In a one-to-one number representations the objects in [Abbildung 14.1a](#) could be represented by  the objects in [Abbildung 14.1b](#) by  respectively.

14.2 Symbolic Number Representation

While the Romans were not the pioneers of representing numerical values through symbols, their system remains the most prominent historical example. Consequently, the Roman method for denoting the quantity of objects is illustrated in figures [Abbildung 14.1a](#) (**V**) and [Abbildung 14.1b](#) (**XXIV**).

Roman numerals are generally intuitive up to the number twelve; however, representing larger values increases complexity significantly. In principle, each symbol within a Roman numeral possesses a fixed value. An exception occurs only when a symbol of lesser value precedes one of greater value, indicating subtraction.

Furthermore, Roman numerals require considerable space. The current year becomes *MMXXVI* an increase in space consumption by 50% compared to the decimal *2026*.

14.3 Number Representation and Positional Notation

14.3.1 Decimal Representation of Numbers

Unlike Roman numerals, decimal numerals possess both a **face value** and a **positional value**. For instance, the quantity of objects shown in Abbildung 14.1a is represented by the digit 5, signifying *five ones*. In Abbildung 14.1b, the quantity becomes 24, representing *two tens plus four ones*.

When expressed as a formal calculation:

$$5 = 5 \times 10^0 \quad \text{and} \quad 24 = 2 \times 10^1 + 4 \times 10^0$$

This distinction demonstrates the mechanics of a **positional numeral system**:

- The **face value** (the digit itself) acts as a coefficient.
- The **positional value** (determined by the digit's position) corresponds to a power of the base.

This differentiation between face and positional values enables a significantly more compact representation of numbers compared to additive symbolic systems such as Roman numerals.

14.3.2 Binary Representation of Numbers

A well-known joke in computer science circles goes:

There are 10 kinds of people:

- those who understand binary numbers, and
- those who don't.

Understanding binary numbers is essential to appreciating this joke.

While decimal numbers are represented in formal calculations as products of single digits and powers of the base 10, binary numbers use base 2. The quantities shown in Abbildung 14.1a and Abbildung 14.1b, when represented as binary numbers, are 101_2 and 11000_2 , respectively.

Expressed as formal calculations:

$$5_{10} = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 101_2$$

and

$$24_{10} = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 11000_2$$

One position in a binary number is called a *bit* (short for *binary digit*). Eight bits compose a *byte*, which is the smallest addressable unit of memory in most computer systems. For example, the ASCII code for the letter ‘A’ is stored in one byte: 01000001_2 .

14.3.3 Hexadecimal Numbers

Hexadecimal numbers use base 16_{10} . Therefore, the quantities shown in Abbildung 14.1a and Abbildung 14.1b become 5_{16} and 18_{16} , respectively.

Because the Arabic numerals used in the decimal system provide only ten digits (0–9), additional symbols are needed to represent hexadecimal numbers. The symbols chosen are the first six uppercase letters of the Latin alphabet. Thus, hexadecimal numbers are composed of the sixteen symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F.

These numbers expressed as a formal calculation:

$$5_{10} = 5 \times 16^0 = 5_{16}$$

$$24_{10} = 1 \times 16^1 + 8 \times 16^0 = 18_{16}$$

Each hexadecimal digit represents exactly four bits (also called a nibble). Hexadecimal numbers are widely used in computer science as the following examples demonstrate:

- Memory addresses: `0x7FFFFFFF`
- RGB color codes: `#FF5733` (red: FF, green: 57, blue: 33)
- Binary-to-hex conversion: Each hex digit represents exactly four bits (e.g., $F_{16} = 1111_2$).

14.4 Number Conversion

The following sections describe the algorithms for the conversion of numbers from one representation to the other.

14.4.1 Decimal to Binary Numbers

First, an empty string is initialized to represent the binary number. In a loop, the number n to be converted is processed using the modulo 2 operation. The result of this operation is prepended as the most significant bit (to the far left) of the existing binary string. Subsequently, the number n is halved using integer division, and the result of this division becomes the new value of n . This process is repeated until n reaches zero.

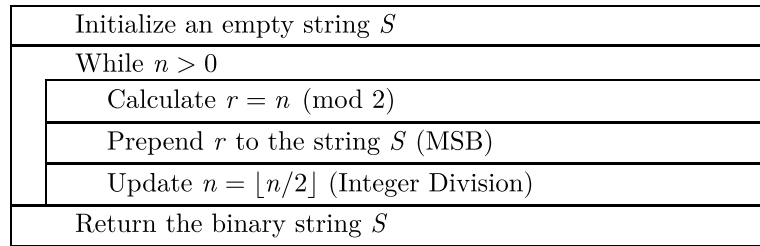


Abbildung 14.2: Nassi-Shneiderman diagram describing the algorithm (MSB = Most Significant Bit).

Task: Please implement the algorithm as described.

```

1 def decimal_to_binary(n: int) -> str:
2     # TODO: implement the aforementioned algorithm
3     pass

```

14.4.2 Binary to Decimal Conversion

To convert binary numbers to decimal numbers, the formal calculation described above must be expressed algorithmically.

1. Initialize a counter e to zero.
2. Initialize a result variable r to zero.
3. Extract the rightmost digit d from the binary number.
4. Multiply d by 2^e and add the result to r .
5. Increment the counter e by one.
6. Repeat steps 3–5 for each remaining digit, moving from right to left, until all digits have been processed.

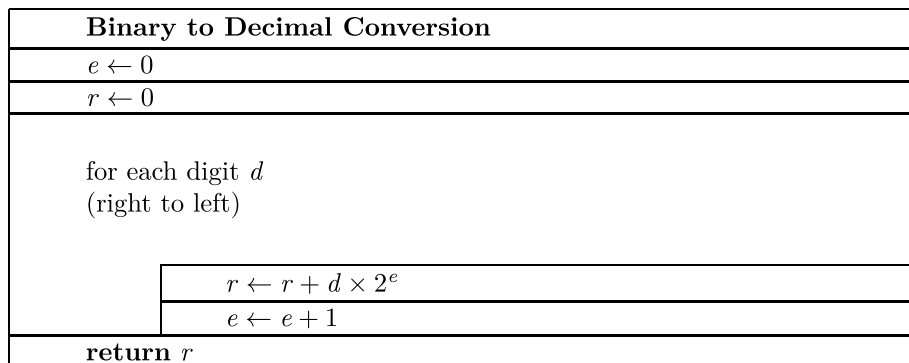


Abbildung 14.3: Nassi-Shneiderman diagram for the algorithm to convert binary numbers to decimal ones.

Task: Please implement the algorithm as described.

```
1 def binary_to_decimal(b: str) -> int:
2     # TODO: implement the aforementioned algorithm
3     pass
```

14.4.3 Decimal to Hexadecimal Conversion

To convert decimal numbers to hexadecimal numbers, the algorithm can be based on the binary conversion algorithm described earlier. However, hexadecimal representation requires additional symbols beyond 0–9. A lookup table maps decimal values to their hexadecimal equivalents, as shown in [Tabelle 14.1](#).

Tabelle 14.1: Conversion table: decimal to hexadecimal.

Dec	Hex	Dec	Hex
0	0	8	8
1	1	9	9
2	2	10	A
3	3	11	B
4	4	12	C
5	5	13	D
6	6	14	E
7	7	15	F

Using this table, the conversion algorithm proceeds as follows:

1. Initialize an empty string to represent the hexadecimal number.
2. While $n > 0$:
 - a. Compute $d = n \bmod 16$.
 - b. Look up the hexadecimal digit corresponding to d in [Tabelle 14.1](#).
 - c. Prepend this digit to the hexadecimal string.
 - d. Update n using integer division: $n \leftarrow \lfloor n/16 \rfloor$.
3. Return the hexadecimal string.

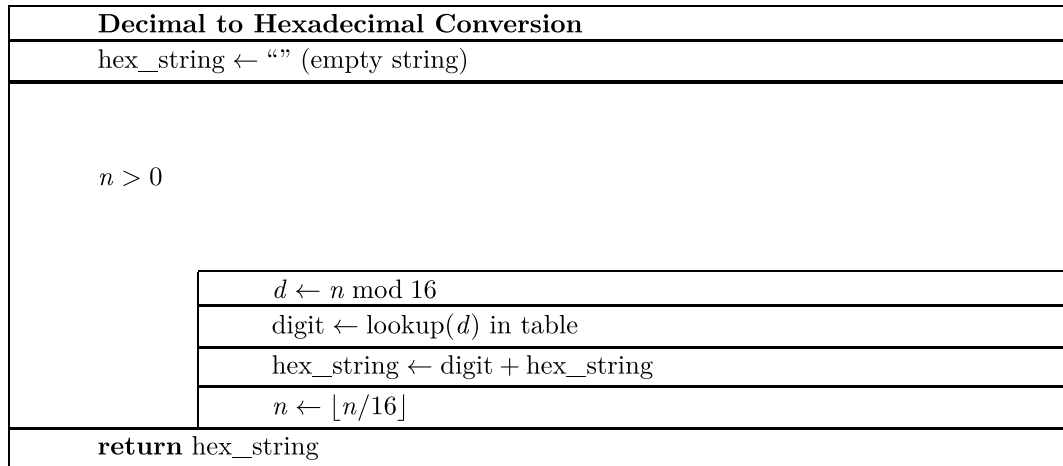


Abbildung 14.4: Nassi-Shneiderman diagram of the algorithm for the decimal to hexadecimal conversion.

Task: Please implement the algorithm as described.

```

1 def decimal_to_hexadecimal(d: int) -> str:
2     # TODO: implement the aforementioned algorithm
3     pass

```

14.4.4 Hexadecimal to Decimal Conversion

As with the conversion from decimal to hexadecimal, the reverse conversion requires only a small adaptation of the binary-to-decimal algorithm.

The formal calculation must account for the hexadecimal digit mapping shown in [Tabelle 14.1](#).

1. Initialize a counter e to zero.
2. Initialize a result variable r to zero.
3. Extract the rightmost digit d from the hexadecimal number.
4. Convert the digit d to its decimal equivalent using [Tabelle 14.1](#).
5. Multiply the decimal value by 16^e and add the result to r .
6. Increment the counter e by one.
7. Repeat steps 3–6 for each remaining digit, moving from right to left, until all digits have been processed.

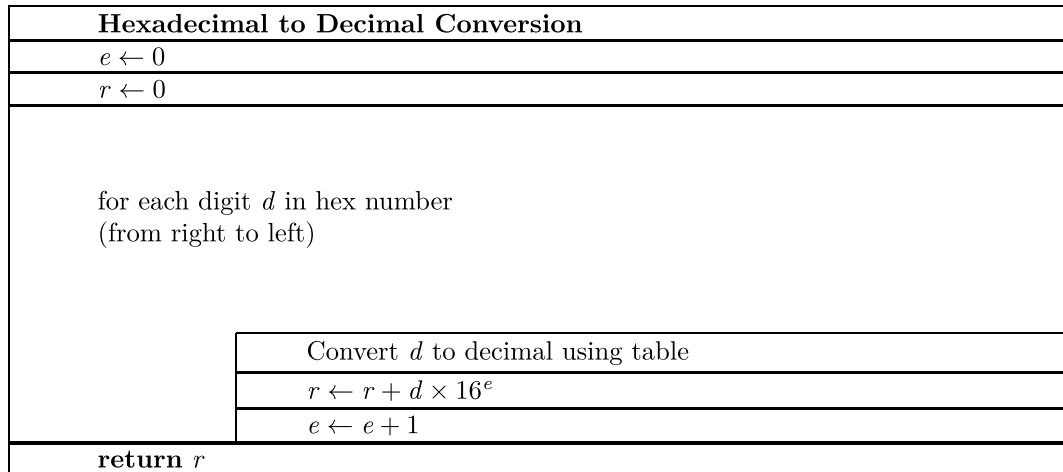


Abbildung 14.5: Nassi-Shneiderman diagram of the algorithm for the hexadecimal to decimal conversion.

Task: Please implement the algorithm as described.

```

1 def hexadecimal_to_decimal(h: str) -> int:
2     # TODO: implement the aforementioned algorithm
3     pass

```

14.4.5 Decimal to Roman Numeral Conversion

To convert decimal numbers to Roman numerals, a greedy algorithm is employed that repeatedly subtracts the largest possible Roman value from the number.

Roman numerals use seven basic symbols and six subtractive combinations, as shown in Tabelle 14.2.

Tabelle 14.2: Roman numeral values in descending order.

Decimal	Roman	Type
1000	M	Basic
900	CM	Subtractive
500	D	Basic
400	CD	Subtractive
100	C	Basic
90	XC	Subtractive
50	L	Basic
40	XL	Subtractive
10	X	Basic
9	IX	Subtractive

Decimal	Roman	Type
5	V	Basic
4	IV	Subtractive
1	I	Basic

Using this table, the conversion algorithm proceeds as follows:

1. Initialize an empty string to represent the Roman numeral.
2. Initialize an index i to point to the first entry in Tabelle 14.2 (largest value).
3. While the decimal number $n > 0$:
 - a. Let v be the decimal value at index i .
 - b. Let s be the Roman symbol(s) at index i .
 - c. If $n \geq v$:
 - Append symbol s to the Roman numeral string.
 - Subtract v from n : $n \leftarrow n - v$.
 - d. Otherwise:
 - Increment index i to move to the next smaller value.
4. Return the Roman numeral string.

Example: Convert 24_{10} to Roman numerals.

Step	n	i	v	s	Action	Result
1	24	0	1000	M	Skip	“ ”
2	24	1	900	CM	Skip	“ ”
3	24	2	500	D	Skip	“ ”
4	24	3	400	CD	Skip	“ ”
5	24	4	100	C	Skip	“ ”
6	24	5	90	XC	Skip	“ ”
7	24	6	50	L	Skip	“ ”
8	24	7	40	XL	Skip	“ ”
9	24	8	10	X	Append	“X”
10	14	8	10	X	Append	“XX”
11	4	8	10	X	Skip	“XX”
12	4	9	9	IX	Skip	“XX”
13	4	10	5	V	Skip	“XX”
14	4	11	4	IV	Append	“XXIV”
15	0	—	—	—	Done	“XXIV”

Result: $24_{10} = \text{XXIV}$

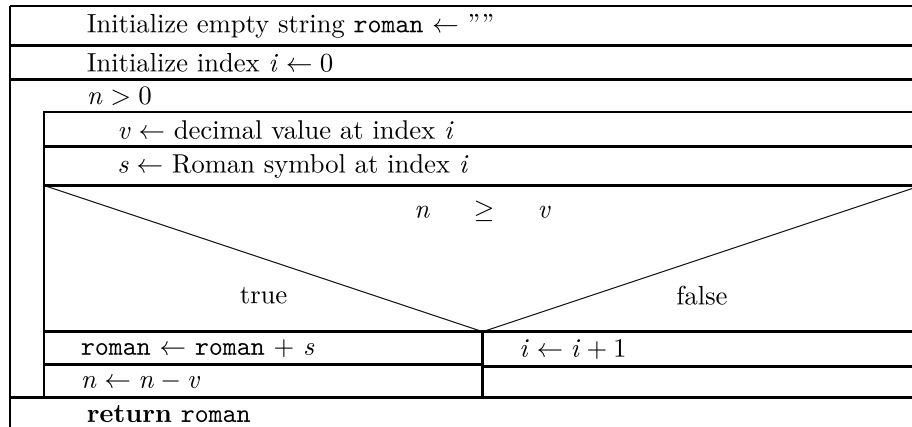


Abbildung 14.6: Nassi-Shneiderman diagram of the algorithm for decimal to Roman numeral conversion.

Task: Please implement the algorithm as described.

```

1  def decimal_to_roman(d: int) -> str:
2      # TODO: implement the aforementioned algorithm
3      pass

```

15 Image Representation in Computers

Images are composed of shapes and colours. We have observed this through our work with the PyTamaro library. However, this experience does not answer the question of how shapes and colours are represented in computer systems. While humans perceive colours visually, computers must encode them as numerical data.

Let us begin with the colour representation.

15.1 Colour Representation

To display a colour on a computer screen two aspects must be considered:

1. The colour model, and
2. the colour encoding.

15.1.1 The Colour Model

At a theoretical level, three distinct colour models exist: light-based additive colour models (RGB), pigment-based subtractive colour models (CMYK), and perceive oriented models (HSL/HSV).

For computer screens, the RGB model is predominant whilst for printers we use the CMYK model. The HSL/HSV model is used in image processing software. Because this is a computer science class, we will focus on the RGB model. The abbreviation RGB stands for red, green, and blue. If the beams of a red, a green, and a blue torch are directed at one spot, the colour of that spot will be white where the three beams overlap, as illustrated in Abbildung 15.1.

In computer screens, the torches are replaced by a grid of red, green, and blue LEDs. A group of three LEDs (one red, one green, one blue) forms a single pixel on the screen. Colours are created by varying the intensity of light emitted by each LED in the pixel group.

A typical computer screen LED can emit light at 256 different intensity levels, ranging from 0 (off) to 255 (maximum brightness). Since each pixel contains three LEDs, and each LED has 256 possible intensity levels, the total number of displayable colours is:

$$256 \times 256 \times 256 = 256^3 = 16,777,216$$

This is called **8-bit colour depth** (or **24-bit true colour**) because each of the three colour channels (red, green, blue) uses 8 bits to represent its 256 intensity levels. The term “24-bit” comes from the total: $8 \text{ bits} \times 3 \text{ channels} = 24 \text{ bits per pixel}$.

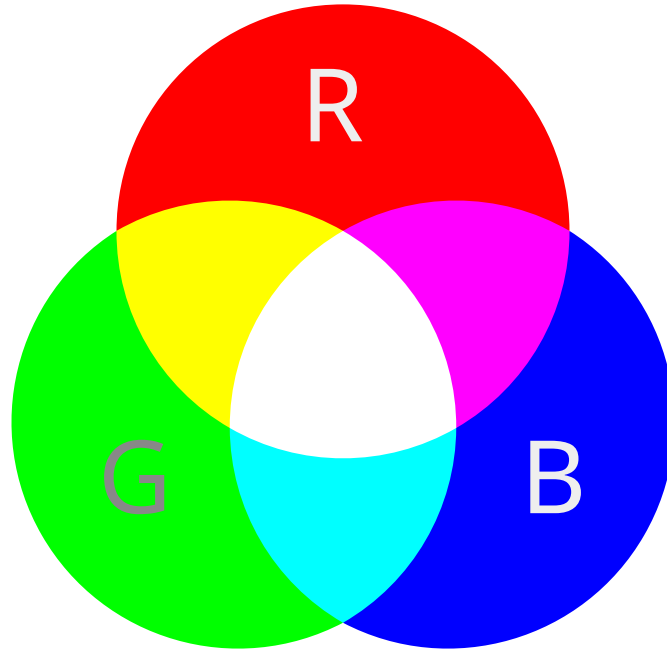


Abbildung 15.1: Additive RGB colour mixing

15.1.2 Colour Encoding

The above explanation leads to the question of how the numerical colour representation is encoded in computer systems.

RGB colour values can be represented in two common notations:

1. **Decimal RGB notation:** Three integers ranging from 0 to 255. For example, RGB(10, 255, 100) represents a vivid green.
2. **Hexadecimal notation:** A six-digit hexadecimal number preceded by #. For example, #0AFF64 represents the same vivid green.

In hexadecimal notation, each pair of hexadecimal digits represents one colour channel. The first two digits represent the red channel, the next two the green channel, and the last two the blue channel:

- #0AFF64 breaks down as:
 - 0A = 10 (Red)
 - FF = 255 (Green)
 - 64 = 100 (Blue)

This hexadecimal notation is preferred in web development and graphic design because it is compact and aligns with how colours are stored in computer memory (8 bits = 2 hexadecimal digits per channel).

15.2 Colour Table

```
1  from pytamaro import *
2
3  # TODO: Create an 8 × 8 RGB colour grid with PyTamaro
4  #
5  # Requirements:
6  # - Create a grid of coloured rectangles (8 rows, 8 columns)
7  # - Vary the red and green values across the grid
8  # - Keep blue at 0 for simplicity
9  # - Each cell should be 40×40 pixels
10 #
11 # Hints:
12 # - Use rgb_color(r, g, b) to create colours
13 # - Use rectangle(width, height, color) to create rectangles
14 # - Use beside() to place rectangles horizontally
15 # - Use above() to stack rows vertically
16 # - Calculate RGB values using: value = row * step or col * step
17 #   where step = 255 // 7 (to get 8 evenly-spaced values from 0 to 255)
```

16 The Concept of an Algorithm

In everyday life we constantly follow step-by-step procedures: cooking recipes, assembly instructions, or tying shoe laces. In computer science we formalise this idea with the concept of an **algorithm**.

16.1 Definition

An **algorithm** is a finite sequence of well-defined, unambiguous instructions that, given some input, produces an output in a finite number of steps.

16.2 Properties of an Algorithm

For a set of instructions to qualify as an algorithm, it must satisfy the following properties:

Property	Meaning
Finiteness	The algorithm terminates after a finite number of steps.
Determinism	Each step has exactly one well-defined successor.
Definiteness	Every instruction is precise and unambiguous.
Input	The algorithm receives zero or more inputs.
Output	The algorithm produces at least one result.
Effectiveness	Every step can actually be carried out (is computable).

16.3 Example: Tying Shoe Laces

An everyday example of an algorithm is tying shoe laces.

- **Input:** An untied shoe with two loose laces.
- **Output:** A tied shoe with a secure bow.
- **Finiteness:** The process ends once the bow is formed.
- **Definiteness:** Each step (cross laces, form loop, wrap around, pull through) can be described precisely.

- **Determinism:** Given the same starting position, the same steps always produce the same knot.
- **Effectiveness:** Every step involves a physical action that can be performed.

When you wrote instructions for your partner to tie a shoe, you experienced first-hand how important *definiteness* is: any vague step (“make a knot”) leads to confusion.

16.4 From Everyday Algorithms to Mathematical Algorithms

Algorithms are not limited to physical tasks. Many important algorithms operate on numbers. One of the oldest known algorithms is the **Euclidean algorithm** for computing the **Greatest Common Divisor (GCD)** of two integers.

The GCD of two integers is the largest positive integer that divides both numbers without a remainder.

Examples: - $\text{gcd}(12, 8) = 4$ (because 4 divides both 12 and 8) - $\text{gcd}(48, 18) = 6$ (because 6 divides both 48 and 18) - $\text{gcd}(56, 15) = 1$ (the numbers share no common factor other than 1)

16.5 The Euclidean Algorithm

The Euclidean algorithm computes the GCD by repeatedly replacing the larger number with the *remainder* of dividing the two numbers. The process continues until the remainder is zero. At that point, the other number is the GCD.

16.5.1 The Algorithm in Natural Language

1. Given two positive integers a and b .
2. Compute the remainder $r = a \bmod b$.
3. Replace a with b and b with r .
4. If $b = 0$, then a is the GCD. **Stop.**
5. Otherwise, go back to step 2.

16.5.2 Checking the Algorithm Properties

- **Input:** Two positive integers a and b .
- **Output:** Their greatest common divisor.
- **Finiteness:** The remainder strictly decreases with each step, so the algorithm always terminates.
- **Determinism:** Each step is uniquely determined (compute remainder, swap values).
- **Definiteness:** “Divide a by b , take the remainder” is unambiguous.
- **Effectiveness:** Integer division and remainder are basic computable operations.

16.5.3 Worked Example: gcd(48, 18)

Step	a	b	$b * q + r$	r
1	48	18	$18 * 2 + 12$	12
2	18	12	$12 * 1 + 6$	6
3	12	6	$6 * 2 + 0$	0

After step 3, $b = 0$, so the GCD is $a = 6$.

16.5.4 Worked Example: gcd(56, 15)

Step	a	b	$b * q + r$	r
1	56	15	$15 * 3 + 11$	11
2	15	11	$11 * 1 + 4$	4
3	11	4	$4 * 2 + 3$	3
4	4	3	$3 * 1 + 1$	1
5	3	1	$1 * 3 + 0$	0

After step 5, $b = 0$, so the GCD is $a = 1$. The two numbers are *coprime* (they share no common factor other than 1).

16.6 Implementing the Euclidean Algorithm in Python

Now it is your turn to translate the algorithm into Python.

Recall the algorithm:

1. Compute the remainder $r = a \bmod b$. (In Python: `a % b`)
2. Replace a with b and b with r .
3. If $b = 0$, return a (the GCD).
4. Otherwise, repeat from step 1.

Hint: You will need a `while` loop that runs as long as `b` is not zero.

Task: Complete the function below so that it returns the GCD of two positive integers.

```
1 def gcd(a: int, b: int) -> int:
2     # The Euclidean Algorithm
3     #
4     # Use a while loop that continues as long as b is not 0.
5     # In each iteration:
```



```

6      # 1. Compute the remainder of a divided by b (hint: use the %
      ↪ operator)
7      # 2. Set a to the old value of b
8      # 3. Set b to the remainder
9      # When b becomes 0, a holds the GCD – return it.
10
11     pass # TODO: replace this line with your implementation

```

16.6.1 Testing Your Implementation

Run the cell below to check whether your `gcd` function produces the correct results.

```

1  # Test cases – all should print True
2  print(gcd(48, 18) == 6)
3  print(gcd(56, 15) == 1)
4  print(gcd(12, 8) == 4)
5  print(gcd(100, 75) == 25)
6  print(gcd(17, 13) == 1)

```

16.7 Ways to Describe an Algorithm

In this notebook we described the Euclidean algorithm in three different ways:

Notation	Example	Pros	Cons
Natural language	“Divide a by b and take the remainder ...”	Easy to understand	Can be ambiguous
Table (trace)	Step-by-step table with values of a , b , r	Shows execution clearly	Only for specific inputs
Programming language	<code>while b != 0: ...</code>	Executable	Tied to one language

17 Algorithm Representation: Pseudocode & Struktogramme

Last week we described the GCD algorithm in three ways: natural language, a trace table, and Python code.

Today we add two more notations:

- **Pseudocode** — precise, structured, language-independent text
- **Struktogramme** (Nassi–Shneiderman diagrams) — visual block diagrams

By the end of this notebook you will be able to translate freely between Python and pseudocode.

17.1 What Is Pseudocode?

Pseudocode is a way of describing an algorithm using structured, human-readable language that *resembles* a programming language but is not tied to any specific one.

Advantage	Explanation
More precise than natural language	Avoids ambiguity
More readable than real code	No syntax details (colons, brackets, imports)
Language-independent	Anyone who knows <i>any</i> language can read it
Widely used	Textbooks, exams, professional documentation

17.2 Pseudocode Conventions

There is no single “official” pseudocode standard, but the conventions below are widely used and will be our standard for this course.

Element	Python	Pseudocode
Assignment	<code>x = 5</code>	<code>x ← 5</code>
Input	<code>x = int(input())</code>	<code>READ x</code>
Output	<code>print(x)</code>	<code>PRINT x</code>
Condition	<code>if x > 0:</code>	<code>IF x > 0 THEN</code>
Else	<code>else:</code>	<code>ELSE</code>

Element	Python	Pseudocode
End block	<i>(indentation)</i>	END IF
While loop	<code>while x > 0:</code>	WHILE x > 0 DO
End loop	<i>(indentation)</i>	END WHILE
For loop	<code>for i in range(n):</code>	FOR i ← 0 TO n-1 DO
End for	<i>(indentation)</i>	END FOR
Function	<code>def gcd(a, b):</code>	FUNCTION gcd(a, b)
Return	<code>return a</code>	RETURN a
Not equal	<code>!=</code>	$\neq$
Modulo	<code>%</code>	MOD

Key idea: Python uses *indentation* to show structure. Pseudocode uses *keywords* (THEN, DO, END IF, END WHILE) to show where blocks begin and end.

17.3 Worked Example: GCD in Pseudocode

Here is the GCD function you wrote last week in Python:

```

1  def gcd(a, b):
2      while b != 0:
3          r = a % b
4          a = b
5          b = r
6      return a

```

And here is the same algorithm in pseudocode:

```

FUNCTION gcd(a, b)
    WHILE b  $\neq$  0 DO
        r ← a MOD b
        a ← b
        b ← r
    END WHILE
    RETURN a
END FUNCTION

```

Notice the correspondences: - `def` → FUNCTION ... END FUNCTION - `while` → WHILE ... DO ...
 END WHILE - `=` (assignment) → $\leftarrow$ - `%` → MOD - `!=` → $\neq$

17.4 Exercise 1: Python → Pseudocode

Read the Python function below carefully. Then **write the equivalent pseudocode on paper** (or in a new markdown cell).

Use the conventions table above as a reference.

```
1 def max_of_three(a, b, c):
2     if a >= b and a >= c:
3         return a
4     elif b >= a and b >= c:
5         return b
6     else:
7         return c
```

Click to reveal the solution

```
FUNCTION max_of_three(a, b, c)
    IF a ≥ b AND a ≥ c THEN
        RETURN a
    ELSE IF b ≥ a AND b ≥ c THEN
        RETURN b
    ELSE
        RETURN c
    END IF
END FUNCTION
```

17.5 Exercise 2: Pseudocode → Python

The pseudocode below describes a **linear search** algorithm. It searches through a list for a given value and returns the index of the first occurrence, or -1 if the value is not found.

```
FUNCTION linear_search(lst, target)
    FOR i ← 0 TO length(lst) - 1 DO
        IF lst[i] = target THEN
            RETURN i
        END IF
    END FOR
    RETURN -1
END FUNCTION
```

Task: Implement this algorithm in Python in the cell below.

```

1 def linear_search(lst: list, target) -> int:
2     pass # TODO: implement the pseudocode above

1 # Test cases – all should print True
2 print(linear_search([10, 20, 30, 40, 50], 30) == 2)
3 print(linear_search([10, 20, 30, 40, 50], 10) == 0)
4 print(linear_search([10, 20, 30, 40, 50], 50) == 4)
5 print(linear_search([10, 20, 30, 40, 50], 99) == -1)
6 print(linear_search([], 5) == -1)

```

17.6 Exercise 3: Pseudocode → Python

The pseudocode below prints every integer from 1 to n . For multiples of 3 it prints "Fizz" instead of the number, for multiples of 5 it prints "Buzz", and for multiples of both 3 and 5 it prints "FizzBuzz".

```

FUNCTION fizzbuzz(n)
  FOR i ← 1 TO n DO
    IF i MOD 3 = 0 AND i MOD 5 = 0 THEN
      PRINT "FizzBuzz"
    ELSE IF i MOD 3 = 0 THEN
      PRINT "Fizz"
    ELSE IF i MOD 5 = 0 THEN
      PRINT "Buzz"
    ELSE
      PRINT i
    END IF
  END FOR
END FUNCTION

```

Task: Implement this algorithm in Python.

```

1 def fizzbuzz(n: int):
2     pass # TODO: implement the pseudocode above

1 # Run this cell to test – expected output:
2 # 1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz,
3 # 11, Fizz, 13, 14, FizzBuzz

```

```
4  fizzbuzz(15)
```

17.7 Structure Diagrams (Struktogramme)

A **Struktogramm** (Nassi–Shneiderman diagram) is a visual way of representing an algorithm using nested rectangles. Each rectangle represents one instruction or control structure.

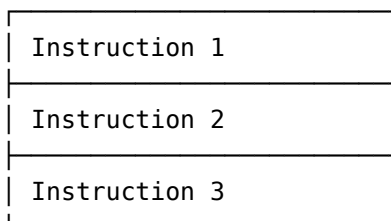
Advantage	Explanation
Visual	Structure is visible at a glance
No arrows	Forces structured programming (no “goto”)
Widely used	Common in German-speaking CS education and exams

17.7.1 The Three Building Blocks

Every Struktogramm is built from just three elements:

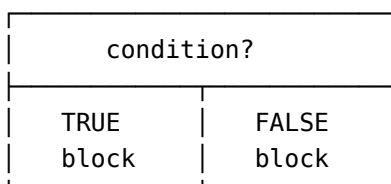
17.7.1.1 1. Sequence

Instructions are executed one after another, top to bottom.



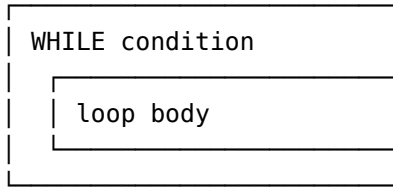
17.7.1.2 2. Selection (IF / ELSE)

A condition splits execution into two branches.

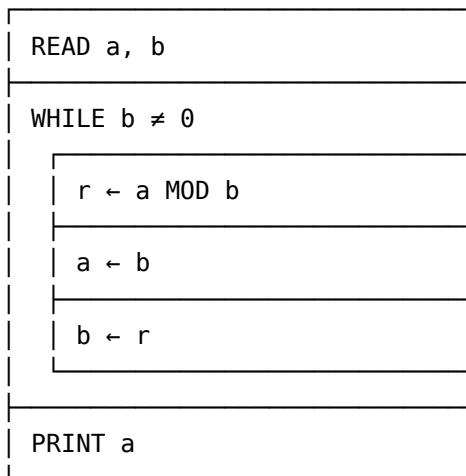


17.7.1.3 3. Iteration (WHILE loop)

A block is repeated as long as a condition holds.



17.7.2 Worked Example: GCD Struktogramm



Reading the diagram: - Top to bottom = execution order (sequence) - The indented box = the loop body (iteration) - **READ** at the top = input, **PRINT** at the bottom = output

17.7.3 Drawing Task (on Paper)

Now draw Struktogramme on paper for the algorithms you translated above:

1. **max_of_three(a, b, c)** — uses sequence and selection.
2. **linear_search(lst, target)** — uses sequence, selection, and iteration.
3. Sum of all integers from 1 to n .

Compare your diagrams with a partner when you're done.

17.8 Summary: Comparing Notations

Notation	Precise?	Executable?	Visual?	Language-independent?
Natural language				
Pseudocode				
Struktogramm				
Python				

No single notation is “best” — each has strengths. Good computer scientists can switch between notations fluently.

18 Sorting Algorithms

18.1 The Sorting Problem

Input: A sequence of n comparable elements (e.g. numbers, strings).

Output: The same elements rearranged in non-descending order.

Example: - Input: [4, 2, 5, 1, 3] - Output: [1, 2, 3, 4, 5]

Sorting is one of the most fundamental operations in computer science. There are many different algorithms that solve this problem — you just invented one (or more!) with your playing cards.

18.2 Pseudocode Conventions (Quick Reference)

Recall from last week:

Element	Python	Pseudocode
Assignment	<code>x = 5</code>	<code>x ← 5</code>
Condition	<code>if x > 0:</code>	<code>IF x > 0 THEN</code>
Else	<code>else:</code>	<code>ELSE</code>
End block	<i>(indentation)</i>	<code>END IF</code>
While loop	<code>while x > 0:</code>	<code>WHILE x > 0 DO</code>
For loop	<code>for i in range(n):</code>	<code>FOR i ← 0 TO n-1 DO</code>
End for	<i>(indentation)</i>	<code>END FOR</code>
Function	<code>def f(a, b):</code>	<code>FUNCTION f(a, b)</code>
Return	<code>return a</code>	<code>RETURN a</code>

New for today — SWAP:

Python	Pseudocode
<code>a, b = b, a</code>	<code>SWAP a AND b</code>
<code>cards[i], cards[j] = cards[j], cards[i]</code>	<code>SWAP cards[i] AND cards[j]</code>

Python lets you swap two variables in one line: `a, b = b, a`. This works because Python evaluates the right side *first*, then assigns both values simultaneously.

18.3 Helper: Visualising Each Step

The function below prints the current state of the list and highlights which positions were just compared. Use it inside your sorting function to trace what is happening.

```
1 def show_step(cards: list, compared: tuple = None, label: str = ""):
2     """Print the list, highlighting the compared positions."""
3     parts = []
4     for idx, val in enumerate(cards):
5         if compared and idx in compared:
6             parts.append(f"[{val}]")
7         else:
8             parts.append(f" {val} ")
9     line = " ".join(parts)
10    if label:
11        print(f"{label:>12s}: {line}")
12    else:
13        print(f"                {line}")
```

```
1 # Example usage
2 demo = [4, 2, 5, 1, 3]
3 show_step(demo, label="start")
4 show_step(demo, compared=(0, 1), label="compare 0,1")
```

18.4 Your Sorting Algorithm

You invented a sorting strategy with the playing cards. Now implement it in Python!

Task:

1. If you haven't already, write pseudocode for your strategy on the worksheet.
2. Translate your pseudocode into the Python function below.
3. Use `show_step()` to visualise what your algorithm does.
4. Test it — does it produce a sorted list for every input?

Don't worry about efficiency or elegance yet. The goal is to get a *working* implementation of *your* strategy.

```

1  def my_sort(cards: list):
2      n = len(cards)
3      show_step(cards, label="start")
4
5      # TODO: implement your sorting strategy here
6      # Use show_step(cards, compared=(i, j), label=...) to trace.
7
8      pass
9
10
11 # Test
12 test = [4, 2, 5, 1, 3]
13 print("Before:", test)
14 my_sort(test)
15 print("After: ", test)

```

```

1  # Once you're happy with your implementation, test it properly:
2  test_cases = [
3      [4, 2, 5, 1, 3],
4      [5, 4, 3, 2, 1],
5      [1, 2, 3, 4, 5],
6      [3, 3, 1, 2, 2],
7      [1],
8  ]
9  for tc in test_cases:
10     original = tc.copy()
11     my_sort(tc)
12     status = "✓" if tc == sorted(original) else "✗"
13     print(f"  {status}  {original} → {tc}")

```

18.5 Named Sorting Algorithms

Now that you have a working sorting function, let's look at two well-known sorting algorithms. You may recognise your own strategy in one of them — or discover a completely different approach.

18.6 Selection Sort

Core idea: Repeatedly find the *minimum* of the unsorted portion and swap it into the next position.

18.6.1 Pseudocode

```
FUNCTION selection_sort(cards, n)
  FOR i ← 0 TO n - 2 DO
    min_pos ← i
    FOR j ← i + 1 TO n - 1 DO
      IF cards[j] < cards[min_pos] THEN
        min_pos ← j
      END IF
    END FOR
    SWAP cards[i] AND cards[min_pos]
  END FOR
END FUNCTION
```

18.6.2 Walkthrough (5 cards: 4, 2, 5, 1, 3)

Pass	Unsorted portion	Minimum	Swap with pos	Result
1	4, 2, 5, 1, 3	1 (pos 3)	pos 0	1, 2, 5, 4, 3
2	2, 5, 4, 3	2 (pos 1)	pos 1 (no move)	1, 2, 5, 4, 3
3	5, 4, 3	3 (pos 4)	pos 2	1, 2, 3, 4, 5
4	4, 5	4 (pos 3)	pos 3 (no move)	1, 2, 3, 4, 5

18.6.3 Exercise: Implement Selection Sort

Translate the pseudocode above into Python. Use `show_step()` to print the state after each pass.

```
1 def selection_sort(cards: list) -> list:
2     n = len(cards)
3     # TODO: implement Selection Sort here
4     # Hint: use the pseudocode as your guide.
5     # Use show_step(cards, compared=(i, j), label=...) to trace.
6     pass
7
8
9 # Test
```

```

10 test = [4, 2, 5, 1, 3]
11 print("Before:", test)
12 selection_sort(test)
13 print("After: ", test)

```

Click to reveal the solution

```

1 def selection_sort(cards: list) -> list:
2     n = len(cards)
3     for i in range(n - 1):
4         min_pos = i
5         for j in range(i + 1, n):
6             if cards[j] < cards[min_pos]:
7                 min_pos = j
8         cards[i], cards[min_pos] = cards[min_pos], cards[i]
9         show_step(cards, label=f"pass {i+1}")

```

18.7 Bubble Sort

Core idea: Walk through the list from left to right, compare adjacent elements, and swap them if they are in the wrong order. Repeat until the list is sorted. The largest unsorted element “bubbles up” to the end in each pass.

18.7.1 Pseudocode

```

FUNCTION bubble_sort(cards, n)
    FOR i ← 0 TO n - 2 DO
        FOR j ← 0 TO n - 2 - i DO
            IF cards[j] > cards[j + 1] THEN
                SWAP cards[j] AND cards[j + 1]
            END IF
        END FOR
    END FOR
END FUNCTION

```

18.7.2 Walkthrough (5 cards: 4, 2, 5, 1, 3)

Pass 1: (compare positions 0–1, 1–2, 2–3, 3–4)

Compare	Before	Swap?	After
pos 0 & 1	4 2 5 1 3	Yes	2 4 5 1 3
pos 1 & 2	2 4 5 1 3	No	2 4 5 1 3
pos 2 & 3	2 4 5 1 3	Yes	2 4 1 5 3
pos 3 & 4	2 4 1 5 3	Yes	2 4 1 3 5

→ After pass 1: 2, 4, 1, 3, 5 — the 5 has bubbled to the end.

Passes 2–3 continue until the list is fully sorted: 1, 2, 3, 4, 5.

18.7.3 Exercise: Implement Bubble Sort

Translate the pseudocode above into Python. Use `show_step()` to print the state after each pass.

```

1  def bubble_sort(cards: list) -> list:
2      n = len(cards)
3      # TODO: implement Bubble Sort here
4
5
6
7  # Test
8  test = [4, 2, 5, 1, 3]
9  print("Before:", test)
10 bubble_sort(test)
11 print("After: ", test)

```

Click to reveal the solution

```

1  def bubble_sort(cards: list) -> list:
2      n = len(cards)
3      for i in range(n - 1):
4          for j in range(n - 1 - i):
5              if cards[j] > cards[j + 1]:
6                  cards[j], cards[j + 1] = cards[j + 1], cards[j]
7              show_step(cards, label=f"pass {i+1}")

```

18.8 Testing Your Implementations

Run the cell below to test whichever sorting function you implemented. Then test `selection_sort` and `bubble_sort` after you've implemented them.

```
1  import random
2
3
4  def test_sort(sort_func, uses_bounds=False):
5      """Test a sorting function on several inputs."""
6      test_cases = [
7          [4, 2, 5, 1, 3],
8          [1, 2, 3, 4, 5],          # already sorted
9          [5, 4, 3, 2, 1],          # reverse sorted
10         [1],                      # single element
11         [3, 3, 1, 2, 2],          # duplicates
12         random.sample(range(1, 11), 10), # random 10 elements
13     ]
14     all_passed = True
15     for i, case in enumerate(test_cases):
16         original = case.copy()
17         expected = sorted(case)
18         if uses_bounds:
19             sort_func(case, 0, len(case) - 1)
20         else:
21             sort_func(case)
22         if case == expected:
23             print(f"Test {i+1}: PASS {original} → {case}")
24         else:
25             print(f"Test {i+1}: FAIL {original} → {case} (expected
26                 ↪ {expected})")
27             all_passed = False
28     if all_passed:
29         print("\n✓ All tests passed!")
30     else:
31         print("\n✗ Some tests failed – check your code.")
```

```
1  # Test your own sorting function:
2  test_sort(my_sort)
```

```
1  # Test Selection Sort:
2  test_sort(selection_sort)
```

```
1 # Test Bubble Sort:
2 test_sort(bubble_sort)
```

18.9 Counting Comparisons

How many comparisons does each algorithm need? The functions below are instrumented versions that count every comparison. Run them on the same input to compare.

```
1 def selection_sort_count(cards: list) -> int:
2     """Selection Sort that returns the number of comparisons."""
3     n = len(cards)
4     comparisons = 0
5     for i in range(n - 1):
6         min_pos = i
7         for j in range(i + 1, n):
8             comparisons += 1
9             if cards[j] < cards[min_pos]:
10                 min_pos = j
11         cards[i], cards[min_pos] = cards[min_pos], cards[i]
12     return comparisons
13
14
15 def bubble_sort_count(cards: list) -> int:
16     """Bubble Sort that returns the number of comparisons."""
17     n = len(cards)
18     comparisons = 0
19     for i in range(n - 1):
20         for j in range(n - 1 - i):
21             comparisons += 1
22             if cards[j] > cards[j + 1]:
23                 cards[j], cards[j + 1] = cards[j + 1], cards[j]
24     return comparisons
```

```
1 # Compare on the same input
2 original = [6, 3, 8, 2, 7, 1]
3
4 cards_a = original.copy()
5 c_sel = selection_sort_count(cards_a)
6 print(f"Selection Sort: {c_sel} comparisons → {cards_a}")
```



```
7  
8 cards_b = original.copy()  
9 c_bub = bubble_sort_count(cards_b)  
10 print(f"Bubble Sort:    {c_bub} comparisons → {cards_b}")
```

19 Algorithms and Computational Complexity

i Arrays: The Basic Data Structure

An array is a basic data structure. It stores a fixed number of elements of the same data type under a single variable name. Sorting these elements is one of the most fundamental tasks in computer science.

19.1 Selection Sort

Selection Sort works as follows:

1. Go through the array to find the lowest value.
2. Move the lowest value to the front of the unsorted part of the array.
3. Repeat this process for the unsorted part as many times as there are values in the array.

In order to evaluate the time complexity of Selection Sort, it is necessary to **analyse** how the number of comparisons increases as a function of the number of elements being sorted. Due to the way Selection Sort operates, approximately $\frac{1}{2}n^2$ comparisons are required for n elements. This means the number of operations increases **quadratically** relative to the input size.

Since we **categorise** an algorithm's efficiency according to its **order of magnitude**, we refer to this as having quadratic time complexity. In this process, constant factors (like the $\frac{1}{2}$) and lower-order terms are disregarded, as the primary concern when evaluating an algorithm is how the time requirement scales with extremely large datasets.

19.2 Big O Notation

To formally capture this focus on the fundamental rate of growth, computer scientists use a specific shorthand known as **Big O notation**.

The 'O' stands for 'Order of', indicating the **complexity class** of an algorithm. Instead of meticulously counting every single operation, Big O notation describes the upper bound of the growth rate. For instance, stating that Selection Sort has a time complexity of $O(n^2)$ tells us that if the input size doubles, the time required will roughly quadruple, regardless of the computer's speed or any minor mathematical constants.

19.3 Merge Sort: A More Efficient Approach

To illustrate why choosing the right complexity class matters, let's compare Selection Sort with **Merge Sort**. While Selection Sort works by repeatedly searching for the minimum, Merge Sort uses a “**divide and conquer**” strategy. It recursively splits the array into two halves until each part contains only one element, and then merges these parts back together in the correct order.

19.3.1 The Merge Step: Combining with Efficiency

The actual efficiency of Merge Sort lies in the **merge step**:

1. Imagine you have two smaller piles of cards, both already sorted from lowest to highest.
2. You only ever need to compare the two “top” cards of each pile.
3. You pick the smaller one, place it into the new, larger array, and move to the next card in that pile.

Because this step only requires a single pass through the elements, merging two sorted lists of total size n takes **linear time**, or $O(n)$. Since the “Divide” phase splits the problem into $\log n$ levels and an $O(n)$ merge operation is performed at each level, the total time complexity becomes:

$$n \cdot \log n = O(n \log n)$$

19.4 Comparison of Growth Rates

The “log-linear” growth of Merge Sort is significantly more efficient than the “quadratic” growth of Selection Sort as the input size n increases.

Tabelle 19.1: Comparison of time complexity between Selection Sort and Merge Sort

Number of Elements(n)	Selection Sort($O(n^2)$)	Merge Sort($O(n \log n)$)	Efficiency(Ratio)
10	100 operations	~33 operations	3x faster
1,000	1,000,000	~10,000	100x faster
1,000,000	1,000,000,000,000	~20,000,000	50,000x faster

For a small list, the difference is negligible. However, for large datasets, Selection Sort becomes unusable, while Merge Sort remains efficient. This is the power of understanding computational complexity: it allows us to predict how an algorithm will perform when data scales up.

Teil IV

Functions

20 Functions: Revision and Consolidation

20.1 Function: A Definition

A function is a self-contained block of code that encapsulates specific logic. By using parameters and return values, it acts as a ‘black box’ that processes inputs into outputs.

Let’s break this definition down into its core components to understand how a function actually works:

1. “Self-contained block”

A function is like a “mini-program” within the main program. It has its own name and its own scope. This means that variables defined inside a function usually don’t interfere with the rest of the code. It promotes modular design, making the code organised and easier to read and to debug.

2. “Encapsulates specific logic”

Encapsulation means “wrapping things up.” There is no need to know how the function actually works to use it. It is sufficient to know what parameters it takes and what values it returns. This is the first step toward abstraction.

3. “Parameters” (The Inputs)

Parameters are the “knobs” or “settings” of a function. They allow for passing information into the function. Without parameters, a function would do exactly the same thing every time. With parameters, it becomes flexible and can handle different inputs each time it is called.

4. “Return Values” (The Outputs)

A function doesn’t just “do” something; it often “gives” something back. The `return` statement is the exit point of the function, handing a result back to the part of the program that called it.

5. The “Black Box” Concept

This is a key concept in computer science. The user provides an input, the “black box” (the function) performs its processing logic, and it produces an output. The user of a function only cares about the interface (Inputs/Outputs), not the implementation.

20.2 Practical Example: The VAT Calculator

In this example, we calculate the net price from a given gross price. In Swiss retail, prices are usually quoted as *gross* — i.e., including VAT. The *net price* is what remains after the VAT portion is removed. Since the standard VAT rate in Switzerland is 8.1 %, we use this as a default parameter. This makes the function easier to use for standard cases while remaining flexible for others.

```
1  def calculate_net_price(gross_price: float, vat_rate: float = 8.1) ->
   ↪   float:
2     """
3     Calculates the net price from a given gross price.
4
5     Args:
6         gross_price (float): The total price including VAT.
7         vat_rate (float): The VAT percentage to be removed.
8                             Defaults to 8.1 (Swiss standard rate).
9
10    Returns:
11        float: The calculated net price, rounded to two decimal places.
12    """
13    # gross = net * (1 + VAT/100) -> net = gross / (1 + VAT/100)
14    gross_factor = 1 + (vat_rate / 100)
15    net_price = gross_price / gross_factor
16
17    return round(net_price, 2)
18
19 # --- Using the "Black Box" ---
20
21 # Case A: Standard rate (8.1% is used automatically)
22 standard_total = calculate_net_price(100.0)
23 print(f"Standard Net Price: {standard_total} CHF")
24
25 # Case B: Reduced rate for everyday goods (2.6%)
26 reduced_total = calculate_net_price(100.0, 2.6)
27 print(f"Reduced Net Price: {reduced_total} CHF")
```

i New Elements Explained

- **Type Hints** (`: float` and `-> float`): These document for the programmer (and the IDE) that the function expects decimal numbers and will return a decimal number. They are like labels on the “knobs” of our black box. *Note: Python does not enforce Type Hints at runtime — they serve documentation and tooling, not validation.*
- **The Docstring** (`""" ... """`): This is the manual for the black box. It explains what the function does, what the arguments (Args) represent, and what the function Returns. In professional IDEs, this text pops up when you hover over the function name. The format used here is the *Google style* for docstrings.
- **Default Parameter** (`vat_rate: float = 8.1`): By assigning a value in the signature, we make this input optional. If the user doesn’t provide a rate, Python automatically uses 8.1.

How this illustrates our five points:

Modularity & Scope: The variables `gross_factor` and `net_price` only exist inside the function. Your main program stays “clean” and doesn’t get cluttered with temporary calculation variables.

Encapsulation: The mathematical formula (dividing by the factor rather than just subtracting a percentage) is hidden. The user doesn’t need to be a maths expert; they just need to call the function.

Default Parameters: By setting `vat_rate=8.1`, we’ve made the function smarter. It assumes the most common case but allows the user to override it for special cases like hotels or groceries.

The Interface: The “interface” here is simply `calculate_net_price(gross_price, vat_rate)`. As long as you know this signature, you can use the tool without ever looking at the code inside the function again.

20.3 Coding Exercise: The BMI Calculator

20.3.1 Objective

Apply your knowledge of functions, encapsulation, and documentation by creating a tool to calculate the Body Mass Index (BMI).

20.3.2 The Task

Write a Python function named `calculate_bmi` that takes a person's weight and height as inputs and returns their BMI. Remember the “Black Box” principle: the user should only need to provide the numbers; your function handles the maths.

20.3.3 The Formula

The BMI is calculated by dividing a person's weight in kilograms by the square of their height in metres:

$$BMI = \frac{\text{weight (kg)}}{\text{height (m)}^2}$$

20.3.4 Requirements

- **Function Signature:** Use Type Hints to specify that both inputs and the return value are floats.
- **Docstring:** Include a complete Docstring at the beginning of the function. Use the Google style (with the sections **Args:** and **Returns:**) as shown in the previous example. Write the prose in British English spelling (e.g., “metres”, “organised”).
- **Encapsulation:** The internal calculation should be performed within the function, and the result should be returned rounded to two decimal places.
- **Testing:** Call your function at the end of your script to calculate the BMI for a person who weighs 80 kg and is 1.85 m tall.

20.3.5 Exercise Template:

```
1 # Define the signature: which parameters with which Type Hints?
2 # What is the return type?
3 def calculate_bmi(...) -> ...:
4     """
5     [Write a brief description of the function here]
6
7     Args:
8         [List the arguments here]
9
10    Returns:
11        [Describe the return value here]
12    """
13    # 1. Implementation: Calculate the BMI using the formula
14
```



```

15     # 2. Return Value: Return the result rounded to 2 decimal places
16     pass
17
18 # --- Test your "Black Box" ---
19 # 1. Call the function with 80.0 and 1.85
20 # 2. Print the result with an appropriate message

1  def calculate_bmi(weight: float, height: float) -> float:
2      """
3      Calculates the body mass index of a person from the person's weight
4      and height.
5
6      Args:
7          weight (float): The person's weight in kilogram.
8          height (float): The person's height in metres.
9      Returns:
10         float: The person's BMI, rounded to two decimal places.
11     """
12     # 1. logic -> implementing the bmi formula
13     bmi = weight / (height ** 2)
14
15     # 2. formatting -> round the return value to 2 decimals
16     return round(bmi, 2)
17
18 print(calculate_bmi(80.0, 1.85))

```

20.3.6 Success Criteria

- The function returns 23.37 for a weight of 80 kg and a height of 1.85 m.
- The Docstring clearly explains what the parameters represent.
- The calculation logic is entirely contained within the function (Encapsulation).
- The function uses the `return` statement correctly rather than just printing the result inside the function.

20.3.7 Optional Extension: A Classifier Function

The BMI is a rough indicator and has well-known limitations (it ignores muscle mass and body composition). As a follow-up exercise, write a second function `classify_bmi(bmi: float) -> str` that returns a category according to the WHO scale (e.g. *underweight*, *normal*, *overweight*, *obese*).

This is the natural bridge to the next topic — conditional statements (`if / elif / else`) — and it also lets you practice **function composition**:

```
1 category = classify_bmi(calculate_bmi(80.0, 1.85))
```

20.4 Coding Exercise: The Cuboid Space Diagonal

20.4.1 Objective

Create a geometry tool that calculates the space diagonal (the longest internal line) of a cuboid. This exercise will help you practice handling multiple parameters and using Python’s mathematical libraries.

20.4.2 The Task

Write a Python function named `calculate_space_diagonal` that accepts the three edge lengths of a cuboid and returns the length of the space diagonal.

20.4.3 The Formula

To find the space diagonal (d) of a cuboid with edge lengths a , b , and c , we use the 3D version of Pythagoras’ Theorem:

$$d = \sqrt{a^2 + b^2 + c^2}$$

20.4.4 Requirements

- **Module Import:** You will need the `sqrt` function from Python’s `math` module. After `import math`, the call `math.sqrt(x)` calculates the square root of a `float`.
- **Function Signature:** Use Type Hints to ensure all three inputs and the return value are floats.
- **Docstring:** Write a professional Docstring in the Google style (sections `Args:` and `Returns:`), using British English spelling (e.g., “metres”, “calculated”).
- **Encapsulation:** The user should not have to worry about the square root or the squaring of numbers; the “Black Box” handles the maths internally.
- **Testing:** Test your function with a cuboid that has the dimensions $a = 5.0$, $b = 10.0$, and $c = 2.0$.

Tip: You can square a number with `x ** 2` (or `x * x`). The square root requires `math.sqrt(x)` after `import math`.

```
1 import math # this works analogously to the import of PyTamaro
2
```

```

3 def calculate_space_diagonal(...) -> ...:
4     # TODO: implement the function according to the given requirements
5     pass
6
7 # --- Test your "Black Box" ---
8 # Call the function with edge lengths 5.0, 10.0, and 2.0
9 # Expected result: approx. 11.36

```

```

1 import math
2
3 def calculate_space_diagonal(a: float, b: float, c: float) -> float:
4     """
5     Calculates the space diagonal of a cuboid from the edge lengths.
6
7     Args:
8         a (float): Length of the first edge.
9         b (float): Length of the second edge.
10        c (float): Length of the third edge.
11
12        The function works independently of the actual units as long as
13        all edges are provided in the same unit.
14
15    Returns:
16        float: The calculated space diagonal of a cuboid in the same
17        unit as the input.
18    """
19
20    d = math.sqrt(a**2 + b**2 + c**2)
21    return d
22
23 calculate_space_diagonal(5.0, 10.0, 2.0)

```

21 Functions II: Calculating Triangles

21.1 Functions within functions

Python functions can not only process Python statements, but they can also call user-defined functions. The implementation of Heron's formula will serve as an example.

21.1.1 Calculating a Triangle's Area Using Heron's Formula

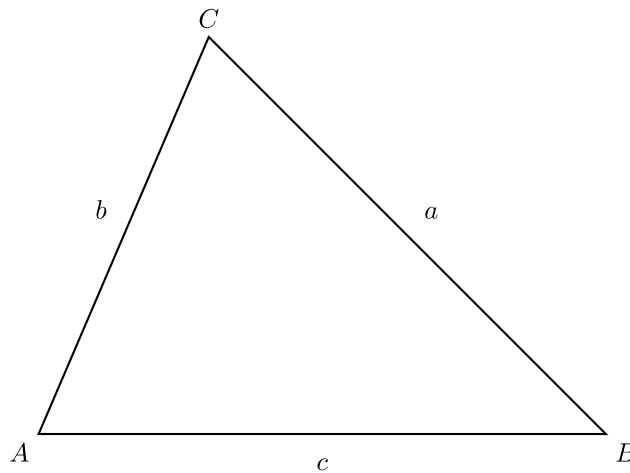


Abbildung 21.1: General Triangle.

To calculate the area T of a triangle, we use Heron's formula

$$T = \sqrt{s(s-a)(s-b)(s-c)}$$

where s denotes the *semiperimeter*. The semiperimeter is calculated by adding the lengths of the triangle's sides and dividing the sum by two.

$$s = \frac{1}{2} \cdot (a + b + c)$$

Coding Task

Implement a function `area()` to calculate the area of a triangle from its sides. Use a subfunction `semiperimeter()` to simplify the logic.

```
1  def semiperimeter(a: float, b: float, c: float) -> float:
2      """Calculates the semiperimeter of a triangle.
3
4      Args:
5          a: Side a of the triangle.
6          b: Side b of the triangle.
7          c: Side c of the triangle.
8
9      Returns:
10         s: The semiperimeter of the triangle.
11     """
12     return (a + b + c) / 2
13
14 def area(a: float, b: float, c: float) -> float:
15     """Calculates the area of a triangle.
16
17     The triangle's area is calculated from its sides using Heron's
18     formula.
19
20     Args:
21         a: Side a of the triangle.
22         b: Side b of the triangle.
23         c: Side c of the triangle.
24
25     Returns:
26         T: The Area of the triangle."""
27     s = semiperimeter(a, b, c)
28     T = (s*(s-a)*(s-b)*(s-c))**(1/2)
29     return T
30
31 print(area(3, 4, 5))
32 # should print 6.0 to the command line
```

6.0

21.1.2 The Altitude of a Triangle

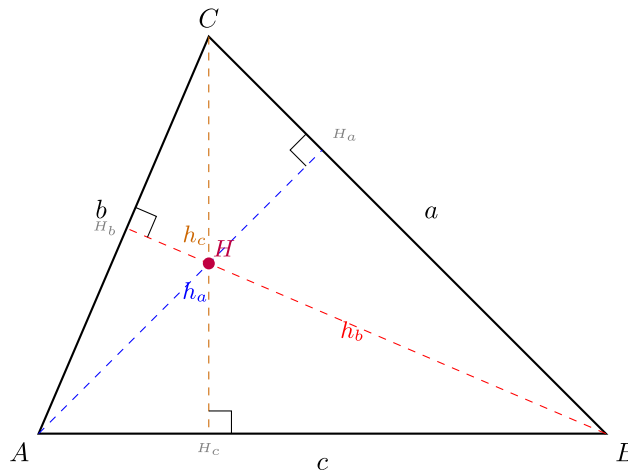


Abbildung 21.2: Altitude of a Triangle.

The altitude h of a triangle is calculated by the formula

$$h_a = \frac{2}{a} \sqrt{s(s-a)(s-b)(s-c)}$$

where s denotes the *semiperimeter*. To calculate the altitude h_b and h_c , switch the respective variables in the formula accordingly.

Coding Task

Implement a function `altitude()` to calculate the altitude of a triangle from its sides. Use as many subfunctions as possible.

```

1  def altitude(a: float, b: float, c: float, base: str = 'c') -> float:
2      """Calculates the altitude of a triangle.
3
4      The altitude is calculated to the base c by default. The base can
5      be overridden if needed.
6
7      Args:
8          a: Side a of the triangle.
9          b: Side b of the triangle.
10         c: Side c of the triangle.
11         base: The base of the altitude. The default value is 'c'
12
13     Returns:

```

```

14         h: The altitude of the triangle."""
15     T = area(a, b, c)
16
17     if base == 'c':
18         h = (2*T)/c
19     elif base == 'a':
20         h = (2*T)/a
21     else:
22         h = (2*T)/b
23
24     return h
25
26 print(altitude(3, 4, 5, 'a'))

```

i Default Values for Function Arguments

If you have to draw the altitude of a triangle, you most likely would draw it from the horizontal side as a base line. Therefore, the base line will frequently be side *c* of your triangle. To account for this probability, Python offers a default value option for function arguments, as demonstrated in the `altitude()` function above.

The syntax requires that arguments with a default value must be placed at the end of the argument list. The default value is assigned using the `=` sign. This default value can be overridden either implicitly as a positional argument by providing a full list of arguments (e.g., `altitude(3, 4, 5, 'a')`), or explicitly as a keyword argument by directly assigning a new value to the parameter (e.g., `altitude(3, 4, 5, base='a')`).

21.1.3 Circumcircle of a Triangle

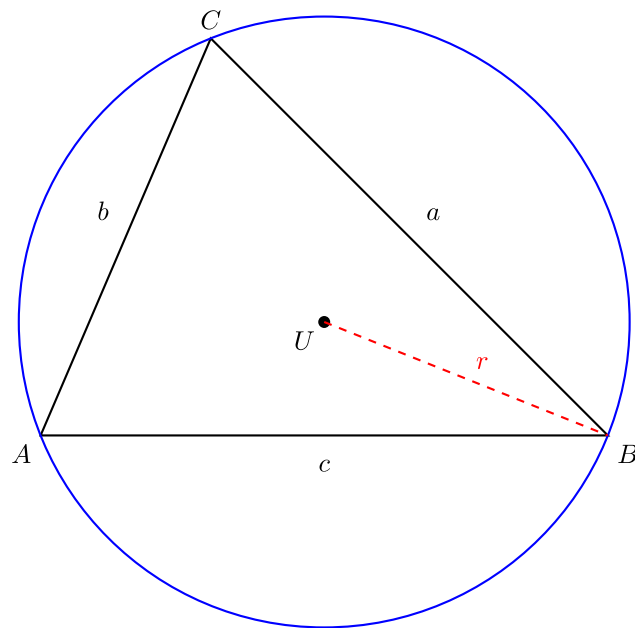


Abbildung 21.3: The Circumcircle of a Triangle.

The radius r of the circumcircle of a triangle is calculated by the formula

$$r = \frac{abc}{4T}$$

where T denotes the area of the triangle.

Coding Task

Implement a function `circumcircle()` to calculate the circumcircle radius of a triangle from its sides. Use as many subfunctions as possible.

```
1 def circumcircle(a: float, b: float, c: float) -> float:
2     """Calculates the circumcircle radius of a triangle.
3
4     Args:
5         a: Side a of the triangle.
6         b: Side b of the triangle.
7         c: Side c of the triangle.
8
9     Returns:
```



```

10         r: The radius of the circumcircle of a triangle.
11         """
12         T = area(a, b, c)
13         n = a * b * c
14         d = 4 * T
15         r = n / d
16         return r
17
18 print(circumcircle(3, 4, 5))

```

21.1.4 The Incircle of a Triangle

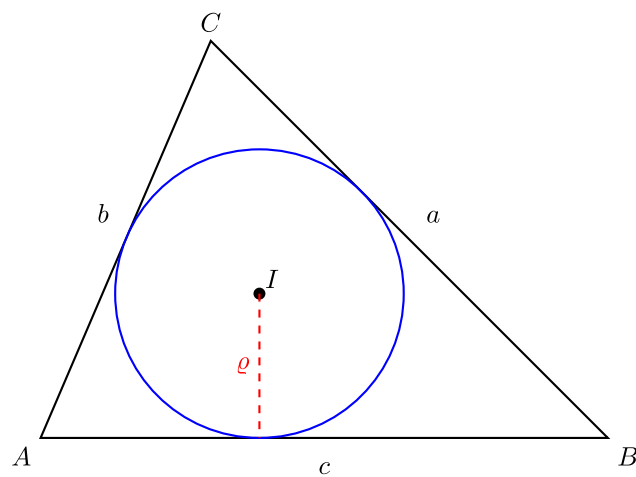


Abbildung 21.4: The Incircle of a Triangle.

The incircle radius ρ (small greek letter rho) of a triangle is calculated by the formula

$$\rho = \frac{T}{s}$$

where T denotes the area of the triangle and s the semiperimeter respectively.

Coding Task

Implement a function `incircle()` to calculate the incircle radius of a triangle from its sides. Use as many subfunctions as possible.

```

1 def incircle(a: float, b: float, c: float) -> float:
2     """Calculates the incircle radius.

```

```

3
4     Args:
5         a: Side a of the triangle.
6         b: Side b of the triangle.
7         c: Side c of the triangle.
8
9     Returns:
10        rho: Incircle radius of the triangle.
11        """
12    T = area(a, b, c)
13    s = semiperimeter(a, b, c)
14    rho = T/s
15    return rho
16
17 print(incircle(3, 4, 5))

```

22 Classes: Triangle as Object

22.1 Introduction

Functions are perfect for avoiding repetitive code (DRY principle). But when you model real-world things—like a collection of triangles defined by their three side lengths—a Python class allows you to bundle both the data and the functions together efficiently.

In the following code cell, the functions from the previous chapter are integrated into a class named `TriangleSSS`.

```
1  class TriangleSSS:
2      """Represents a geometric triangle based on its three side lengths.
3
4      Attributes:
5          a (float): The length of the first side.
6          b (float): The length of the second side.
7          c (float): The length of the third side.
8      """
9
10     def __init__(self, a: float, b: float, c: float):
11         """Initialises the triangle with the three side lengths.
12
13         Args:
14             a (float): Length of side a.
15             b (float): Length of side b.
16             c (float): Length of side c.
17         """
18         self.a = a
19         self.b = b
20         self.c = c
21
22     def _get_semiperimeter(self):
23         """Calculates the semiperimeter of the triangle.
24
25         Returns:
26             float: The semiperimeter of the triangle.
27         """
28         return (self.a + self.b + self.c) / 2
```

```

29
30     def get_area(self):
31         """Calculates the area of the triangle using Heron's formula.
32
33         Returns:
34             float: The area of the triangle.
35         """
36         s = self._get_semiperimeter()
37         T = (s * (s - self.a) * (s - self.b) * (s - self.c)) ** (1/2)
38         return T
39
40     def get_altitude(self, base='a'):
41         """Calculates the altitude of the triangle relative to a
42             ↪ specific base side.
43
44         Args:
45             base (str, optional): The base side to which the altitude
46             ↪ relates
47                 ('a', 'b', or 'c'). Defaults to 'a'.
48
49         Returns:
50             float: The altitude relative to the chosen base side.
51         """
52         T = self.get_area()
53         if base == 'a':
54             h = (2 * T) / self.a
55         elif base == 'b':
56             h = (2 * T) / self.b
57         else:
58             h = (2 * T) / self.c
59         return h
60
61     def get_circumcircle_radius(self):
62         """Calculates the radius of the circumcircle of the triangle.
63
64         Returns:
65             float: The radius of the circumcircle.
66         """
67         T = self.get_area()
68         n = self.a * self.b * self.c
69         d = 4 * T
70         r = n / d
71         return r

```

```

71     def get_incircle_radius(self):
72         """Calculates the radius of the incircle of the triangle.
73
74         Returns:
75             float: The radius of the incircle.
76         """
77         T = self.get_area()
78         s = self._get_semiperimeter()
79         rho = T / s
80         return rho

```

To use the `TriangleSSS` class, we need to create (instantiate) a new triangle object. The following code cell shows how to do this.

```

1 triangle = TriangleSSS(3, 4, 5)

```

When calling `TriangleSSS(3, 4, 5)`, a very special function is automatically executed behind the scenes: the `__init__()` method (often referred to as the constructor).

The values 3, 4, and 5 are passed into the parameters `a`, `b`, and `c`. The `self` keyword then ensures that these values are bound to the newly created instance and stored in memory as attributes (`self.a`, `self.b`, and `self.c`). It is precisely through this step that the triangle object initialises and “remembers” its individual side lengths.

You can easily verify that these attributes have been successfully saved by accessing them directly using dot notation - for instance, running `print(triangle.a)` will output 3.

The newly instantiated object `triangle` allows us to access the methods of the `TriangleSSS` class. Methods are simply functions defined inside a class. They are called using *dot notation*, as shown in the example below.

```

1 triangle.get_area()

```

6.0

Normally, we call a Python function by passing arguments inside the parentheses. However, thanks to the `self` argument in our method definitions, the method already knows the values of `a`, `b`, and `c` (by accessing `self.a`, `self.b`, and `self.c`). The variable `self` always refers to the specific instance of our object.

22.2 Enhanced Triangle Class

In geometry, a triangle is uniquely defined if we know specific combinations of its side lengths and angles. This is determined by the four classic congruence criteria:

SSS (Side-Side-Side): All three side lengths are known.

SAS (Side-Angle-Side): Two sides and the enclosed angle are known.

ASA / SAA (Angle-Side-Angle / Side-Angle-Angle): One side and two angles are known.

SsA (Side-Side-Angle): Two sides and an angle opposite the longer side are known.

While our initial `TriangleSSS` class worked perfectly when given three sides, a truly versatile geometric model should handle any valid combination. Ideally, we want a single, elegant Python class capable of dynamically initialising a triangle based on whichever of these four criteria the user provides.

The following implementation achieves exactly this by identifying the given parameters and validating them against mathematical rules before creating the object.

```
1  import math
2
3  class Triangle:
4      def __init__(self, a=None, b=None, c=None, alpha=None, beta=None,
5          ↪ gamma=None):
6          """Initialise a Triangle instance using valid congruence
7          ↪ criteria.
8
9          Args:
10             a (float, optional): Length of side a. Defaults to None.
11             b (float, optional): Length of side b. Defaults to None.
12             c (float, optional): Length of side c. Defaults to None.
13             alpha (float, optional): Angle alpha opposite to side a.
14             ↪ Defaults to None.
15             beta (float, optional): Angle beta opposite to side b.
16             ↪ Defaults to None.
17             gamma (float, optional): Angle gamma opposite to side c.
18             ↪ Defaults to None.
19
20             Raises:
21             ValueError: If the number of provided parameters is not
22             ↪ exactly three,
23             if the parameters do not form a unique triangle, or if
24             ↪ mathematical rules (e.g. triangle inequality) are
25             ↪ violated.
26             """
27
28             # 1. Initialise all attributes to prevent subsequent
29             ↪ AttributeError
30             self.a = a
31             self.b = b
32             self.c = c
```

```

24     self.alpha = alpha
25     self.beta = beta
26     self.gamma = gamma
27
28     # 2. Determine which values were actually passed to the
    ↪ constructor
29     given_sides = [k for k, v in [('a', a), ('b', b), ('c', c)] if v
    ↪ is not None]
30     given_angles = [k for k, v in [('alpha', alpha), ('beta', beta),
    ↪ ('gamma', gamma)] if v is not None]
31
32     num_sides = len(given_sides)
33     num_angles = len(given_angles)
34
35     # A triangle mathematically always requires exactly 3 parameters
36     if num_sides + num_angles != 3:
37         raise ValueError("A triangle must be defined by exactly 3
    ↪ parameters.")
38
39     # 3. Validate mathematical congruence theorems
40     if num_sides == 3:
41         # --- SSS (Side-Side-Side) ---
42         if not (a + b > c and a + c > b and b + c > a):
43             raise ValueError("Triangle inequality violated: The sum
    ↪ of two sides must be greater than the third.")
44
45     elif num_sides == 2 and num_angles == 1:
46         # --- SAS or SsA (Side-Angle-Side / Side-Side-Angle) ---
47         angle_name = given_angles[0]
48         opposite_mapping = {'alpha': 'a', 'beta': 'b', 'gamma': 'c'}
49         opposite_side = opposite_mapping[angle_name]
50
51         if opposite_side in given_sides:
52             # Case: SsA (Angle is opposite one of the given sides)
53             other_side = [s for s in given_sides if s !=
    ↪ opposite_side][0]
54             val_opposite = {'a': a, 'b': b, 'c': c}[opposite_side]
55             val_other = {'a': a, 'b': b, 'c': c}[other_side]
56
57             if val_opposite <= val_other:
58                 raise ValueError(f"SsA error: Angle '{angle_name}'
    ↪ must be opposite the longer side.")
59
60     elif num_sides == 1 and num_angles == 2:

```

```

61         # --- ASA / SAA (Angle-Side-Angle / Side-Angle-Angle) ---
62         angle_values = [v for v in [alpha, beta, gamma] if v is not
↪ None]
63         if sum(angle_values) >= 180:
64             raise ValueError("The sum of the given angles must be
↪ less than 180 degrees.")
65
66         elif num_angles == 3:
67             # --- AAA (Angle-Angle-Angle) ---
68             raise ValueError("AAA (Three angles) does not define a
↪ unique triangle due to a lack of scaling.")
69
70         # 4. Mathematically solve the complete triangle to populate all
↪ missing sides and angles
71         self._solve_triangle()
72
73     def _solve_triangle(self):
74         """Private helper method to calculate all missing sides and
↪ angles."""
75         def rad(deg): return math.radians(deg)
76         def deg(rad): return math.degrees(rad)
77
78         # Case 1: SSS
79         if self.a and self.b and self.c:
80             self.alpha = deg(math.acos((self.b**2 + self.c**2 -
↪ self.a**2) / (2 * self.b * self.c)))
81             self.beta = deg(math.acos((self.a**2 + self.c**2 -
↪ self.b**2) / (2 * self.a * self.c)))
82             self.gamma = 180.0 - self.alpha - self.beta
83
84         # Case 2: 2 sides, 1 angle (SAS or SsA)
85         elif sum(s is not None for s in [self.a, self.b, self.c]) == 2:
86             sides = {'a': self.a, 'b': self.b, 'c': self.c}
87             angles = {'alpha': self.alpha, 'beta': self.beta, 'gamma':
↪ self.gamma}
88             given_s = [k for k, v in sides.items() if v is not None]
89             given_a = [k for k, v in angles.items() if v is not None][0]
90             angle_val = angles[given_a]
91
92             enclosed_map = {('a', 'b'): 'gamma', ('b', 'a'): 'gamma',
93                             ('b', 'c'): 'alpha', ('c', 'b'): 'alpha',
94                             ('a', 'c'): 'beta', ('c', 'a'): 'beta'}
95
96             if given_a == enclosed_map[tuple(given_s)]:

```



```

97         # Sub-case: SAS
98         s1, s2 = given_s[0], given_s[1]
99         v1, v2 = sides[s1], sides[s2]
100        v3 = math.sqrt(v1**2 + v2**2 - 2 * v1 * v2 *
↪ math.cos(rad(angle_val)))
101        missing_s = [s for s in ['a', 'b', 'c'] if s not in
↪ given_s][0]
102        setattr(self, missing_s, v3)
103        # Recalculate angles as SSS
104        self.alpha = deg(math.acos((self.b**2 + self.c**2 -
↪ self.a**2) / (2 * self.b * self.c)))
105        self.beta = deg(math.acos((self.a**2 + self.c**2 -
↪ self.b**2) / (2 * self.a * self.c)))
106        self.gamma = 180.0 - self.alpha - self.beta
107    else:
108        # Sub-case: SsA
109        opposite_side = {'alpha': 'a', 'beta': 'b', 'gamma':
↪ 'c'}[given_a]
110        other_side = [s for s in given_s if s !=
↪ opposite_side][0]
111        val_opp, val_oth = sides[opposite_side],
↪ sides[other_side]
112
113        sin_other = math.sin(rad(angle_val)) * val_oth / val_opp
114        other_angle_val = deg(math.asin(sin_other))
115        other_angle_name = {'a': 'alpha', 'b': 'beta', 'c':
↪ 'gamma'}[other_side]
116        setattr(self, other_angle_name, other_angle_val)
117
118        missing_a = [a for a in ['alpha', 'beta', 'gamma'] if
↪ getattr(self, a) is None][0]
119        setattr(self, missing_a, 180.0 - angle_val -
↪ other_angle_val)
120
121        missing_s = {'alpha': 'a', 'beta': 'b', 'gamma':
↪ 'c'}[missing_a]
122        missing_s_val = val_opp * math.sin(rad(getattr(self,
↪ missing_a))) / math.sin(rad(angle_val))
123        setattr(self, missing_s, missing_s_val)
124
125        # Case 3: 1 side, 2 angles (ASA / SAA)
126        elif sum(s is not None for s in [self.a, self.b, self.c]) == 1:
127            if self.alpha is None: self.alpha = 180.0 - self.beta -
↪ self.gamma

```

```

128         elif self.beta is None: self.beta = 180.0 - self.alpha -
            ↪ self.gamma
129         elif self.gamma is None: self.gamma = 180.0 - self.alpha -
            ↪ self.beta
130
131         known_s_name = [s for s in ['a', 'b', 'c'] if getattr(self,
132 ↪ s) is not None][0]
133         known_s_val = getattr(self, known_s_name)
134         known_a_val = getattr(self, {'a': 'alpha', 'b': 'beta', 'c':
135 ↪ 'gamma'}[known_s_name])
136
137         factor = known_s_val / math.sin(rad(known_a_val))
138         if self.a is None: self.a = factor *
            ↪ math.sin(rad(self.alpha))
139         if self.b is None: self.b = factor *
            ↪ math.sin(rad(self.beta))
140         if self.c is None: self.c = factor *
            ↪ math.sin(rad(self.gamma))
141
142         # =====
143         # Methods derived from the formulas in Durandi, Werner, Baoswan
144         # Dzung Wong, Markus Kriener, Hansruedi Künsch, Alfred Vogelsanger,
145         # Jörg Waldvogel, Samuel Byland, u. a. Formeln, Tabellen, Begriffe:
146         # Mathematik - Physik - Chemie. 8., Aktualisierte Auflage.
147         # Herausgegeben von Verein Schweizerischer Mathematik- und
148         # Physiklehrkräfte und Werner Durandi. Zürich: Orell Füssli Verlag,
149         # 2022; pp. 86.
150         # =====
151
152         def get_semiperimeter(self):
153             """Calculate the semiperimeter (s) of the triangle.
154
155             Returns:
156                 float: The semiperimeter value.
157             """
158             # Formula:  $s = (a + b + c) / 2$ 
159             return (self.a + self.b + self.c) / 2
160
161         def get_exterior_angle(self, vertex='alpha'):
162             """Calculate the exterior angle for a given interior angle
            ↪ vertex.

```

```

163         vertex (str): The interior angle name ('alpha', 'beta',
↪ 'gamma'). Defaults to 'alpha'.
164
165     Returns:
166         float: The exterior angle in degrees.
167     """
168     # Formula: alpha' = 180° - alpha
169     interior_angle = getattr(self, vertex)
170     return 180.0 - interior_angle
171
172 def get_area(self):
173     """Calculate the area (A) of the triangle using Heron's formula.
174
175     Returns:
176         float: The area of the triangle.
177     """
178     # Formula: A = sqrt(s * (s - a) * (s - b) * (s - c)) (HERON)
179     s = self.get_semiperimeter()
180     return math.sqrt(s * (s - self.a) * (s - self.b) * (s - self.c))
181
182 def get_altitude(self, base='a'):
183     """Calculate the altitude (h) relative to a specific base side.
184
185     Args:
186         base (str): The base side identifier ('a', 'b', 'c').
↪ Defaults to 'a'.
187
188     Returns:
189         float: The height (altitude) on the chosen base side.
190     """
191     # Formula: h_a = c * sin(beta) or h_a = (2 / a) * Area
192     base_val = getattr(self, base)
193     return (2 * self.get_area()) / base_val
194
195 def get_median(self, side='a'):
196     """Calculate the length of the median (Seitenhalbierende) for a
↪ side.
197
198     Args:
199         side (str): The side identifier ('a', 'b', 'c'). Defaults to
↪ 'a'.
200
201     Returns:
202         float: The length of the median line.

```

```

203         """
204         # Formula: s_a = 0.5 * sqrt(2*b^2 + 2*c^2 - a^2)
205         if side == 'a':
206             return 0.5 * math.sqrt(2 * self.b**2 + 2 * self.c**2 -
207                                     ↪ self.a**2)
208         elif side == 'b':
209             return 0.5 * math.sqrt(2 * self.a**2 + 2 * self.c**2 -
210                                     ↪ self.b**2)
211         else:
212             return 0.5 * math.sqrt(2 * self.a**2 + 2 * self.b**2 -
213                                     ↪ self.c**2)
214
215     def get_angle_bisector(self, angle='alpha'):
216         """Calculate the length of the internal angle bisector
217         ↪ (Winkelhalbierende).
218
219         Args:
220             angle (str): The angle identifier ('alpha', 'beta',
221             ↪ 'gamma'). Defaults to 'alpha'.
222
223         Returns:
224             float: The length of the angle bisector line.
225         """
226         # Formula: w_alpha = sqrt(b * c * [1 - (a / (b + c))^2])
227         if angle == 'alpha':
228             return math.sqrt(self.b * self.c * (1 - (self.a / (self.b +
229                                     ↪ self.c))**2))
230         elif angle == 'beta':
231             return math.sqrt(self.a * self.c * (1 - (self.b / (self.a +
232                                     ↪ self.c))**2))
233         else:
234             return math.sqrt(self.a * self.b * (1 - (self.c / (self.a +
235                                     ↪ self.b))**2))
236
237     def get_incircle_radius(self):
238         """Calculate the radius of the incircle (rho).
239
240         Returns:
241             float: The incircle radius.
242         """
243         # Formula: rho = A / s
244         return self.get_area() / self.get_semiperimeter()
245
246     def get_excircle_radius(self, side='a'):

```

```

239         """Calculate the radius of the excircle (Ankreisradius rho_i)
           ↪ for a side.
240
241         Args:
242             side (str): The side identifier ('a', 'b', 'c') where the
           ↪ circle touches. Defaults to 'a'.
243
244         Returns:
245             float: The excircle radius.
246         """
247         # Formula: rho_a = A / (s - a)
248         s = self.get_semiperimeter()
249         side_val = getattr(self, side)
250         return self.get_area() / (s - side_val)
251
252     def get_circumcircle_radius(self):
253         """Calculate the radius of the circumcircle (r).
254
255         Returns:
256             float: The circumcircle radius.
257         """
258         # Formula: r = (a * b * c) / (4 * A)
259         return (self.a * self.b * self.c) / (4 * self.get_area())
260
261     def __repr__(self):
262         data = {k: round(v, 2) for k, v in self.__dict__.items() if not
           ↪ k.startswith('_')}
263         return f"{self.__class__.__name__}({data})"

```

22.2.1 Inspecting the Triangle Instance

When printing a Python object or evaluating it directly inside a notebook cell, the default output is typically a generic string showing the object's class name and its memory address. To provide a clear and meaningful summary of a triangle's properties, we can implement the special `__repr__` method.

The implementation above dynamically inspects the object's internal attribute dictionary (`self.__dict__`). To ensure the output remains clean and legible, it automatically filters out any private helper attributes (those starting with an underscore) and rounds all public side lengths and angles to two decimal places. This guarantees that whenever a **Triangle** instance is invoked, it displays a neat, readable overview of its current geometric state.

To test this, in the cell below a **Triangle** `triangle` is instantiated and then printed to the command line.

```

1 triangle = Triangle(3, 4, 5)
2 print(triangle)

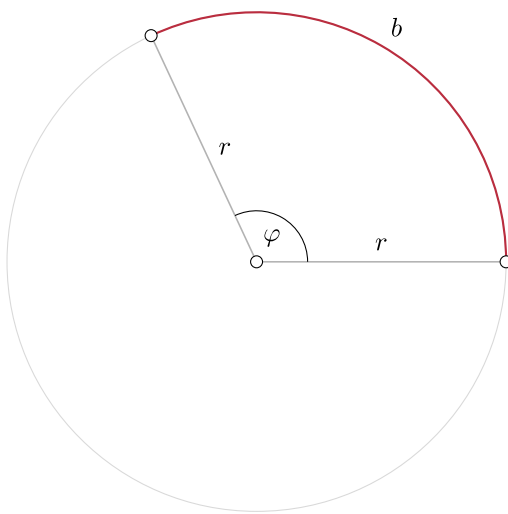
```

Triangle({'a': 3, 'b': 4, 'c': 5, 'alpha': 36.87, 'beta': 53.13, 'gamma': 90.0})

22.3 Coding Task: Circle Class

After studying the `Triangle` class you should be able, to implement a Python class `Circle` on your own. To do so, you'll find the mathematical properties of a circle below.

i Angles in Radian Measure



The radian measure $\widehat{\varphi}$ of an angle is the ratio of the length of the arc intersected by the angle φ on a circle to the radius of the circle. It is also denoted by $\text{arc } \varphi$.

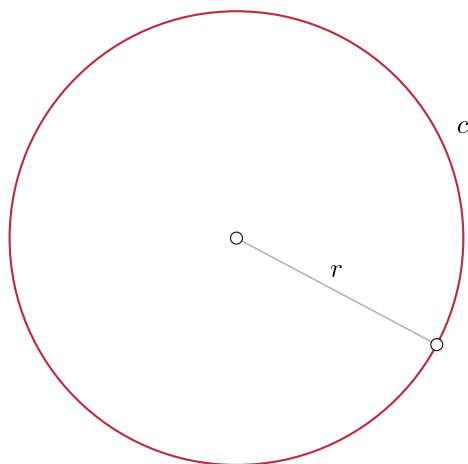
$$\widehat{\varphi} = \frac{b}{r} = \text{arc } \varphi = \frac{\pi}{180^\circ} \varphi \iff \varphi = \frac{180^\circ}{\pi} \widehat{\varphi}$$

On the unit circle ($r = 1$), we have: $\widehat{\varphi} = b$

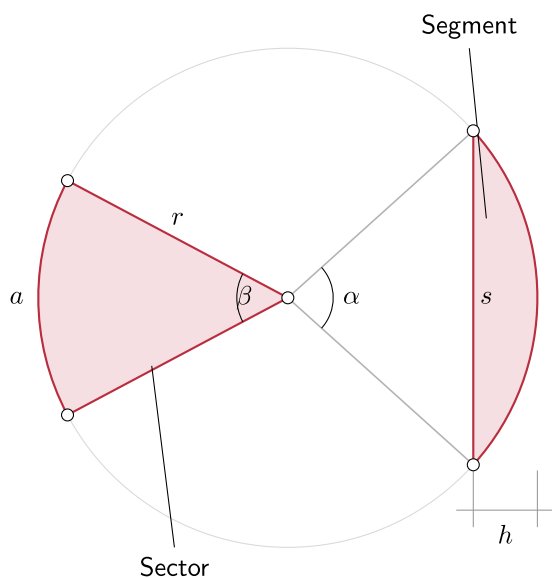
The unit of radian measure is the radian (rad). An angle of 1 rad corresponds to a circular arc that has the same length as the radius.^a

^aDurandi u. a., *Formeln, Tabellen, Begriffe*, p. 91.

To define a circle, either its radius or its circumference is sufficient. With those data the following elements can be calculated:



- *Circumference:* $c = 2\pi r$
- *Radius:* $r = \frac{c}{2\pi}$



- **Area:** $A = \pi r^2$
- **Arc Length:** $a = \hat{\beta}r = \frac{\pi}{180^\circ}\beta r$
- **Sector area:** $A = \frac{\hat{\beta}r^2}{2} = \frac{\pi\beta r^2}{360^\circ} = \frac{ar}{2}$
- **Chord length:** $s = 2r \sin \frac{\alpha}{2}$
- **Segment height:** $h = r(1 - \cos \frac{\alpha}{2})$
- **Segment area:** $A = \frac{r^2}{2}(\text{arc } \alpha - \sin \alpha)$

22.3.1 Implement Your own Circle Class

In the following cell you'll find the skeleton of a `Circle` class. Implement the missing parts. You don't need to implement your own `__repr__` method.

⚠ Computers calculate angles in **radians** (see box above). To convert angles given in degrees use the function `math.radians()`

```
1 import math
2
3 class Circle:
4     def __init__(self, r=None, c=None):
5         """Initialises a Circle via either radius or circumference.
6
7         Raises:
8             ValueError: If neither or both parameters are provided.
9         """
10        if r is not None and c is not None:
11            raise ValueError("Provide either radius 'r' OR circumference
12                               ↪ 'c', not both.")
13        if r == None and c == None:
14            raise ValueError("A circle must be initialised with either
15                               ↪ 'r' or 'c'.")
16
17        if r is not None:
18            if r < 0:
19                raise ValueError("Radius cannot be negative.")
20            c = 2 * math.pi * r
21        elif c is not None:
22            if c < 0:
23                raise ValueError("Circumference cannot be negative.")
24            r = c / (2 * math.pi)
25
26        self.r = r
27        self.c = c
28
29    def get_area(self):
30        return math.pi * self.r ** 2
31
32    def get_arc_length(self, beta):
33        """Calculates arc length for angle beta in degrees."""
34        return (math.pi / 180) * beta * self.r
35
36    def get_sector_area(self, beta):
37        """Calculates sector area for angle beta in degrees."""
38        a = self.get_arc_length(beta)
39        return (a * self.r) / 2
```



```

39     def get_chord_length(self, alpha):
40         """Calculates chord length for angle alpha in degrees."""
41         alpha_rad = math.radians(alpha)
42         return 2 * self.r * math.sin(alpha_rad / 2)
43
44     def get_segment_height(self, alpha):
45         """Calculates segment height for angle alpha in degrees."""
46         alpha_rad = math.radians(alpha)
47         return self.r * (1 - math.cos(alpha_rad / 2))
48
49     def get_segment_area(self, alpha):
50         """Calculates segment area for angle alpha in degrees."""
51         alpha_rad = math.radians(alpha) # arc alpha is just the angle
    ↪ in radians!
52         return (self.r**2 / 2) * (alpha_rad - math.sin(alpha_rad))

```

```

1 # test
2 circle = Circle(r=1)
3 alpha = 180
4 beta = 90
5 print(round(circle.get_area(), 2))
6 print(round(circle.get_arc_length(beta), 2))
7 print(round(circle.get_chord_length(beta), 2))
8 print(round(circle.get_segment_area(alpha), 2))
9 print(round(circle.get_segment_height(alpha), 2))

```

23 Functions: Exercises

23.1 Area of a trapezium

Given the height h and the parallel sides a and b of a trapezium, its area is calculated by the following formula:

$$A = \frac{a + b}{2} \cdot h$$

Implement this formula as a Python function in the cell below.

```
1  def trapezium_area(a: float, b: float, h: float) -> float:
2      """Calculate the area of a trapezium.
3
4      Args:
5          a (float): The length of the first parallel side.
6          b (float): The length of the second parallel side.
7          h (float): The perpendicular height of the trapezium.
8
9      Returns:
10         float: The calculated area of the trapezium.
11     """
12     return ((a + b) / 2) * h
13
14 # test case
15
16 print(trapezium_area(2, 4, 6))
```

23.2 Volume of a right circular cylinder

The volume of a right circular cylinder is calculated by the formula

$$V = \pi \cdot r^2 \cdot h$$

where

- π is a constant with a value of 3.14,

- r denotes the radius of the base, and
- h denotes the height of the right circular cylinder.

Implement the formula as a Python function in the cell below.

```

1  def get_cylinder_volume(r: float, h: float) -> float:
2      """Calculate the volume of a cylinder.
3
4      Args:
5          r (float): The radius of the cylinder's base.
6          h (float): The height of the cylinder.
7
8      Returns:
9          float: The calculated volume of the cylinder.
10     """
11     PI = 3.14 # Approximated value of pi to two decimal
12              # places as a local constant
13     return PI * r**2 * h
14
15 # test case
16
17 get_cylinder_volume(1, 1)

```

23.3 Progression decisions

The ordinance for promoting students of a Gymnasium in the canton of Zurich reads as follows:

§ 9. Bedingungen

Die Bedingungen für die definitive Aufnahme bzw. Promotion sind erfüllt, wenn in allen Promotionsfächern, die im betreffenden Semester unterrichtet werden,

- die doppelte Summe aller Notenabweichungen von 4 nach unten nicht grösser ist als die Summe aller Notenabweichungen von 4 nach oben und
- nicht mehr als drei Noten unter 4 erteilt werden.

Implement the check of the conditions for a progression as a Python function in the cell below.

```

1  def does_the_student_pass(grades: list[float]) -> bool:
2      """Determine whether a student qualifies for definitive promotion.
3
4      This function implements specific academic regulations based on a

```

```

5     grading scale where 4.0 is the passing mark. A student passes if
6     both of the following conditions are met:
7     1. Twice the sum of all grade deviations below 4.0 is not greater
8         than the sum of all grade deviations above 4.0.
9     2. No more than three grades are below 4.0.
10
11     Args:
12         grades (list[float]): A list of grades achieved in the
13             subjects relevant for promotion in the semester.
14
15     Returns:
16         bool: True if the promotion conditions are fulfilled,
17             False otherwise.
18     """
19     sub_fours = [g - 4 for g in grades if g < 4]
20     sup_fours = [g - 4 for g in grades if g >= 4]
21     aggregated_sub_fours = abs(2 * sum(sub_fours))
22     aggregated_sup_fours = sum(sup_fours)
23     compensation = aggregated_sub_fours <= aggregated_sup_fours
24     number_sub_fours = len(sub_fours)
25     return compensation and (number_sub_fours <= 3)
26
27 # test cases
28 progress_secured = [5, 4.5, 4.5, 3.5, 5, 4, 4, 4, 4.5, 5, 5, 4]
29 failed_a = [5, 4.5, 4.5, 3.5, 4, 4, 4, 3, 4.5, 4, 4, 4]
30 failed_b = [5, 5, 4.5, 3.5, 4, 3.5, 4, 3, 4.5, 3.5, 5, 5.5]
31
32 print(f'The student progresses:
33     ↳ {does_the_student_pass(progress_secured)}')
34 print(f'The student progresses a): {does_the_student_pass(failed_a)}')
35 print(f'The student progresses b): {does_the_student_pass(failed_b)}')

```

24 Triangulation: Inheriting from Triangle

Land surveying has traditionally rested on the application of trigonometric functions to triangles — which is why it is also called *triangulation*. In this chapter we survey a distance across inaccessible terrain and, in doing so, revisit the whole progression of this course:

1. **Helper functions** that call one another (as in Kapitel 21.1, *Functions within functions*).
2. **Bundling** those helpers into a class (as with `TriangleSSS`, see Kapitel 22.1).
3. **Inheritance** — the new idea: a survey triangle *is a Triangle*, so it can inherit everything the `Triangle` class already knows and add only what surveying needs.

Trigonometric Fundamentals

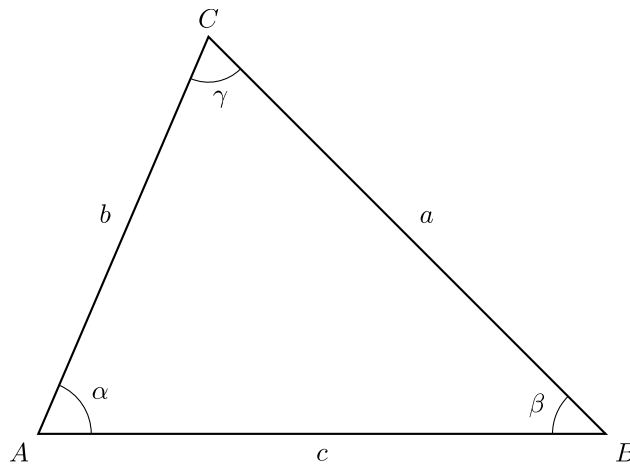


Abbildung 24.1: General Triangle.

In a general triangle (Abbildung 24.1) the sine rule and the cosine rule hold:

Sine rule

$$\frac{a}{\sin \alpha} = \frac{b}{\sin \beta} = \frac{c}{\sin \gamma}$$

Cosine rule

$$c^2 = a^2 + b^2 - 2 a b \cos \gamma$$

24.1 The Surveying Problem

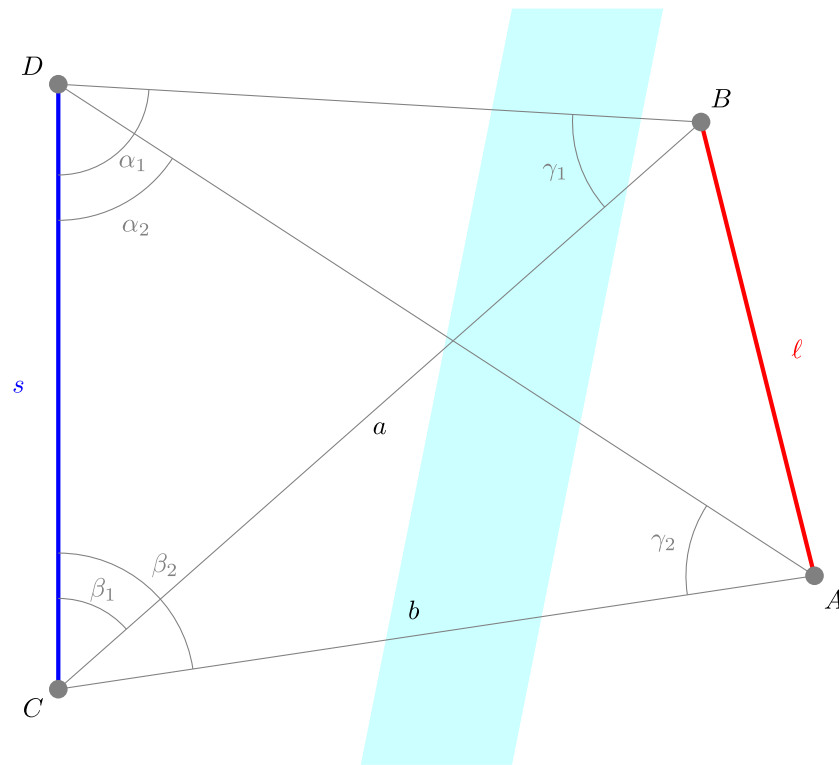


Abbildung 24.2: Schematic diagram.

We are looking for the distance $\overline{AB}$ (ℓ) across inaccessible terrain (Abbildung 24.2). What we *can* measure is the baseline $\overline{CD}$ (s) and the angles α_1, α_2 (at point D) and β_1, β_2 (at point C).

The idea is to link the baseline $s = \overline{CD}$ to the sought distance by means of **two auxiliary triangles**. From triangle CDB we obtain the leg $\overline{CB}$, from triangle CDA the leg $\overline{CA}$. Both share vertex C ; the angle enclosed between them is $\beta_2 - \beta_1$. The target distance then follows from the cosine rule in triangle CAB :

$$\ell = \sqrt{a^2 + b^2 - 2ab \cos(\beta_2 - \beta_1)}$$

24.2 Stage 1 — Helper Functions

We begin exactly as in the Heron example: a *main* function `triangulation()` that calls several *helper* functions. This keeps each helper small, readable and independently testable.

Three helpers are needed:

- `angle_converter()` — bearings may be read from a compass calibrated in degrees, in artillery **mils** (A‰) or in **gon**. This helper normalises them to degrees.
- `angle_diff()` — each interior angle is the *modular* difference of two compass bearings. Using `% 360` automatically handles the case where the two readings straddle north.
- `sine_rule()` — returns the side opposite a given angle.

Default Values and Modular Subtraction

`angle_diff()` relies on modular arithmetic: with a positive divisor, Python's `%` operator always returns a value in $[0, 360)$, so the classical extra step “*if the result is negative, add 360*” becomes unnecessary. For example $40^\circ - 110^\circ = -70 \equiv 290 \pmod{360}$.

```

1  import math

1  def angle_converter(angle: float,
2                      mil: bool = False,
3                      gon: bool = False) -> float:
4      """Normalises an angle to degrees.
5
6      Supports input in degrees (default), artillery mils or gon. Only one
7      alternative mode may be active at a time.
8
9      Args:
10         angle (float): The angle value to convert.
11         mil (bool, optional): If True,
12             interpret `angle` as artillery mils
13             (6400 mil = 360°). Defaults to False.
14         gon (bool, optional): If True, interpret `angle` as gon
15             (400 gon = 360°). Defaults to False.
16
17     Returns:
18         float: The angle expressed in degrees.
19
20     Raises:
21         ValueError: If both `mil` and `gon` are True.
22     """
23     if mil and gon:
24         raise ValueError("Only 'mil=True' (artillery) OR \
25                             'gon=True' may be selected.")
26
27     if mil:
28         return angle * (360 / 6400)
29     elif gon:
30         return angle * (360 / 400)

```

```

31     else:
32         return angle

```

```

1  def angle_diff(bearing1: float, bearing2: float) -> float:
2      """Modular difference (bearing1 - bearing2) modulo 360 degrees.
3
4      Automatically accounts for crossing north by mapping negative
5      differences into the positive cycle [0, 360).
6
7      Args:
8          bearing1 (float): The first compass bearing, in degrees.
9          bearing2 (float): The bearing to subtract, in degrees.
10
11     Returns:
12         float: The interior angle in the range [0, 360) degrees.
13     """
14     return (bearing1 - bearing2) % 360

```

```

1  def sine_rule(length: float,
2               opposite_angle: float,
3               ref_angle: float) -> float:
4      """Calculates a side of a triangle using the sine rule.
5
6      Determines the side opposite `opposite_angle`, starting from the
7      known side `length` opposite `ref_angle`:
8
9          required side = length * sin(opposite_angle) / sin(ref_angle)
10
11     Args:
12         length (float): The known side.
13         opposite_angle (float): Angle opposite the side we are looking
14         for, in degrees.
15         ref_angle (float): Angle opposite the known side `length`, in
16         degrees.
17
18     Returns:
19         float: The length of the side we are looking for.
20     """
21     return (length * math.sin(math.radians(opposite_angle)) /
22            math.sin(math.radians(ref_angle)))

```

With the helpers in place, the main function forms the four interior angles from the eight bearings, builds the two auxiliary triangles via the sine rule, and closes with the cosine

rule.

```
1  def triangulation(s: float,
2                      alpha1_a: float, alpha1_b: float,
3                      alpha2_a: float, alpha2_b: float,
4                      beta1_a: float, beta1_b: float,
5                      beta2_a: float, beta2_b: float,
6                      mil: bool = False, gon: bool = False) -> float:
7      """Calculates an inaccessible distance by triangulation from compass
8          ↪ bearings.
9
10         From the accessible baseline CD (`s`) and four pairs of azimuth
11         bearings, two auxiliary triangles are formed. The sine rule yields
12         the legs CB and CA; the cosine rule in triangle C-A-B then gives
13         the target distance, the enclosed angle at C being beta2 - beta1.
14
15         Args:
16             s (float): The measured, accessible baseline (CD).
17             alpha1_a, alpha1_b (float): Bearings for partial
18             angle alpha 1 (at D).
19             alpha2_a, alpha2_b (float): Bearings for partial
20             angle alpha 2 (at D).
21             beta1_a, beta1_b (float): Bearings for partial
22             angle beta 1 (at C).
23             beta2_a, beta2_b (float): Bearings for partial
24             angle beta 2 (at C).
25             mil (bool, optional): Interpret all bearings as mils.
26             Defaults to False.
27             gon (bool, optional): Interpret all bearings as gon.
28             Defaults to False.
29
30         Returns:
31             float: The length of the inaccessible target distance (l).
32             """
33         # Each pair of bearings gives one interior angle (modular
34         # difference), normalised to degrees.
35         bearings = [alpha1_a, alpha1_b, alpha2_a, alpha2_b,
36                     beta1_a, beta1_b, beta2_a, beta2_b]
37         partial = [angle_diff(bearings[i], bearings[i + 1])
38                     for i in range(0, len(bearings), 2)]
39         alpha1, alpha2, beta1, beta2 = (angle_converter(a, mil, gon)
40                                         for a in partial)
41
42         # Auxiliary triangles on the baseline s: gamma lies opposite s.
```

```

42     gamma1 = 180 - (alpha1 + beta1)
43     gamma2 = 180 - (alpha2 + beta2)
44
45     # Sine rule -> the legs CB and CA (opposite alpha1 and alpha2).
46     side_cb = sine_rule(s, alpha1, gamma1)
47     side_ca = sine_rule(s, alpha2, gamma2)
48
49     # Cosine rule in triangle C-A-B; enclosed angle at
50     # C is beta2 - beta1.
51     return math.sqrt(side_cb**2 + side_ca**2
52                      - 2 * side_cb * side_ca
53                      * math.cos(math.radians(beta2 - beta1)))

```

```

1  # Worked example: alpha1 = 50°, alpha2 = 30°, beta1 = 60°, beta2 = 100°
2  distance = triangulation(100,
3                          50, 0, 30, 0, # alpha1, alpha2 at D
4                          60, 0, 100, 0) # beta1, beta2 at C
5  print(round(distance, 2))

```

52.48

24.3 Stage 2 — Bundling into a Class

The helper functions work, but the measured data (baseline and bearings) travel through the code as a long list of loose arguments. Just as we moved from free functions to the `TriangleSSS` class, we can **bundle** the data and the calculation into a single object. The helpers become *private methods* (leading underscore), mirroring `_get_semiperimeter()`.

```

1  class Survey:
2      """Bundles a triangulation measurement into a single object.
3
4      Attributes:
5          baseline (float): The accessible baseline CD (s).
6          bearings_D (tuple): Four bearings measured at
7          D (alpha1, alpha2).
8          bearings_C (tuple): Four bearings measured at C (beta1, beta2).
9      """
10
11     def __init__(self, baseline, bearings_D, bearings_C,
12                 mil=False, gon=False):
13         """Stores the measured quantities on the instance.

```

```

14
15     Args:
16         baseline (float): The accessible baseline CD.
17         bearings_D (tuple[float, float, float, float]):
18             Bearing pairs for alpha1 and alpha2, measured at D.
19         bearings_C (tuple[float, float, float, float]):
20             Bearing pairs for beta1 and beta2, measured at C.
21         mil (bool, optional): Interpret bearings as mils.
22             Defaults to False.
23         gon (bool, optional): Interpret bearings as gon.
24             Defaults to False.
25     """
26     self.baseline = baseline
27     self.bearings_D = bearings_D
28     self.bearings_C = bearings_C
29     self.mil = mil
30     self.gon = gon
31
32     def _angle(self, bearing1, bearing2):
33         """Private helper: interior angle from two bearings,
34             in degrees."""
35         return angle_converter(angle_diff(bearing1, bearing2),
36                                 self.mil, self.gon)
37
38     def get_distance(self):
39         """Calculates the inaccessible target distance AB.
40
41         Returns:
42             float: The length of the target distance (l).
43         """
44         alpha1 = self._angle(self.bearings_D[0], self.bearings_D[1])
45         alpha2 = self._angle(self.bearings_D[2], self.bearings_D[3])
46         beta1 = self._angle(self.bearings_C[0], self.bearings_C[1])
47         beta2 = self._angle(self.bearings_C[2], self.bearings_C[3])
48
49         side_cb = sine_rule(self.baseline, alpha1, 180 -
50                             (alpha1 + beta1))
51         side_ca = sine_rule(self.baseline, alpha2, 180 -
52                             (alpha2 + beta2))
53
54         return math.sqrt(side_cb**2 + side_ca**2
55                           - 2 * side_cb * side_ca *
56                             math.cos(math.radians(beta2 - beta1)))

```

```

1 survey = Survey(baseline=100,
2                 bearings_D=(50, 0, 30, 0),
3                 bearings_C=(60, 0, 100, 0))
4 print(round(survey.get_distance(), 2))

```

52.48

24.4 Stage 3 — Inheritance: an Auxiliary Triangle *is a* Triangle

Look closely at what `get_distance()` does: it builds two triangles on the baseline and reads a leg off each. But we *already have* a fully featured **Triangle** class from the previous chapter — it can solve a triangle from a side and two angles (the **ASA** case) and exposes `get_area()`, `get_semiperimeter()`, `__repr__` and more.

Rather than re-deriving the legs by hand, we let a new class **inherit** from **Triangle**. An auxiliary survey triangle *is a* triangle — the textbook signal for inheritance.

The **is-a** Relationship and `super()`

When a class `AuxiliaryTriangle(Triangle)` is written, `AuxiliaryTriangle` becomes a **subclass** (child) of `Triangle`. It automatically gains every method of the parent. Inside `__init__`, the call `super().__init__(...)` runs the parent's constructor — here the parent even *solves* the triangle for us, filling in all missing sides and angles. We then add only the one small method surveying needs.

Contrast this with *composition* (a `Survey` that merely *has* two triangles): inheritance is the right tool when the new type genuinely **is a** specialised kind of the old one.

First, the **Triangle** class from the previous chapter must be available. In your book this cell is already present; it is repeated here so the notebook runs on its own.

```

1 class Triangle:
2     """Represents a triangle, solvable from valid congruence criteria
3     ↪ (abridged).
4
5     Only the ASA/SAA and SSS paths needed for surveying are shown here;
6     the full class with all four congruence cases is developed in the
7     previous chapter.
8     """
9     def __init__(self, a=None, b=None, c=None,
10                  alpha=None, beta=None, gamma=None):
11         self.a, self.b, self.c = a, b, c
12         self.alpha, self.beta, self.gamma = alpha, beta, gamma
13

```

```

14         given_sides = [k for k, v in
15                         [('a', a), ('b', b), ('c', c)] if v is not None]
16         given_angles = [k for k, v in
17                         [('alpha', alpha), ('beta', beta), ('gamma',
↪ gamma)]]
18                         if v is not None]
19         if len(given_sides) + len(given_angles) != 3:
20             raise ValueError("A triangle must be defined by exactly \
21                               3 parameters.")
22
23         self._solve_triangle()
24
25     def _solve_triangle(self):
26         """Fills in all missing sides and angles
27         (SSS and ASA/SAA paths)."""
28         def rad(d): return math.radians(d)
29         def deg(r): return math.degrees(r)
30
31         if self.a and self.b and self.c: # SSS
32             self.alpha = deg(math.acos((self.b**2 +
33                                         self.c**2 - self.a**2)
34                                         / (2 * self.b * self.c)))
35             self.beta = deg(math.acos((self.a**2 + self.c**2 -
36                                         self.b**2)
37                                         / (2 * self.a * self.c)))
38             self.gamma = 180.0 - self.alpha - self.beta
39
40         elif sum(s is not None for s in [self.a, self.b, self.c]) == 1:
41             ↪ # ASA / SAA
42             if self.alpha is None:
43                 self.alpha = 180.0 - self.beta - self.gamma
44             elif self.beta is None:
45                 self.beta = 180.0 - self.alpha - self.gamma
46             elif self.gamma is None:
47                 self.gamma = 180.0 - self.alpha - self.beta
48
49             known_s = [s for s in ['a', 'b', 'c']
50                         if getattr(self, s) is not None][0]
51             known_a = {'a': 'alpha', 'b': 'beta', 'c': 'gamma'}[known_s]
52             factor = getattr(self, known_s) / math.sin(rad(getattr(self,
↪ known_a)))
53             if self.a is None:
54                 self.a = factor * math.sin(rad(self.alpha))
55             if self.b is None:

```

```

55         self.b = factor * math.sin(rad(self.beta))
56         if self.c is None:
57             self.c = factor * math.sin(rad(self.gamma))
58
59     def get_semiperimeter(self):
60         return (self.a + self.b + self.c) / 2
61
62     def get_area(self):
63         s = self.get_semiperimeter()
64         return math.sqrt(s * (s - self.a) * (s - self.b) * (s - self.c))
65
66     def __repr__(self):
67         data = {k: round(v, 2) for k, v in self.__dict__.items()
68                 if not k.startswith('_')}
69         return f"{self.__class__.__name__}({data})"

```

Now the subclass. Each auxiliary triangle sits on the baseline s (side c) with the two measured base angles α (at D) and β (at C) — precisely the **ASA** case. `super().__init__()` hands these to the parent, which solves the triangle completely. The leg we want, $\overline{CB}$ resp. $\overline{CA}$, is the side opposite α , i.e. side a .

```

1  class AuxiliaryTriangle(Triangle):
2      """A survey triangle on the shared baseline – a specialised
3      ↪ Triangle.
4
5      Constructed from the baseline and the two base angles
6      (the ASA case), it inherits everything Triangle can do and
7      adds one surveying method.
8      """
9
10     def __init__(self, baseline: float, alpha: float, beta: float):
11         """Builds the auxiliary triangle from baseline and base angles.
12
13         Args:
14             baseline (float): The shared baseline  $s$  (becomes side  $c$ ).
15             alpha (float): Base angle at  $D$ , opposite the leg we seek.
16             beta (float): Base angle at  $C$ .
17         """
18         # ASA: side  $c$  = baseline, adjacent angles  $\alpha$  and  $\beta$ .
19         super().__init__(c=baseline, alpha=alpha, beta=beta)
20
21     def leg_from_C(self) -> float:
22         """The leg from  $C$  to the far target point
23         (opposite  $\alpha$ , i.e. side  $a$ )."""

```

```
23         return self.a
```

The Survey class becomes noticeably simpler: it no longer computes legs itself but delegates to two AuxiliaryTriangle objects, then closes with the cosine rule.

```
1  class TriangulationSurvey:
2      """Chains two AuxiliaryTriangles on one baseline to find AB."""
3
4      def __init__(self, baseline, bearings_D, bearings_C,
5                  mil=False, gon=False):
6          self.baseline = baseline
7          self.bearings_D = bearings_D
8          self.bearings_C = bearings_C
9          self.mil = mil
10         self.gon = gon
11
12     def _angle(self, bearing1, bearing2):
13         return angle_converter(angle_diff(bearing1, bearing2),
14                                 self.mil, self.gon)
15
16     def get_distance(self):
17         """Calculates the target distance via two
18             inherited triangles."""
19         alpha1 = self._angle(self.bearings_D[0], self.bearings_D[1])
20         alpha2 = self._angle(self.bearings_D[2], self.bearings_D[3])
21         beta1 = self._angle(self.bearings_C[0], self.bearings_C[1])
22         beta2 = self._angle(self.bearings_C[2], self.bearings_C[3])
23
24         triangle_cdb = AuxiliaryTriangle(self.baseline, alpha1, beta1)
25         triangle_cda = AuxiliaryTriangle(self.baseline, alpha2, beta2)
26
27         side_cb = triangle_cdb.leg_from_C()
28         side_ca = triangle_cda.leg_from_C()
29
30         return math.sqrt(side_cb**2 + side_ca**2
31                         - 2 * side_cb * side_ca *
32                            math.cos(math.radians(beta2 - beta1)))
33
34
35 survey = TriangulationSurvey(baseline=100,
36                             bearings_D=(50, 0, 30, 0),
37                             bearings_C=(60, 0, 100, 0))
38 print(round(survey.get_distance(), 2))
```

52.48

Because `AuxiliaryTriangle` *is a* `Triangle`, every inherited method just works — and `isinstance()` confirms the relationship.

```
1 aux = AuxiliaryTriangle(100, 50, 60)
2 print(aux)                      # inherited __repr__
3 print("Area:", round(aux.get_area(), 2)) # inherited get_area()
4 print("Is a Triangle:", isinstance(aux, Triangle))
```

```
AuxiliaryTriangle({'a': 81.52, 'b': 92.16, 'c': 100, 'alpha': 50, 'beta': 60, 'gamma': 70.0})
Area: 3529.95
Is a Triangle: True
```

24.5 Coding Task

Coding Task: A Height by Triangulation

The same principle measures the **height of an inaccessible tower**. From two points P and Q on level ground, a distance d apart and in line with the tower's base, you measure the elevation angles φ_P and φ_Q to the top.

Implement a class `TowerSurvey` that inherits from `Triangle`. Build the auxiliary triangle PQT (where T is the top) using `super().__init__()` with the appropriate ASA data, inherit the sine-rule solution for the slant distance $\overline{QT}$, and add one method `get_height()` that returns the tower height. Reuse — do not re-derive — the machinery you already inherit.

Teil V

Object Oriented Programming

25 Interest-Bearing Accounts: Inheriting from Account

Double-entry bookkeeping records every business transaction as an *entry* that debits one account and credits another. In this chapter we build a small ledger in Python and, in doing so, revisit the whole progression of this course:

1. **A data-carrying class** that models a single booking (**Entry**).
2. **Bundling** state and behaviour into a class that keeps a running balance (**Account**).
3. **Inheritance** — the new idea: a current account *is an Account*, so it can inherit everything the **Account** class already knows and add only what interest-bearing accounts need.

Bookkeeping Fundamentals

Every entry touches two accounts. By convention an account's balance (its *Saldo*) moves according to which side the entry lands on:

- a **debit** on the account increases its balance;
- a **credit** on the account decreases it.

The *interest* on a balance held over a period follows the simple day-count formula

$$\text{interest} = \text{balance} \times \frac{\text{rate}}{360} \times \text{days},$$

where the year is taken as 360 days (the *30/360* convention common in Swiss practice).

25.1 The Design at a Glance

Before writing any code, it helps to picture where we are heading. Abbildung 25.1 is a **UML class diagram** of the three classes in this chapter and how they relate.

```
1 classDiagram
2     class Entry {
3         +int debit
4         +int credit
5         +float amount
6         +date date
7         +dict accounting_scheme
8         +__str__() str
```

```

9      }
10     class Account {
11         +str name
12         +int accountNr
13         +float startAmount
14         +list~Entry~ entries
15         +float saldo
16         +add_entry(entry) None
17         +calculate_saldo() float
18     }
19     class CurrentAccount {
20         +float interest_rate
21         +list saldi
22         +add_entry(entry) None
23         +calculate_interest(end_date) float
24     }
25     Account <|-- CurrentAccount : is-a (inherits)
26     Account o-- Entry : has-many

```

Two relationships appear here, and telling them apart is the whole point of the chapter:

- The **hollow triangle** arrow (`Account <|-- CurrentAccount`) is *inheritance*: a `CurrentAccount` is an `Account`. It inherits every attribute and method of the parent and adds `interest_rate`, `saldi` and `calculate_interest`. Note that `add_entry` appears in *both* boxes — the subclass **overrides** it.
- The **hollow diamond** arrow (`Account o-- Entry`) is *aggregation*: an `Account` **has** a list of `Entry` objects. This is composition, not inheritance — an account is not a kind of entry.

Reading a UML Class Diagram

Each box has three compartments: the class **name**, its **attributes** (data) and its **methods** (behaviour). A + marks a public member. The arrow *type* encodes the relationship — an *is-a* link (inheritance) uses a hollow triangle pointing at the parent, whereas a *has-a* link (aggregation) uses a hollow diamond at the containing class.

25.2 Stage 1 — A Class for a Single Entry

We begin with the smallest unit of the ledger: one booking. An `Entry` stores the debit account number, the credit account number, an amount and a date. It also carries the chart of accounts (*Kontenrahmen*) so that it can print itself in a human-readable form.

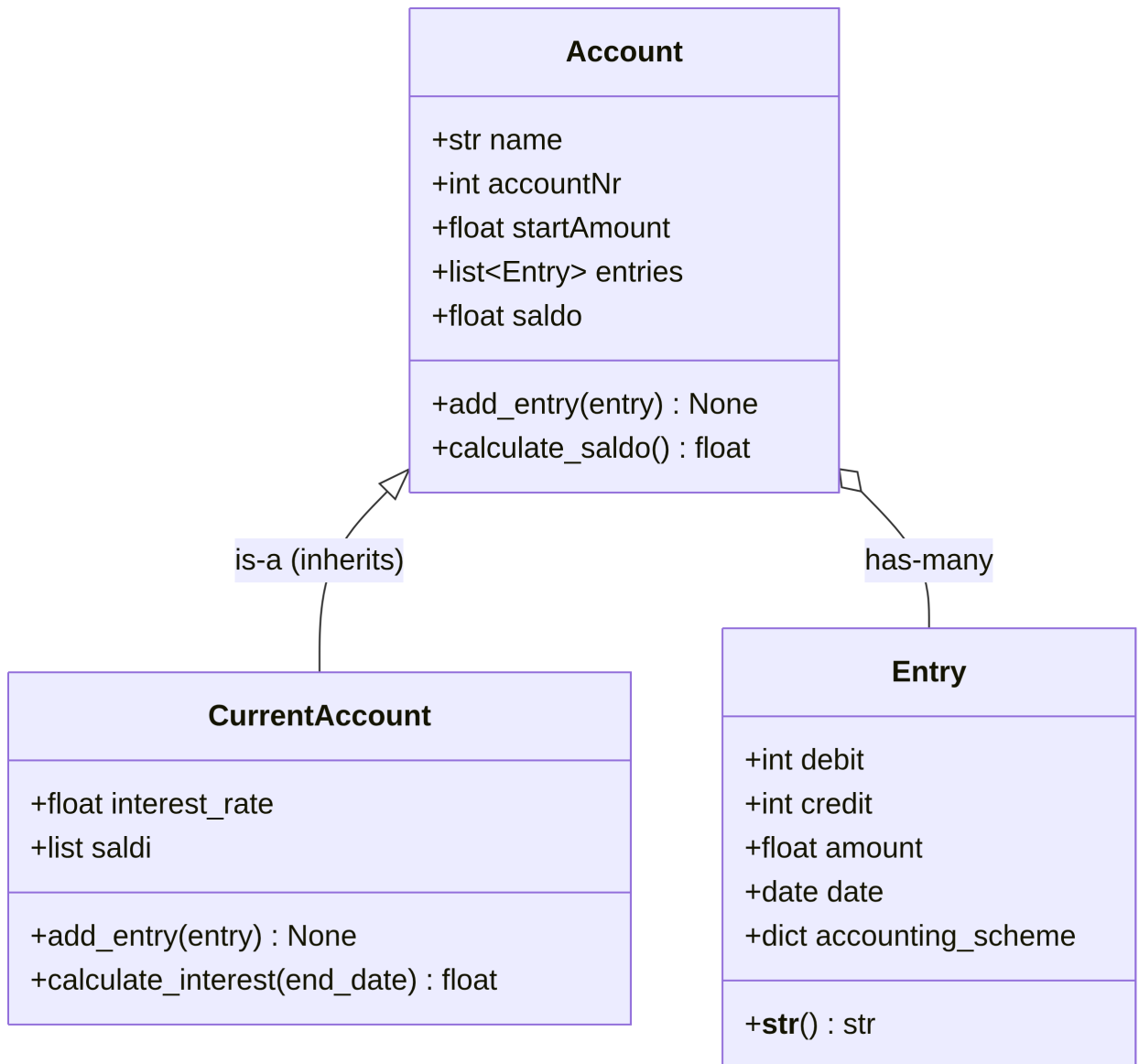


Abbildung 25.1: UML class diagram: CurrentAccount inherits from Account; each Account holds Entry objects.

The `__str__` Method

When an object appears in `print()`, Python looks for a `__str__` method and uses whatever string it returns. Defining `__str__` lets an `Entry` describe itself — date, both account names and the amount — instead of showing the default `<Entry object at 0x...>`.

```
1  from datetime import date

1  class Entry:
2      """A single double-entry booking (one debit, one credit).
3
4      Attributes:
5          debit (int): Account number debited.
6          credit (int): Account number credited.
7          amount (float): The amount booked.
8          date (date): The date of the booking.
9      """
10
11     def __init__(self, debit: int,
12                  credit: int,
13                  amount: float,
14                  date: date) -> None:
15         self.debit = debit
16         self.credit = credit
17         self.amount = amount
18         self.date = date
19         # Abridged chart of accounts (Kontenrahmen KMU) for readable
20         # ↪ output.
21         self.accounting_scheme = {
22             1000: "Kasse",
23             1020: "Bankguthaben",
24             2000: "Verbindlichkeiten aus Lieferungen und Leistungen
25             ↪ (Kreditoren)",
26             6950: "Finanzertrag",
27         }
28
29     def __str__(self) -> str:
30         text = (
31             f"{self.date.strftime('%d %b %Y')}: "
32             f"{self.accounting_scheme[self.debit]} "
33             f"({self.debit}) - {self.accounting_scheme[self.credit]} "
34             f"({self.credit}) {self.amount}"
35         )
```

```
34         return " ".join(text.split())
```

A first booking: 50.00 moved from *Bankguthaben* (1020) to *Kasse* (1000). Printing the object triggers `__str__`.

```
1 first_entry = Entry(1000, 1020, 50.00, date(2026, 8, 28))
2 print(first_entry)
```

25.3 Stage 2 — Bundling into an Account

A single entry is of little use on its own. An **account** collects the entries that touch it and can compute its balance at any time. We bundle that state (name, number, opening balance, list of entries) and the behaviour (`add_entry`, `calculate_saldo`) into one class.

`calculate_saldo` walks through every entry: an entry that **debits** this account raises the balance, one that **credits** it lowers the balance, and entries touching neither are skipped.

```
1 class Account:
2     """A ledger account that collects entries and computes its balance.
3
4     Attributes:
5         name (str): Human-readable account name.
6         accountNr (int): The account number.
7         startAmount (float): Opening balance.
8         entries (list[Entry]): All entries booked to this account.
9         saldo (float | None): Most recently computed balance.
10    """
11
12    def __init__(self,
13                  name: str,
14                  accountNr: int,
15                  startAmount: float) -> None:
16        self.name = name
17        self.accountNr = accountNr
18        self.startAmount = startAmount
19        self.entries = []
20        self.saldo = None
21
22    def add_entry(self, entry: Entry) -> None:
23        """Appends an entry to the account and reports it.
24
25        Args:
```

```

26         entry (Entry): The booking to add.
27     """
28     self.entries.append(entry)
29     print(f'Entry "{entry}" added.')
30
31     def calculate_saldo(self) -> float:
32         """Computes the balance from the opening amount and all entries.
33
34         A debit on this account increases the balance; a credit
35         decreases it.
36
37         Returns:
38             float: The resulting balance.
39         """
40         saldo = self.startAmount
41         for entry in self.entries:
42             if entry.debit == self.accountNr:
43                 saldo += entry.amount
44             elif entry.credit == self.accountNr:
45                 saldo -= entry.amount
46             else:
47                 continue
48         print(f'Saldo: {saldo}')
49         self.saldo = saldo
50         return saldo

```

We open the cash account 1000 with 10.00, book two entries and read off the balance.

```

1  a1000 = Account('Cash', 1000, 10.00)
2  a1000.add_entry(Entry(1000, 1020, 50.0, date(2026, 8, 28)))
3  a1000.add_entry(Entry(2000, 1000, 5.00, date(2026, 8, 28)))
4  a1000.calculate_saldo()

```

25.4 Stage 3 — Inheritance: a Current Account *is an* Account

A **current account** (*Kontokorrent*) behaves exactly like an ordinary account — it collects entries and has a balance — but it does something extra: it earns **interest**. That is the textbook signal for inheritance: a current account *is an* account, plus a little more. In the UML diagram (Abbildung 25.1) this is the hollow-triangle link from **CurrentAccount** to **Account**.

Rather than copying all of **Account**, we let a new class **inherit** from it and add only what interest requires.

The is-a Relationship and `super()`

Writing `class CurrentAccount(Account)` makes `CurrentAccount` a **subclass** (child) of `Account`. It automatically gains every method of the parent — `add_entry`, `calculate_saldo` and the constructor. Inside its own `__init__`, the call `super().__init__(...)` runs the parent's constructor, so `name`, `accountNr`, `startAmount`, `entries` and `saldo` are set up for us. We then add just the new attributes `interest_rate` and `saldi`.

Contrast this with *composition* (the hollow-diamond **has-a** link between `Account` and `Entry`): inheritance is the right tool when the new type genuinely **is a** specialised kind of the old one.

25.4.1 A First Version

We build the subclass step by step. The constructor takes one extra argument, `interest_rate`, hands the rest to the parent via `super().__init__(...)`, and keeps a dated history of balances (`saldi`) so that interest can later be computed period by period. `add_entry` is **overridden**: it first calls `super().add_entry(entry)` to keep the inherited behaviour, then records the new balance together with the entry's date.

For `calculate_interest` we write the calculation out in full, in the most direct way we can think of — index by index. It works; we will scrutinise it afterwards.

```
1  class CurrentAccount(Account):
2      """An interest-bearing account – first version.
3
4      Inherits entry handling and balance calculation from Account and
5      adds an interest rate together with a dated history of balances.
6
7      Attributes:
8          interest_rate (float): Annual interest rate (e.g. 0.05 for 5%).
9          saldi (list[tuple[float, date]]): Balance history as
10             (balance, date) pairs, opened with the start amount.
11      """
12
13     def __init__(self,
14                 name: str,
15                 accountNr: int,
16                 startAmount: float,
17                 interest_rate: float) -> None:
18         super().__init__(name, accountNr, startAmount) # parent sets up
19         ↪ the account
20         self.interest_rate = interest_rate             # the new
21         ↪ attributes
22         self.saldi = []
```



```

21         self.saldi.append((startAmount, date(2026, 1, 1)))
22
23     def add_entry(self, entry: Entry) -> None:
24         """Extends Account.add_entry by recording the new dated
25         ↳ balance."""
26         super().add_entry(entry) #
27         ↳ inherited behaviour
28         self.saldi.append((self.calculate_saldo(), entry.date)) # plus
29         ↳ history
30
31     def calculate_interest(self, end_date=date(2026, 12, 31)) -> float:
32         """Computes 30/360 interest across the balance history.
33
34         Each balance earns interest over the number of days it was held,
35         i.e. until the next balance change (and finally until end_date).
36         If end_date is the year-end, the total is booked as
37         ↳ Finanzertrag.
38         """
39         number_of_days = []
40         interests = []
41         end_date = end_date
42         i = 0
43         saldi = [saldo[0] for saldo in self.saldi]
44         day_list = [saldo[1] for saldo in self.saldi]
45         while i < (len(day_list) - 1):
46             d = (day_list[i+1] - day_list[i]).days
47             number_of_days.append(d)
48             i += 1
49         rest_days = (end_date - day_list[-1]).days
50         number_of_days.append(rest_days)
51         for i in range(len(saldi)):
52             interest = (saldi[i] * self.interest_rate / 360) *
53             ↳ number_of_days[i]
54             interests.append(interest)
55         if end_date == date(2026, 12, 31):
56             entry = Entry(1020, 6950, sum(interests), end_date)
57             self.add_entry(entry)
58         return sum(interests)

```

We open a bank current account 1020 with 100.00 at 5 % interest, book a movement in March, and compute the interest to the year-end.

```

1 bank = CurrentAccount('Bank', 1020, 100.00, 0.05)
2 bank.add_entry(Entry(1020, 1000, 150.00, date(2026, 3, 1)))

```

```
3 print(bank.calculate_interest())
```

25.4.2 Refactoring `calculate_interest`

The first version is correct, but it does more bookkeeping of its own than the problem requires. Reading it closely, a few things stand out:

- Two accumulator lists (`number_of_days`, `interests`) are filled with `append` inside loops.
- A manual index `i` and a `while` loop walk through consecutive dates — a classic place for off-by-one errors.
- The line `end_date = end_date` does nothing.

Python lets us say the same thing more directly. The days between consecutive balances are just the differences of **adjacent pairs**, which `zip(day_list, day_list[1:])` produces without any index. Both loops then collapse into **list comprehensions**, and the intent — “each balance, times its days, times the daily rate” — becomes readable in one line.

Pairing Adjacent Elements with `zip`

`zip(seq, seq[1:])` pairs each element with its successor: from `[d0, d1, d2]` it yields `(d0, d1)` and `(d1, d2)`. Iterating over those pairs replaces the manual index arithmetic `day_list[i+1] - day_list[i]` and removes the risk of mis-counting the loop bounds.

The behaviour is identical — only `calculate_interest` changes; everything inherited from `Account` stays untouched.

```
1 class CurrentAccount(Account):
2     """An interest-bearing account – refactored version.
3
4     Identical behaviour to the first version; only calculate_interest
5     is rewritten using zip and list comprehensions.
6     """
7
8     def __init__(self,
9                 name: str,
10                accountNr: int,
11                startAmount: float,
12                interest_rate: float) -> None:
13         super().__init__(name, accountNr, startAmount)
14         self.interest_rate = interest_rate
15         self.saldi = []
16         self.saldi.append((startAmount, date(2026, 1, 1)))
17
```

```

18     def add_entry(self, entry: Entry) -> None:
19         """Extends Account.add_entry by recording the new dated
20         ↳ balance."""
21         super().add_entry(entry)
22         self.saldi.append((self.calculate_saldo(), entry.date))
23
24     def calculate_interest(self, end_date=date(2026, 12, 31)) -> float:
25         """Computes 30/360 interest across the balance history.
26
27         Args:
28             end_date (date, optional): End of the interest period.
29             Defaults to 31 December 2026.
30
31         Returns:
32             float: The total interest over the period.
33         """
34         saldi = [saldo[0] for saldo in self.saldi]
35         day_list = [saldo[1] for saldo in self.saldi]
36
37         # Days between consecutive balances (adjacent pairs via zip)...
38         number_of_days = [
39             (nxt - cur).days
40             for cur, nxt in zip(day_list, day_list[1:])
41         ]
42         # ... plus the remaining days from the last balance to end_date.
43         number_of_days.append((end_date - day_list[-1]).days)
44
45         # Charge each balance interest over the days it was held.
46         interests = [
47             saldo * self.interest_rate / 360 * days
48             for saldo, days in zip(saldi, number_of_days)
49         ]
50
51         if end_date == date(2026, 12, 31):
52             entry = Entry(1020, 6950, sum(interests), end_date)
53             self.add_entry(entry)
54         return sum(interests)

```

Running the same example confirms that the refactored version yields exactly the same interest — same result, clearer code.

```

1 bank = CurrentAccount('Bank', 1020, 100.00, 0.05)
2 bank.add_entry(Entry(1020, 1000, 150.00, date(2026, 3, 1)))
3 print(bank.calculate_interest())

```

Because `CurrentAccount` *is an* `Account`, every inherited method just works — and `isinstance()` confirms the relationship.

```
1 print("Is an Account:", isinstance(bank, Account))
2 print("Balance:", bank.saldo)
```

Literaturverzeichnis

Durandi, Werner, Baoswan Dzung Wong, Markus Kriener, u. a. *Formeln, Tabellen, Begriffe: Mathematik - Physik - Chemie*. 8., aktualisierte Auflage. Herausgegeben von Verein Schweizerischer Mathematik- und Physiklehrkräfte und Werner Durandi. Zürich: Orell Füssli Verlag, 2022.